The School of Mathematics

THE UNIVERSITY
of EDINBURGH

Efficient uncertainty quantification and risk assessment in partial differential equation models

by

Anastasia Istratuca

Dissertation Presented for the Degree of
MSc in Computational Applied Mathematics

August 2021

Supervised by
Dr Aretha Teckentrup

Abstract

We study the computational efficiency of the Monte Carlo and Multilevel Monte Carlo approaches in the context of elliptic partial differential equations with random coefficients. These equations emerge, for instance, in uncertainty quantification and risk assessment for groundwater flow models. We focus on a specific quantity on interest, namely the point evaluation of the solution, and numerically demonstrate for one- and two-dimensional model problems the effectiveness of the Multilevel technique compared to standard Monte Carlo. We consider coefficients represented as random variables and random fields, and we show that, in both cases, the former method is superior. We employ FEniCS to approximate the solution to the equation model and apply the Karhunen-Loève expansion to generate samples from the random field coefficient. For the latter approach, we adopt both uniform and level dependent truncations of the expansion in order to increase the computational savings prompted by the Multilevel Monte Carlo estimator.

Acknowledgments

Own Work Declaration

I declare that this thesis was composed by myself, that the work contained herein is my own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

Contents

1 Introduction **1**
1.1 Groundwater flow model . . . 1
1.2 Thesis Contribution . . . 2
1.3 Outline of Thesis . . . 2

2 Monte Carlo Methods **3**
2.1 Overview . . . 3
2.2 Simple MC for PDEs with random coefficients . . . 4

3 Multilevel Monte Carlo Methods **7**
3.1 Overview . . . 7
3.2 MLMC for PDEs with random coefficients . . . 9

4 Methodology **14**
4.1 FEniCS . . . 14
4.2 Karhunen-Loève expansions . . . 16

5 Numerical Results **22**
5.1 1D example . . . 22
5.2 2D example with random variable coefficient . . . 25
5.3 2D example with random field coefficient . . . 31
5.3.1 Covariance function with correlation length 0.3 and variance 1 . . . 31
5.3.2 Covariance function with correlation length 0.1 and variance 1 . . . 34

6 Conclusions and Further Work **38**

Appendices **42**

A Monte Carlo implementation for 1D example **42**

B Monte Carlo implementation for 2D example with random variable **44**

C Multilevel Monte Carlo implementation for 2D example with random variable **47**

D Monte Carlo implementation for 2D example with random field **51**

E Multilevel Monte Carlo implementation for 2D example with random field **56**

F Multilevel Monte Carlo implementation for 2D example with level dependent truncation of random field KL-expansion **62**

List of Figures

1 Cross-section of subsurface at the Waste Isolation Pilot Plan site in New Mexico [33]. 2

2 Plot created using ParaView of the mesh and the solution to the PDE (1.1) subject to Dirichlet boundary conditions (4.11) and Neumann conditions (4.12) for an arbitrary sample of the random field $k(\mathbf{x}, \omega)$. The approximation was obtained using FEniCS with a stepsize $h = 1/32$ (2048 triangles), linear polynomials and $f(\mathbf{x}) \equiv 1$. 16

3 Graph of $g(x)$ as defined in Eq. (4.31) for $x \in \left(-\frac{5\pi}{2}, \frac{5\pi}{2}\right)$ and $\rho = 1$. 19

4 Performance plot on a log scale for $\mu = 0$ and $\sigma^2 = 1$ in 1D using Monte Carlo. The quantity of interest is the mean value of the pressure at $x = 3/4$. 24

5 Performance plot on a log scale for $\mu = 0$ and $\sigma^2 = 1$ in 1D using Monte Carlo. The quantity of interest is the mean value of the effective conductivity. 25

6 Plots of α and γ on a log scale for $\mu = 0$ and $\sigma^2 = 1$ in 2D using Monte Carlo. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$. 27

7 Performance plot on a log scale for $\mu = 0$ and $\sigma^2 = 1$ in 2D using Monte Carlo. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$. 28

8 Plots of α, β and γ on a log scale, and of number of samples N_l necessary per level for $\mu = 0$ and $\sigma^2 = 1$ in 2D using Multilevel Monte Carlo with $l = 0, \ldots, 7$. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$. 29

9 Performance plots on a log scale for $\mu = 0$ and $\sigma^2 = 1$ in 2D using Multilevel Monte Carlo and Monte Carlo. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$. 30

10 Plots of α and γ on a log scale for $\rho = 0.3$ and $\sigma^2 = 1$ in 2D using Multilevel Monte Carlo with $l = 0, \ldots, 6$. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$. 32

11 Plots of β on a log scale, and of the number of samples N_l necessary per level for $\rho = 0.3$ and $\sigma^2 = 1$ in 2D using Multilevel Monte Carlo with $l = 0, \ldots, 6$. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$. . . 32

12 Performance plots on a log scale for $\rho = 0.3$ and $\sigma^2 = 1$ in 2D using Monte Carlo and Multilevel Monte Carlo. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$. 33

13 Plot of δ on a log scale and performance plot of uniform versus level dependent truncated KL-expansion of random field coefficient for $\rho = 0.3$ and $\sigma^2 = 1$ in 2D using Multilevel Monte Carlo with $l = 0, \ldots, 6$. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$. 34

14 Plots of α, β and γ on a log scale, and the number of samples N_l necessary per level for $\rho = 0.1$ and $\sigma^2 = 1$ in 2D using Multilevel Monte Carlo with $l = 0, \ldots, 4$. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$. 35

15 Plots of δ and γ on a log scale for $\rho = 0.1$ and $\sigma^2 = 1$ in 2D using Multilevel Monte Carlo with a level dependent truncation of KL-expansion of $k(\mathbf{x}, \omega)$ and $l = 0, \ldots, 7$. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$. 36

16 Performance plots on a log scale for $\rho = 0.1$ and $\sigma^2 = 1$ in 2D using Monte Carlo and Multilevel Monte Carlo with a uniform and level dependent truncation of the KL-expansion of $k(\mathbf{x}, \omega)$. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$. 37

Abbreviations and Acronyms

1D One dimensional

2D Two dimensional

CPU Central Processing Unit

DOLFIN Dynamic Object-oriented Library for FINite element computation

FEniCS Finite Element Computational Software

FLOPS Floating Point Operations

IID Independent and Identically Distributed

KL-expansion Karhunen-Loève expansion

LD Level Dependent

MC Monte Carlo

MLMC Multilevel Monte Carlo

MSE Mean Squared Error

ODE Ordinary Differential Equation

PDE Partial Differential Equation

RAM Random Access Memory

RMSE Root Mean Squared Error

Mathematical Notation

In this thesis, we adhere to the following mathematical notations:

- we write $a \lesssim b$ if a/b is uniformly bounded independent of any parameters, for two positive quantities a and b.
- we write $a \eqsim b$ if $a \lesssim b$ and $b \lesssim a$, for two positive quantities a and b.
- we write $f(x) = \mathcal{O}(g(x))$ if there exist positive real constant M and real value x_0 such that $|f(x)| \leq Mg(x)$, $\forall x \geq x_0$, for two real-valued functions $f, g : \mathbb{R} \to \mathbb{R}$.
- the ceiling function is denoted by $\lceil x \rceil$, that is, the unique integer n satisfying the inequalities $x \leq n < x + 1$.
- deterministic and random vectors are denoted by bold lowercase and uppercase letters, such as $\mathbf{x}$ and $\mathbf{X}$, respectively.
- samples of a random variable are denoted by uppercase letters, for example $X^{(i)}$, with the superscript i representing the ith realisation.

1 Introduction

Partial differential equations (PDEs) describe the relationship between various partial derivatives of a multivariate function. They are pervasive in mathematically-oriented disciplines, and hence extensively used to model physical phenomena, such as electrodynamics [47], fluid [11] and quantum mechanics [24]. Such models, with the aid of computer simulations, often give suitable representations of physical phenomena. However, they generally encompass parameters for which an exact approximation is impractical. Consequently, their estimation from measurements is inevitable, which prompts the following question: given that certain parameters are not known exactly, how can we quantify the impact of the inferred uncertainty on the outcome of the model?

In this report, we adopt a probabilistic approach to answering the prior question. Particularly, we represent the coefficients of the PDE model as random parameters and describe their uncertainty in terms of statistical quantities, such as probability density functions or covariance functions. Subsequently, we determine statistical estimates, such as expected values or variances, related to a quantity of interest which depends on the solution of the PDE, for instance a specific functional. This process is known as uncertainty quantification.

The preferred approach for the problem at hand rests on standard Monte Carlo (MC) simulations. However, these entail sampling from the random coefficients, which, in turn, implies directly sampling from their distribution if they are modelled as random variables, or applying other potentially costly methods if they are modelled, for instance, as random fields. We are then required to solve the resulting PDE, which is often too complex to be accomplished analytically, and it is therefore necessary to adopt a numerical solver. Afterwards, we approximate the quantity of interest in order to produce one MC sample. Since many such samples are required to achieve accurate results, MC estimations are generally expensive in the current context.

A novel approach which aims to address this problem implements a Multilevel Monte Carlo (MLMC) algorithm. Such methods hinge on a multilevel variance reduction technique for the standard MC method [22]. For the problem at hand, this is achieved by first expressing the quantity we wish to estimate on a relatively coarse grid. Thereafter, we successively add to it "correction" terms arising from adjusting the grid. Since most of the uncertainty can be captured on the coarse grid, fewer samples are thus needed on the finest grid to obtain truthful approximations . This contributes to a notable reduction in the computational cost associated with MLMC methods compared to standard MC approaches.

In addition, successful MLMC simulations have been performed in diverse physical applications [2, 1, 4, 7]. In this report, we focus on the work described in [7], namely applying MC and MLMC to quantify uncertainty and assess risk in elliptic PDE models with random coefficients arising from modelling groundwater flow, and evaluating their computational performance.

1.1 Groundwater flow model

PDE models of subsurface flow simulate fluid movement below ground or in other porous media. For instance, an explicit example is the problem of simulating the transport of contaminated substances potentially leaking from deep geological repositories of radioactive waste. On a long timescale, the groundwater flowing beneath earth's surface could carry such dangerous radionuclides back to the human environment. Consequently, accurate mathematical models and computational simulations are fundamental in assessing the efficacy and risk of such repositories.

Many of the models simulating groundwater flow stand on Darcy's Law [8] - the linear relation between pressure difference and flow rate. In [42], Schweizer combines Darcy's Law with the *Law of conservation of mass* to obtain the following partial differential equation:

$$-\nabla \cdot (k(\mathbf{x})\nabla p(\mathbf{x})) = f(\mathbf{x}), \quad \mathbf{x} \in D. \tag{1.1}$$

Here, p is the pressure of the fluid, $f := -\nabla \cdot g$, where g represents the source terms, and k denotes the hydraulic conductivity - that is, the ease with which a fluid can move through porous media or fractures, such as the rocks beneath earth's surface, under a given pressure gradient.

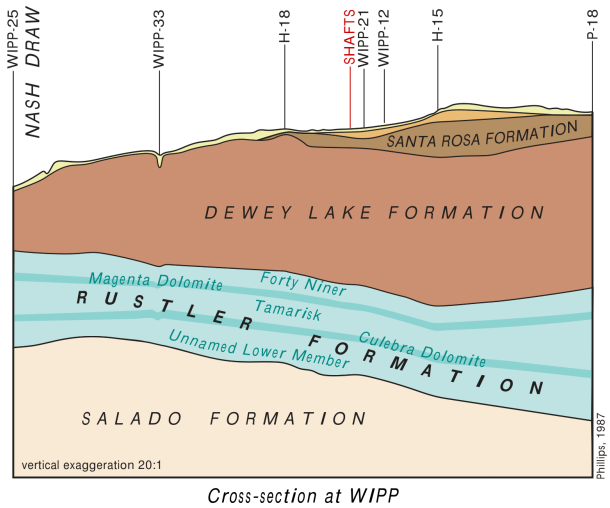


Figure 1: Cross-section of subsurface at the Waste Isolation Pilot Plan site in New Mexico [33].

To justify why $k(\mathbf{x})$ is a function rather than a constant, Figure 1 depicts a model of the subterranean surface at the Waste Isolation Pilot Plan test site in New Mexico [33]. Clearly, the pace of the groundwater flow depends on its depth and range. Hence, for an accurate computer simulation, the value of the hydraulic conductivity k is required at every point in the domain on which the model is defined. For this reason, an immediate obstacle is the fact that k can only be measured at a finite, usually small, number of geographical points.

Accordingly, to account for the inferred uncertainty in the parameter k, we first assume, for simplicity, that k is a random variable, whose distribution is appropriate for the application under consideration. Subsequently, we study the implementation of k as a random field, which is a more common approach to modelling the hydraulic conductivity [31, 9]. The law of the field is typically estimated from the given measurements, which is in fact a considerable challenge on its own, due to the scarce availability of the data and the effort required to collect it.

1.2 Thesis Contribution

This thesis aims to numerically demonstrate theoretical properties already established related to the computational complexity of the Monte Carlo and Multilevel Monte Carlo algorithms in the context of the application introduced above. The main novelty proposed in this report is the analysis of their performance with respect to a specific functional, namely the point-evaluation of the pressure. We also investigate the efficiency of the MLMC method with an integrated level dependent truncation of the Karhunen-Loève expansion of the random field coefficient. Here, we use numerically determined convergence rates, rather than theoretical ones.

1.3 Outline of Thesis

The remainder of this thesis is organised as follows. Chapter 2 offers a brief survey of the Monte Carlo method, first in a general framework, then in the setting of PDEs with random coefficients. In turn, Chapter 3 provides a concise review of the Multilevel Monte Carlo approach, adhering to a structure analogous to the one of the preceding chapter. Moreover, the methodology required for developing the MC and MLMC techniques, namely FEniCS and the Karhunen-Loève expansion of a random field, are introduced in Chapter 4. Finally, we conclude this report with Chapter 5, where we outline the results achieved in the one- and two-dimensional cases of (1.1), with the coefficient k modelled both as a random variable and as a random field.

2 Monte Carlo Methods

In this section, we canvass the standard Monte Carlo theory, largely based on [35]. Such methods are adopted in almost every scientific subject of study: physical sciences, engineering, statistics, finance, and computing, including machine learning and graphics. Monte Carlo is even applied in some areas that are not always considered to be quantitative, such as music theory. For this reason, we first establish a theoretical foundation for using this technique in a general setting, subsequently focusing on its application to ellipitc PDEs with random coefficients.

While throughout this thesis we focus on standard Monte Carlo simulations, they can be replaced by other unbiased estimators, such as Randomized Quasi-Monte Carlo [34]. In addition, these methods have also been successfully applied to elliptic PDEs with random coefficients [19].

2.1 Overview

The essence of Monte Carlo based approaches rests on simulating a system through random sampling in order to study it. In other words, the potential of such methods relies on exploiting randomness to solve challenges that might, in fact, be of deterministic nature. Correspondingly, in a simple Monte Carlo problem, we express the quantity we wish to compute as the expectation of a random variable Q, specifically:

$$\hat{Q} = \mathbb{E}[Q]. \tag{2.1}$$

In order to estimate $\hat{Q}$, we independently and randomly sample $Q^{(1)}, Q^{(2)}, \ldots, Q^{(N)}$ from the distribution of Q, where N, the number of samples, is typically large. The approximate value of $\hat{Q}$ is then given by a simple finite sum, as follows:

$$\hat{Q}_N = \frac{1}{N} \sum_{i=1}^{N} Q^{(i)}. \tag{2.2}$$

Commonly, $Q = f(\mathbf{X})$ is a linear or nonlinear functional of $\mathbf{X}$, where $\mathbf{X}$ is a random variable over an infinite dimensional probability space $(\Omega, \mathcal{F}, \mathbb{P})$ that takes values in $D \subset \mathbb{R}^d$, with probability density function $p(\mathbf{x})$. In fact, $\mathbf{X}$ can represent anything from a point in the Euclidean space to a path taken by a particle or an image. In addition, $f : D \to \mathbb{R}$, so that Q denotes a scalar-valued random variable, particularly $Q : \Omega \to \mathbb{R}$. Hence, by definition, it follows that:

$$\hat{Q} = \mathbb{E}[f(\mathbf{X})] = \int_D f(\mathbf{x}) p(\mathbf{x}) d\mathbf{x}. \tag{2.3}$$

However, the integral in the above equation is more often than not rather challenging to compute analytically. In contrast, we can easily implement Monte Carlo to estimate $\hat{Q}$, as long as $Q = f(\mathbf{X})$ is a quantity that can be averaged.

Under the assumption that $\hat{Q}$ exists and that $\{Q^{(i)}\}_{i=1}^N$ are independent and identically distributed (IID) with the same distribution as Q, the *strong law of large numbers* implies that:

$$\mathbb{P}\left(\lim_{N \to \infty} |\hat{Q}_N - \hat{Q}| = 0 \right) = 1. \tag{2.4}$$

That is to say, the estimator $\hat{Q}_N$ converges to the quantity of interest $\hat{Q}$ almost surely. Stronger or weaker statements can be made, however they are beyond the purpose of this thesis [39].

While Eq. (2.4) suggests that the Monte Carlo approximation will eventually result in a sufficiently small error between $\hat{Q}_N$ and $\hat{Q}$, it does not indicate how many samples of Q are required in order to achieve this. What is more, for a given random sample $Q^{(i)}$, the statement does not disclose any information about the error.

Accordingly, we shall now consider the estimator $\hat{Q}_N$ more closely. As shown in [27], $\hat{Q}_N$ is itself a random variable, since it is defined as a finite sum of IID random variables. Therefore, the estimator has its individual expectation and variance, which can, in fact, be derived in terms of the mean value $\mathbb{E}[Q]$.

Specifically, the expectation of the estimator is given by:

$$\mathbb{E}[\hat{Q}_N] = \frac{1}{N}\sum_{i=1}^{N}\mathbb{E}[Q^{(i)}] \overset{\text{IID}}{=} \hat{Q}. \tag{2.5}$$

In other words, Eq. (2.5) suggests that simple Monte Carlo is unbiased. Additionally, since $\{Q^{(i)}\}_{i=1}^{N}$ are independent, its variance can be derived similarly, as follows:

$$\mathbb{V}[\hat{Q}_N] = \mathbb{V}\left[\frac{1}{N}\sum_{i=1}^{N}Q^{(i)}\right] = \sum_{i=1}^{N}\frac{1}{N^2}\mathbb{V}[Q^{(i)}] = \frac{1}{N}\left(\frac{1}{N}\sum_{i=1}^{N}\mathbb{V}[Q^{(i)}]\right) \overset{\text{IID}}{=} \frac{1}{N}\mathbb{V}[Q] = \frac{\sigma^2}{N}, \tag{2.6}$$

assuming that $\mathbb{V}[Q] = \sigma^2 < \infty$. In particular, Eq. (2.6) illustrates that as the number of samples of Q increases, the variance of the estimator decreases as $1/N$.

To emphasise the significance of Eq. (2.6), consider the root mean squared error (RMSE), defined as follows:

$$e(\hat{Q}_N) := \left(\mathbb{E}[(\hat{Q}_N - \hat{Q})^2]\right)^{\frac{1}{2}}. \tag{2.7}$$

This can be further manipulated using Eq. (2.6) to obtain:

$$e(\hat{Q}_N) = \left(\mathbb{E}[(\hat{Q}_N - \mathbb{E}[\hat{Q}_N])^2]\right)^{\frac{1}{2}} = \sqrt{\mathbb{V}[\hat{Q}_N]} = \frac{\sigma}{\sqrt{N}}, \tag{2.8}$$

or, in other words:

$$e(\hat{Q}_N) = \mathcal{O}(N^{-1/2}). \tag{2.9}$$

The computational ϵ-cost $C_\epsilon(\hat{Q}_N)$ is then quantified by the number of floating point operations (FLOPS) required in order to satisfy $e(\hat{Q}_N) < \epsilon$.

Furthermore, to illustrate the effect of the relation above, suppose, for instance, that we wish to decrease the RMSE by a factor of 10. Then Eq. (2.9) suggests that this requires a 100-fold increase in computation. Moreover, one million times as much computation is required for three more digits of accuracy. As a consequence, standard Monte Carlo methods are undoubtedly ill suited for problems which postulate high accuracy solutions.

Nonetheless, Monte Carlo simulations are extremely reliable compared to alternatives [35]. Yet, it must be highlighted that they sometimes fail to produce an accurate approximation. In order to safely apply such approaches, we should first ensure that the quantity we wish to approximate, $\hat{Q}$, exists and is finite. In other words, we must have $-\infty < \hat{Q} = \mathbb{E}[Q] < \infty$. When $\mathbb{E}[\,|Q|\,] = \infty$, then it is likely that $\mathbb{E}[Q] = \infty$, $\mathbb{E}[Q] = -\infty$, or that $\mathbb{E}[\max(Q,0)] = \mathbb{E}[\max(-Q,0)] = \infty$, in which case $\mathbb{E}[Q]$ is not even an element of $\{\pm\infty\}$.

2.2 Simple MC for PDEs with random coefficients

This section largely follows [46]. Respectively, $\mathbf{X}$, as defined at the beginning of Section 2.1, now represents the solution to the PDE model under consideration, and the quantity of interest $Q = f(\mathbf{X})$ is a functional of the corresponding PDE solution. In addition, $\mathbf{X}$ and Q are generally inaccessible random variables since they cannot typically be computed analytically. Accordingly, let $\mathbf{X}_M$ denote the numerical approximation of the solution $\mathbf{X}$, and Q_M the functional Q of the discretised solution, where $M = h^{-d}$ stands for the number of degrees of freedom. Here, d is the dimension of the domain on which the PDE is defined and h is the stepsize used in the discretisation of the spatial domain. As a result, adopting a sufficiently large M is equivalent to choosing a fine enough spatial approximation.

Furthermore, we assume that:

$$\mathbb{E}[Q_M] \to \mathbb{E}[Q] \text{ as } M \to \infty, \tag{2.10}$$

with order of convergence (in mean) α, which is problem dependent, so that:

$$|\mathbb{E}[Q_M - Q]| \lesssim M^{-\alpha}. \tag{2.11}$$

The standard Monte Carlo estimator for $\mathbb{E}[Q_M]$ is then computed as in Eq. (2.2):

$$\widehat{Q}^{\mathrm{MC}}_{M,N} := \frac{1}{N}\sum_{i=1}^{N} Q_M^{(i)}, \tag{2.12}$$

where $Q_M^{(i)}$ is the i-th sample of Q_M and N is the total number of independent samples computed.

Afterwards, the error between the Monte Carlo estimator $\widehat{Q}^{\mathrm{MC}}_{M,N}$ and the exact value $\mathbb{E}[Q]$ is quantified using the RMSE, as defined in Eq. (2.7). To emphasise the effect of replacing Q with its approximation Q_M, let us instead expand the mean squared error (MSE), as follows:

$$\begin{aligned}
e\left(\widehat{Q}^{\mathrm{MC}}_{M,N}\right)^2 &\overset{\mathrm{def}}{=} \mathbb{E}\left[\left(\widehat{Q}^{\mathrm{MC}}_{M,N} - \mathbb{E}[Q]\right)^2\right] \\
&= \mathbb{E}\left[\left(\widehat{Q}^{\mathrm{MC}}_{M,N} - \mathbb{E}\left[\widehat{Q}^{\mathrm{MC}}_{M,N}\right] + \mathbb{E}\left[\widehat{Q}^{\mathrm{MC}}_{M,N}\right] - \mathbb{E}[Q]\right)^2\right] \\
&= \mathbb{E}\left[\left(\widehat{Q}^{\mathrm{MC}}_{M,N} - \mathbb{E}\left[\widehat{Q}^{\mathrm{MC}}_{M,N}\right]\right)^2\right] + \left(\mathbb{E}\left[\widehat{Q}^{\mathrm{MC}}_{M,N}\right] - \mathbb{E}[Q]\right)^2 \\
&= \mathbb{V}\left[\widehat{Q}^{\mathrm{MC}}_{M,N}\right] + \left(\mathbb{E}\left[\widehat{Q}^{\mathrm{MC}}_{M,N}\right] - \mathbb{E}[Q]\right)^2 \\
&= \frac{1}{N}\mathbb{V}[Q_M] + \left(\mathbb{E}[Q_M - Q]\right)^2.
\end{aligned} \tag{2.13}$$

The last equation follows from Eq. (2.5) and (2.6), which illustrate that $\mathbb{V}\left[\widehat{Q}^{\mathrm{MC}}_{M,N}\right] = N^{-1}\mathbb{V}[Q_M]$ and $\mathbb{E}\left[\widehat{Q}^{\mathrm{MC}}_{M,N}\right] = \mathbb{E}[Q_M]$. In addition, the link between the second and third equations in (2.13) follows from simply expanding the square, where the product term vanishes:

$$\begin{aligned}
&\mathbb{E}\left[2\left(\widehat{Q}^{\mathrm{MC}}_{M,N} - \mathbb{E}\left[\widehat{Q}^{\mathrm{MC}}_{M,N}\right]\right)\left(\mathbb{E}\left[\widehat{Q}^{\mathrm{MC}}_{M,N}\right] - \mathbb{E}[Q]\right)\right] \\
&= 2\,\mathbb{E}\left[\widehat{Q}^{\mathrm{MC}}_{M,N}\,\mathbb{E}\left[\widehat{Q}^{\mathrm{MC}}_{M,N}\right] - \widehat{Q}^{\mathrm{MC}}_{M,N}\,\mathbb{E}[Q] - \left(\mathbb{E}\left[\widehat{Q}^{\mathrm{MC}}_{M,N}\right]\right)^2 + \mathbb{E}\left[\widehat{Q}^{\mathrm{MC}}_{M,N}\right]\mathbb{E}[Q]\right] \\
&= 2\left(\left(\mathbb{E}\left[\widehat{Q}^{\mathrm{MC}}_{M,N}\right]\right)^2 - \mathbb{E}\left[\widehat{Q}^{\mathrm{MC}}_{M,N}\right]\mathbb{E}[Q] - \left(\mathbb{E}\left[\widehat{Q}^{\mathrm{MC}}_{M,N}\right]\right)^2 + \mathbb{E}\left[\widehat{Q}^{\mathrm{MC}}_{M,N}\right]\mathbb{E}[Q]\right) \\
&= 0
\end{aligned} \tag{2.14}$$

Correspondingly, Eq. (2.13) delineates the two terms that contribute to the error of the estimator $\widehat{Q}^{\mathrm{MC}}_{M,N}$. On the one hand, the first term represents the sampling error, given by the variance of the MC estimator. As also indicated in Eq. (2.8), this decays linearly as the number of samples increases. On the other hand, the second term describes the error introduced by replacing Q with its approximation Q_M, and arises from the discretisation of the spatial domain.

In addition to illustrating the contribution of the two sources of error in the MC estimator, Eq. (2.13) also provides a mechanism for finding the number of samples N and of spatial degrees of freedom M such that this estimator achieves a RMSE of ϵ. Respectively, a sufficient condition to satisfy $e\left(\widehat{Q}^{\mathrm{MC}}_{M,N}\right) < \epsilon$ is that each term in Eq. (2.13) is less than $\epsilon^2/2$, which can, in turn, be accomplished as follows:

$$N > 2\,\mathbb{V}[Q_M]\,\epsilon^{-2} \implies \frac{1}{N}\mathbb{V}[Q_M] < \frac{\epsilon^2}{2}, \tag{2.15}$$

and

$$M > (\sqrt{2}C)^{1/\alpha}\epsilon^{-1/\alpha} \implies (CM^{-\alpha})^2 < \frac{\epsilon^2}{2} \overset{\text{Eq. (2.11)}}{\implies} (\mathbb{E}[Q_M - Q])^2 < \frac{\epsilon^2}{2}. \tag{2.16}$$

Next, to ensure N and M are integers, we can simply choose $N = \lceil 2\mathbb{V}[Q_M]\epsilon^{-2}\rceil$ and $M = \lceil(\sqrt{2}C)^{1/\alpha}\epsilon^{-1/\alpha}\rceil$.

Finally, let us assume that the cost of computing one sample $Q_M^{(i)}$ of Q_M satisfies:

$$C\left(Q_M^{(i)}\right) \lesssim M^{\gamma}, \tag{2.17}$$

for some $\gamma > 0$. Since we require N samples in order to calculate the estimator (2.12), it is straightforward to recognise that the cost of computing $\widehat{Q}_{M,N}^{\text{MC}}$ is:

$$C\left(\widehat{Q}_{M,N}^{\text{MC}}\right) \lesssim NM^{\gamma}. \tag{2.18}$$

Further, let us assume that $\mathbb{V}[Q_M]$ is approximately constant independent of M. As $N \gtrsim \epsilon^{-2}$ and $M \gtrsim \epsilon^{-1/\alpha}$ from Eq. (2.15) and (2.16), respectively, the total computational cost of achieving a RMSE less than ϵ, as defined in Section 2.1, is given by:

$$C_\epsilon\left(\widehat{Q}_{M,N}^{\text{MC}}\right) \lesssim (\epsilon^{-2})(\epsilon^{-1/\alpha})^{\gamma} \implies C_\epsilon\left(\widehat{Q}_{M,N}^{\text{MC}}\right) \lesssim \epsilon^{-2-\gamma/\alpha}. \tag{2.19}$$

To emphasise the significance of Eq. (2.19), consider the best case scenario in which the exact solution $\mathbf{X}$ to the PDE is known. Clearly, the discretisation error is 0 in this case, and the only contribution to the cost originates from choosing a sufficiently large N. In other words:

$$C_\epsilon\left(\widehat{Q}_{M,N}^{\text{MC}}\right) \lesssim \epsilon^{-2}. \tag{2.20}$$

However, this is rarely the case. In fact, as shown in Section 5, the addition of the $\epsilon^{-\gamma/\alpha}$ term to $C_\epsilon\left(\widehat{Q}_{M,N}^{\text{MC}}\right)$ prompts, more often than not, a dramatic increase in the computational cost. Hence, for the application under consideration, to achieve a satisfactory accuracy using Monte Carlo based approaches, minimising the cost is pivotal. In fact, this can be achieved by exploiting the Multilevel Monte Carlo technique, which is the topic of the next section.

3 Multilevel Monte Carlo Methods

This section introduces the MLMC method as an alternative algorithm to computing the quantity of interest $\mathbb{E}[Q]$. The aforementioned procedure is an extension of the Control Variates approach [36], which is a widely used variance reduction technique associated with MC methods. As well as retaining all the benefits related to the standard MC simulations, this routine also remarkably diminishes the computational cost required to achieve a fixed accuracy.

Introduced by Giles in [16], MLMC techniques are now being used in a variety of applications, such as pricing discrete time Bermudan options [5], approximating distribution functions [50], or simulating Coulomb collisions [40]. Here, we focus on quantifying uncertainty in elliptic PDEs with random coefficients, which is the subject of Section 3.2.

3.1 Overview

Building on [18], the underlying intuition for MLMC approaches is to cluster most of the computational effort in economical estimations on coarse levels with low accuracy. Afterwards, comparatively fewer expensive calculations are performed on finer levels with progressively higher precision, in order to evaluate samples which constitute "correction terms".

As aforementioned, MLMC methods hinge on the Control Variates technique. In particular, suppose we are interested in estimating the mean value of a quantity of interest Q, as in Eq. (2.1). Additionally, suppose there is another random variable Y, called the control variate, which is positively correlated to Q, and whose expected value $\mathbb{E}[Y]$ is known. Next, due to the linearity of expectation, we can express the mean value $\mathbb{E}[Q]$ in terms of $\mathbb{E}[Y]$, as follows:

$$\mathbb{E}[Q] = \mathbb{E}[Y] + \mathbb{E}[Q - Y]. \tag{3.1}$$

Next, based on the control variate Y and using N IID samples, we can define the following estimator for $\mathbb{E}[Q]$:

$$\hat{Q}_N^{\mathrm{CV}} = \lambda \mathbb{E}[Y] + \frac{1}{N}\sum_{i=1}^{N}\left(Q^{(i)} - \lambda Y^{(i)}\right) = \frac{1}{N}\sum_{i=1}^{N}\left(Q^{(i)} - \lambda\left(Y^{(i)} - \mathbb{E}[Y]\right)\right), \tag{3.2}$$

where the optimal value for λ is $\rho\sqrt{\mathbb{V}[Q]/\mathbb{V}[Y]}$ [18], with ρ denoting the correlation between Q and Y. What is more, due to the linearity of expectation, the estimator (3.2) is unbiased, since:

$$\mathbb{E}\left[\hat{Q}_N^{\mathrm{CV}}\right] = \frac{1}{N}\sum_{i=1}^{N}\left(\mathbb{E}[Q^{(i)}] - \lambda\left(\mathbb{E}[Y^{(i)}] - \mathbb{E}[Y]\right)\right) = \mathbb{E}[Q]. \tag{3.3}$$

Further, assuming the pairs $\{Q^{(i)}, Y^{(i)}\}_{i=1}^{N}$ are independent for each $i = 1, \ldots, N$, the variance of the estimator (3.2) is given by:

$$\mathbb{V}\left[\hat{Q}_N^{\mathrm{CV}}\right] = \frac{1}{N^2}\sum_{i=1}^{N}\left(\mathbb{V}[Q^{(i)}] - \lambda^2\mathbb{V}[Y^{(i)}]\right) = \frac{1}{N}\mathbb{V}[Q - \lambda Y]. \tag{3.4}$$

With the optimal choice for $\lambda = \rho\sqrt{\mathbb{V}[Q]/\mathbb{V}[Y]}$, we next have:

$$\begin{aligned}\mathbb{V}\left[\hat{Q}_N^{\mathrm{CV}}\right] &= \frac{1}{N}\mathbb{V}[Q] - \frac{1}{N}\lambda^2\mathbb{V}[Y] = \frac{1}{N}\mathbb{V}[Q] - \frac{1}{N}\rho^2\frac{\mathbb{V}[Q]}{\mathbb{V}[Y]}\mathbb{V}[Y] \\ &= \frac{1}{N}\mathbb{V}[Q] - \frac{1}{N}\rho^2\mathbb{V}[Q] = \frac{1}{N}\mathbb{V}[Q](1-\rho^2).\end{aligned} \tag{3.5}$$

In other words, the Control Variates procedure leads to a decline in the variance of the estimator (3.2) by a factor of $1 - \rho^2$ compared to the standard MC estimator (2.2).

Building on this, suppose wish to estimate $\mathbb{E}[Q_1] := \mathbb{E}[Q]$ and that there exists some approx-

imation Q_0 of Q_1, which can be simulated with a much lower cost. Then, similarly as above, the mean value of the quantity of interest can be expressed in terms of $\mathbb{E}[Q_0]$, as follows:

$$\mathbb{E}[Q_1] = \mathbb{E}[Q_0] + \mathbb{E}[Q_1 - Q_0]. \tag{3.6}$$

Applying simple MC estimate to each quantity on the right-hand side above with N_0 and N_1 IID samples, respectively, we obtain the two-level version of MLMC. Specifically:

$$\hat{Q}_1^{\text{MLMC}} = \frac{1}{N_0}\sum_{i=1}^{N_0} Q_0^{(i)} + \frac{1}{N_1}\sum_{i=1}^{N_1}\left(Q_1^{(i)} - Q_0^{(i)}\right). \tag{3.7}$$

Hence, here we avoid approximating $\mathbb{E}[Q_1]$ directly. Instead, we estimate the expectation of the quantity of interest on the cheaper level $l = 0$, to which we add the expected value of the correction term $Q_1 - Q_0$. In other words, the random variable Q_0 acts a control variate for Q_1.

There are a few key observations to be noted here. First, we are using standard MC to estimate each quantity, hence the estimator (3.7) is unbiased. Second, in order for the difference $Q_1^{(i)} - Q_0^{(i)}$ to be small and have a small variance, it must computed using the same underlying sample $\omega^{(i)} \in \Omega$, whose construction depends on the application; a specific example is given in Section 3.2. Moreover, in feasible models, the approximation on level $l = 0$ must be chosen sufficiently accurate to provide a basic level of detail on the problem; the exact value depends, again, on the question. Finally, the expectation $\mathbb{E}[Q_0]$ is not known and must be estimated, unlike the mean value of the control variate Y above. Consequently, the optimal choice of λ cannot be established exactly, and it is instead chosen to be equal to one [18].

Once we have defined the two-level estimator (3.7), the multilevel generalisation follows instinctively. To this end, suppose we wish to estimate $\mathbb{E}[Q_L] := \mathbb{E}[Q]$ and that there exists a sequence $Q_0, \ldots, Q_{L-1}$ which approximates Q_L increasingly more accurately, but also with a greater computational cost. Applying a simple telescopic sum and using the linearity of the expectation operator, we obtain the following identity:

$$\mathbb{E}[Q_L] = \mathbb{E}[Q_0] + \sum_{l=1}^{L}\mathbb{E}[Q_l - Q_{l-1}]. \tag{3.8}$$

Similar to the two-level estimator (3.7), we implement a simple MC estimate for each of the terms on the RHS in the equation above, yielding the MLMC estimator of $\mathbb{E}[Q_L]$:

$$\hat{Q}_L^{\text{MLMC}} = \frac{1}{N_0}\sum_{i=1}^{N_0} Q_0^{(i)} + \sum_{l=1}^{L}\left(\frac{1}{N_l}\sum_{i=1}^{N_l}\left(Q_l^{(i)} - Q_{l-1}^{(i)}\right)\right). \tag{3.9}$$

Here, N_l denotes the number of IID samples on each level $l = 0, \ldots, L$, and, again, the difference $Q_l^{(i)} - Q_{l-1}^{(i)}$ is computed using the same underlying sample $\omega^{(i)} \in \Omega$.

While throughout this thesis we use standard MC to estimate the terms on each level in the MLMC estimator (3.9), similar to Section 2, this could be replaced by a different unbiased estimator, such as Randomized Quasi-Monte Carlo [13].

Additionally, the multilevel estimator (3.9) is also unbiased, fact which is a direct consequence of the linearity of expectation:

$$\begin{aligned}
\mathbb{E}\left[\hat{Q}_L^{\text{MLMC}}\right] &= \frac{1}{N_0}\sum_{i=1}^{N_0}\mathbb{E}\left[Q_0^{(i)}\right] + \sum_{l=1}^{L}\left(\frac{1}{N_l}\sum_{i=1}^{N_l}\left(\mathbb{E}\left[Q_l^{(i)}\right] - \mathbb{E}\left[Q_{l-1}^{(i)}\right]\right)\right) \\
&= \mathbb{E}[Q_0] + \sum_{l=1}^{L}\left(\mathbb{E}[Q_l] - \mathbb{E}[Q_{l-1}]\right) \\
&= \mathbb{E}[Q_L].
\end{aligned} \tag{3.10}$$

Further, since all the samples $\{Q_l^{(i)} - Q_{l-1}^{(i)}\}_{i=1}^{N_l}$ are independent for each $i = 1, \ldots, N_l$ and each N_l, $l = 0, \ldots, L$, we note that the variance of the MLMC estimator (3.9) is given by:

$$\begin{aligned}\mathbb{V}\left[\hat{Q}_L^{\mathrm{MLMC}}\right] &= \frac{1}{(N_0)^2}\sum_{i=1}^{N_0}\mathbb{V}\left[Q_0^{(i)}\right] + \sum_{l=1}^{L}\left(\frac{1}{(N_l)^2}\sum_{i=1}^{N_l}\mathbb{V}\left[Q_l^{(i)} - Q_{l-1}^{(i)}\right]\right) \\ &= \frac{1}{N_0}\mathbb{V}[Q_0] + \sum_{l=1}^{L}\frac{1}{N_l}\mathbb{V}[Q_l - Q_{l-1}]. \end{aligned} \tag{3.11}$$

Intuitively, the closer the approximations $\{Q_i\}_{i=1}^{L-1}$ are to Q_L, the smaller the variance $\mathbb{V}[Q_l - Q_{l-1}]$ becomes. Thus, we can expect the number of samples N_l to decrease as l increases.

Moreover, in accordance with Eq. (2.8), the RMSE between the MLMC estimator (3.9) and the quantity we wish to estimate $\mathbb{E}[Q_L] = \mathbb{E}[Q]$, assuming Q_L is not itself an approximation of Q, can be written as follows:

$$\begin{aligned} e\left(\hat{Q}_L^{\mathrm{MLMC}}\right) &:= \left(\mathbb{E}\left[\left(\hat{Q}_L^{\mathrm{MLMC}} - \mathbb{E}[Q]\right)^2\right]\right)^{\frac{1}{2}} \\ &= \left(\mathbb{E}\left[\left(\hat{Q}_L^{\mathrm{MLMC}} - \mathbb{E}\left[\hat{Q}_L^{\mathrm{MLMC}}\right]\right)^2\right]\right)^{\frac{1}{2}} \\ &= \sqrt{\mathbb{V}\left[\hat{Q}_L^{\mathrm{MLMC}}\right]} \\ &= \sqrt{\frac{1}{N_0}\mathbb{V}[Q_0] + \sum_{l=1}^{L}\frac{1}{N_l}\mathbb{V}[Q_l - Q_{l-1}]}. \end{aligned} \tag{3.12}$$

Unlike in the standard MC case, we cannot immediately infer the precise order of convergence of the RMSE in terms of N_l, $l = 0, \ldots, L$. Nonetheless, it is still possible to provide a bound for the computational complexity $C_\epsilon\left(\hat{Q}_L^{\mathrm{MLMC}}\right)$ of the MLMC estimator. This is known in the literature as the Complexity Theorem, and it comes in marginally different forms ([16, 7, 18, 17]). Here, we focus on Theorem 1 in [7], which is given in the context of, though not limited to, the cost of MLMC estimators of functionals of the solution to a PDE with random coefficients, which is the issue explored in the next section.

3.2 MLMC for PDEs with random coefficients

Conforming to [7], let us reconsider the framework at the beginning of Section 2.2. In addition, let $\{M_l : l = 0, \ldots, L\}$ define the MLMC levels, so that $M_0 < M_1 < \ldots < M_L := M$, and suppose that there exists an integer $s \in \mathbb{N} \setminus \{1\}$ such that:

$$M_l = s\, M_{l-1}, \quad \forall l = 1, \ldots, L. \tag{3.13}$$

For simplicity, let $Y_l := Q_{M_l} - Q_{M_{l-1}}$, with $Y_0 := Q_{M_0}$, and let $\hat{Y}_{l,N_l}^{\mathrm{MC}}$ denote the standard MC approximation of $\mathbb{E}[Y_l]$, $l = 0, \ldots, L$. With these new definitions, from Eq. (3.9), the MLMC estimator can be re-written as:

$$\hat{Q}_M^{\mathrm{MLMC}} = \sum_{l=0}^{L}\hat{Y}_{l,N_l}^{\mathrm{MC}}. \tag{3.14}$$

As mentioned in Section 3.1, when computing the MC approximation of Y_l, it is essential that the quantity $Q_{M_l}^{(i)} - Q_{M_{l-1}}^{(i)}$ in (3.9) is approximated using the same underlying sample $\omega^{(i)} \in \Omega$ on both levels l and $l-1$. In the current context, $\omega^{(i)}$ represents a sample of the random coefficients which occur in the definition of the PDE to be solved.

What is more, unlike in Section 3.1, Q_M is now itself an estimate of the quantity of interest

Q. Consequently, expanding the MSE between Q_M and Q as in (2.13) yields:

$$\begin{aligned} e\left(\hat{Q}_M^{\text{MLMC}}\right) &:= \mathbb{E}\left[\left(\hat{Q}_L^{\text{MLMC}} - \mathbb{E}[Q]\right)^2\right] \\ &= \mathbb{V}\left[\hat{Q}_M^{\text{MLMC}}\right] + \left(\mathbb{E}[Q_M - Q]\right)^2 \\ &= \sum_{l=0}^{L} \frac{1}{N_l}\mathbb{V}[Y_l] + \left(\mathbb{E}[Q_M - Q]\right)^2 \end{aligned} \tag{3.15}$$

Therefore, similar to the MSE analysis of the standard MC approximation, Eq. (3.15) highlights the two contributions to the error of the MLMC estimator: its sample variance and bias. Thus, a sufficient condition to achieve an overall RMSE of ϵ is that each term in Eq. (3.15) is less than $\epsilon^2/2$. Since the approximation error is identical to the one in (2.13), $M = M_L$ can be chosen as in Eq. (2.16), that is $M \gtrsim \epsilon^{1/\alpha}$.

Further, let $C_l := C\left(Y_l^{(i)}\right)$ and $V_l := \mathbb{V}[Y_l]$ denote the cost and variance of one individual sample Y_l, respectively. Then the computational cost of the MLMC estimator is given by:

$$C\left(\hat{Q}_M^{\text{MLMC}}\right) = \sum_{l=0}^{L} N_l C_l. \tag{3.16}$$

Using Eq. (3.16), the optimal number of samples N_l for each level $l = 0, \ldots, L$ can be determined by solving an optimisation problem. Considering the N_l to be continuous variables, this can be formulated as follows:

$$\begin{aligned} \min_{N_l} \quad & \sum_{l=0}^{L} N_l C_l \\ \text{s.t.} \quad & \sum_{l=0}^{L} \frac{1}{N_l} V_l = \frac{\epsilon^2}{2}. \end{aligned} \tag{3.17}$$

In other words, the objective of (3.17) is to minimise the number of samples N_l, $l = 0, \ldots, L$, and consequently the variance of the estimator $Q_M^{\hat{\text{MLMC}}}$, for a fixed computational cost. Additionally, the constraint ensures that the overall variance $\mathbb{V}\left[Q_M^{\hat{\text{MLMC}}}\right]$ is $\epsilon^2/2$, as required by Eq. (3.15).

A transparent way to solve the optimisation problem (3.17) is by applying the *method of Lagrange multipliers*. Accordingly, let us denote the Lagrange multiplier corresponding to the constraint by λ, and formulate the Lagrangian function as follows:

$$\mathcal{L}\left(\{N_l\}_{l=0}^{L}, \lambda\right) = \sum_{l=0}^{L} N_l C_l + \lambda\left(\sum_{l=0}^{L} \frac{1}{N_l} V_l - \frac{\epsilon^2}{2}\right). \tag{3.18}$$

Further, we compute the derivative of $\mathcal{L}\left(\{N_l\}_{l=0}^{L}, \lambda\right)$ with respect to each of its arguments and equate all resulting quantities to zero. Thus, the derivative of $\mathcal{L}$ with respect to N_l yields:

$$\frac{\partial \mathcal{L}}{\partial N_l} = C_l - \lambda V_l \frac{1}{N_l^2} = 0. \tag{3.19}$$

Hence, Eq. (3.19) indicates that the optimal number of samples N_l is given by:

$$N_l = \sqrt{\lambda}\sqrt{\frac{V_l}{C_l}}, \quad l = 0, \ldots, L. \tag{3.20}$$

In addition, to find the Lagrange multiplier λ, we can repeat the same procedure as above for the second argument of the Lagrangian function. Since, by definition, this reinstates the

constraint, we obtain the following:

$$\sum_{l=0}^{L} N_l^{-1} V_l = \frac{\epsilon^2}{2} \implies \sum_{l=0}^{L} \left(\sqrt{\lambda}\sqrt{V_l C_l^{-1}}\right)^{-1} V_l = \frac{\epsilon^2}{2} \implies \frac{1}{\sqrt{\lambda}} \sum_{l=0}^{L} \sqrt{C_l V_l} = \frac{\epsilon^2}{2}. \tag{3.21}$$

Therefore, we have that the Lagrangian multiplier is given by:

$$\sqrt{\lambda} = 2\epsilon^{-2} \sum_{l=0}^{L} \sqrt{C_l V_l} \tag{3.22}$$

Substituting for λ in Eq. (3.20) yields the following explicit expression for N_l:

$$N_l = 2\epsilon^{-2} \left(\sum_{l=0}^{L} \sqrt{C_l V_l}\right) \sqrt{\frac{V_l}{C_l}}, \quad l = 0, \ldots, L. \tag{3.23}$$

Finally, substituting for N_l in Eq. (3.16), we have that the computational cost for achieving an overall variance of $\epsilon^2/2$ can be formulated as:

$$\begin{aligned} C_\epsilon\left(\hat{Q}_M^{\mathrm{MLMC}}\right) &= \sum_{l=0}^{L} 2\epsilon^{-2} \left(\sum_{l=0}^{L} \sqrt{C_l V_l}\right) \sqrt{\frac{V_l}{C_l}} C_l \\ &= 2\epsilon^{-2} \left(\sum_{l=0}^{L} \sqrt{C_l V_l}\right)^2. \end{aligned} \tag{3.24}$$

Once we have derived an expression for the ϵ-cost $C_\epsilon\left(\hat{Q}_M^{\mathrm{MLMC}}\right)$, we proceed to establishing a number of intuitive arguments for the computational efficiency of MLMC methods compared to simple MC approaches. Firstly, as mentioned in Section 3.1, a minimum value for M_0 is required in order to provide an elementary level of detail on the problem. In fact, for the application at hand, this relates to the spatial regularity of the PDE solution. Despite this, once chosen, the coarsest level $l = 0$ need not be modified, in which case C_0 does not grow as $\epsilon \to 0$.

Secondly, recall hypothesis (2.10), which, here, can be re-written as:

$$\mathbb{E}[|Q_{M_l} - Q|] \to 0 \text{ as } l \to \infty. \tag{3.25}$$

In addition, let us assume that Q_{M_l} converges to Q in mean square, that is:

$$\mathbb{E}[|Q_{M_l} - Q|^2] \to 0 \text{ as } l \to \infty. \tag{3.26}$$

Combining assumptions (3.25) and (3.26), we obtain the following:

$$\mathbb{V}[Q_{M_l} - Q] \coloneqq \mathbb{E}\left[(Q_{M_l} - Q)^2\right] + (\mathbb{E}[Q_{M_l} - Q])^2 \to 0 \text{ as } l \to \infty. \tag{3.27}$$

Hence, assuming independence of $\{Q_{M_l} - Q\}_{l=0}^{L}$, we have:

$$\mathbb{V}[Y_l] \coloneqq \mathbb{V}[Q_{M_l} - Q_{M_{l-1}}] = \mathbb{V}[Q_{M_l} - Q] + \mathbb{V}[Q - Q_{M_{l-1}}] \to 0 \text{ as } l \to \infty. \tag{3.28}$$

Thus, it is possible to select $N_l \to 1$ as $l \to \infty$. In other words, a diminished number of samples is indeed required on finer levels to estimate $\mathbb{E}[Y_l]$.

Lastly, let us consider Eq. (3.24) again. In particular, an important remark is that the computational complexity of the MLMC estimator (3.14) hinges on whether the product $C_l\, V_l$ increases or decreases with l, that is, whether or not the variance V_l decays more rapidly than the cost C_l grows. As a result, let us analyse the following three scenarios:

- If the variance V_l declines faster with level l, the prominent contribution to the cost originates from the $C_0 V_0$ term, that is $C_\epsilon \approx \epsilon^{-2}\, C_0\, V_0$. To illustrate the cost savings in

comparison to the simple MC estimator, recall Eq. (2.18). Substituting for $N\,\lceil 2\mathbb{V}[Q_M]\epsilon^{-2}\rceil$ in (2.18), we obtain $C\left(\hat{Q}^{\mathrm{MC}}_{M,N}\right) \approx \mathbb{V}[Q_M]\epsilon^{-2}M_L^{\gamma}$. Assuming that the cost of computing Q_{M_L} is similar to that of approximating $Q_{M_L} - Q_{M_{L-1}}$, and that $\mathbb{V}[Q_{M_L}] \approx \mathbb{V}[Q_{M_0}]$, the simple MC cost is given by $\epsilon^{-2}C_L\,V_0$. Consequently, in contrast to the standard MC simulation, the cost savings here are approximately $C_0/C_L \simeq (M_0/M_L)^{\gamma} \simeq \epsilon^{\gamma/\alpha}$. In other words, the MLMC computational complexity is reduced by the ratio of the cost of estimating samples of Q_0 to that of generating samples of $Q_{M_L} - Q_{M_{L-1}}$.

- If, on the other hand, the cost C_l increases faster than the variance V_l decays, the primary contribution to the cost stems from the finest level L, so that $C_\epsilon \approx \epsilon^{-2}\,C_L\,V_L$. Hence, the cost savings here compared to the standard MC approach are roughly V_L/V_0, corresponding to the ratio of the variance $\mathbb{V}[Q_{M_L} - Q_{M_{L-1}}]$ to the variance $\mathbb{V}[Q_{M_0}] \approx \mathbb{V}[Q_{M_L}]$, which is a small quantity.

- If the product $C_l\,V_l$ does not vary with level, the total cost is given by $\epsilon^{-2}L^2V_0\,C_0 = \epsilon^{-2}L^2V_L\,C_L$ [17].

The three cases delineated above are described more rigorously in Theorem 1 in [7], which states the following:

Theorem 3.1 *Let $\hat{Y}_l := \hat{Y}^{MC}_{l,N_l}$ and suppose that there exist positive constants α, β, $\gamma > 0$ such that $\alpha \geq \frac{1}{2}\min(\beta,\gamma)$, and the following hold:*

(i) $|\mathbb{E}[Q_{M_l} - Q]| \lesssim M_l^{-\alpha}$,

(ii) $\mathbb{V}[Y_l] \lesssim M_l^{-\beta}$,

(iii) $C_l \lesssim M_l^{\gamma}$.

Then, for any $\epsilon < e^{-1}$, there exists a value L (and correspondingly $M \equiv M_L$) and a sequence $\{N_l\}_{l=0}^{L}$ such that:

$$e\left(\hat{Q}^{MLMC}_M\right)^2 := \mathbb{E}\left[\left(\hat{Q}^{MLMC}_M - \mathbb{E}[Q]\right)^2\right] < \epsilon^2,$$

and

$$C\left(\hat{Q}^{MLMC}_M\right) \lesssim \begin{cases} \epsilon^{-2}, & \text{if } \beta > \gamma, \\ \epsilon^{-2}(\log\epsilon)^2, & \text{if } \beta = \gamma, \\ \epsilon^{-2-(\gamma-\beta)/\alpha}, & \text{if } \beta < \gamma. \end{cases}$$

Proof. The proof is given in Appendix A of [7]. □

We have already seen how to compute the optimal values of $\{N_l\}_{l=0}^{L}$ from the sample variances V_l, as given in Eq. (3.23). Further, to find L, or equivalently M_L, there are two alternatives. On the one hand, as mentioned at the beginning of this section, we can elect $M_L = \lceil(\sqrt{2}C)^{1/\alpha}\epsilon^{-1/\alpha}\rceil$ and M_0 sufficiently large, and subsequently find an integer $s \in \mathbb{N}\setminus\{1\}$ such that Eq. (3.13) is satisfied.

On the other hand, we can first choose $s \in \mathbb{N}\setminus\{1\}$ and define levels $l = 0,\ldots,L$ according to Eq. (3.13). Next, we start on level zero and increase the number of levels L until the discretisation error $(\mathbb{E}[Q_M - Q])^2$ is less than $\epsilon^2/2$, since $\{N_l\}_{l=0}^{L}$ are chosen so that the variance of the estimator is also less than $\epsilon^2/2$. However, the former cannot be computed exactly, as Q is an inaccessible random variable. To this end, we further assume that there exists an integer $M' \in \mathbb{N}$ such that the decline in $|\mathbb{E}[Q_M - Q]|$ is in fact monotonic for $M \geq M'$ and satisfies:

$$|\mathbb{E}[Q_M - Q]| = C\,M^{-\alpha}, \quad \alpha > 0. \tag{3.29}$$

Accordingly, one way to estimate the discretisation error above involves extrapolating and afterwards applying Eq. (3.29), as follows:

$$\begin{aligned}
|\mathbb{E}\left[Q_{M_L} - Q_{M_{L-1}}\right]| &= |\mathbb{E}\left[Q_{M_L} - Q\right] + \mathbb{E}\left[Q - Q_{M_{L-1}}\right]| \\
&= |CM_L^{-\alpha} - C(s^{-1}M_L)^{-\alpha}| \\
&= |CM_L^{-\alpha}|\,|1 - s^{\alpha}| \\
&= |\mathbb{E}\left[Q_M - Q\right]|\,|1 - s^{\alpha}|. \qquad (3.30)
\end{aligned}$$

Here, on the second line we make use of Eq. (3.13), so that $M_{L-1} = s^{-1}M_L$. Hence, since the quantity on the left-hand side is computable, the discretisation error can be approximated via:

$$|\mathbb{E}\left[Q_M - Q\right]| = \left|\mathbb{E}\left[Q_{M_L} - Q_{M_{L-1}}\right]\right| |1 - s^{\alpha}|^{-1}. \qquad (3.31)$$

Correspondingly, the adaptive version of the MLMC algorithm, in which the optimal number of levels L is computed ad hoc, can be implemented in practice as in [7]:

Algorithm 3.1 *Computes the MLMC estimate of the mean value of the quantity of interest Q with RMSE smaller than some given accuracy ϵ.*

(1) Start with $L = 0$.

(2) Estimate $\mathbb{V}[Y_L]$ by the sample variance of an initial number of samples.

(3) Calculate the optimal number of samples N_l, $l = 0, \ldots, L$ based on Eq. (3.23)*.*

(4) Evaluate extra samples at each level as needed for the new N_l.

(5) If $L \geq 1$, test for convergence using Eq. (3.30)*.*

(6) If not converged, set $L = L + 1$ and go back to step two.

It is worth highlighting that the third step in Algorithm 3.1 restricts the overall variance of the MLMC estimator to not exceed $\epsilon^2/2$, while the fifth step intends, in turn, to achieve a remaining bias not greater than $\epsilon/\sqrt{2}$. Consequently, the algorithm computes a MLMC estimate of $\mathbb{E}[Q]$ with a RMSE which satisfies $e\left(\hat{Q}_M^{\text{MLMC}}\right) < \epsilon$, in accordance with Eq. (3.15). Further functional details related to the above algorithm are discussed in Section 5, while an example of its Python implementation is given in Appendix C.

Additionally, an important observation is that the ideas introduced in this section are not limited to applications arising from PDEs with random coefficients. We solely illustrate them in this context for simplicity and concreteness.

Finally, returning to the application under consideration, recall Eq. (1.1). Independent of the quantity of interest, for a complete implementation of the MLMC algorithm we must be able to compute a sample $Q_l^{(i)}$, $l = 0, \ldots, L$. Specifically, we ought to accomplish the following two tasks: sample from k, particularly if $k = k(\mathbf{x}, \omega)$ is a random field rather than simply a random variable, and, for a given (fixed) sample ω, numerically solve the PDE (1.1) on two consecutively finer grids. These constitute the topics of the next section.

4 Methodology

In this section, we give a brief overview of the two mechanisms we exploit in the numerical experiments presented in Section 5. In particular, we describe how FEniCS can be used to numerically solve the 2D version of the PDE (1.1), with k modelled both as a random variable and as a random field. Additionally, we offer a technique for generating samples of the random field $k(\mathbf{x}, \omega)$, based on its Karhunen-Loève (KL) expansion [15].

4.1 FEniCS

Even in the most elementary scenario, Eq. (1.1) cannot be easily (if at all) solved analytically, and, as a consequence, we must resort to a numerical solver. In Python, a straightforward open-source software to engage is FEniCS (Finite Element Computational Software). Its computational high-performance `C++` backend DOLFIN (Dynamic Object-oriented Library for FINite element computation) serves as the main problem solving environment and user interface. It facilitates constructing user-defined functions, particularly as PDE coefficients, feature we make use of in Section 5.3, where we implement k as as a random field.

Thus, in order to be able to solve a PDE numerically with FEniCS, we must first derive its variational formulation. According to [30], the key steps for turning a PDE into a variational problem is to multiply it by a function v, known as a test function, integrate the resulting equation over the domain D, and perform integration by parts on terms with second-order derivatives. The function to be approximated, here p, is referred to as a trial function.

In our case, multiplying Eq. (1.1) by v and integrating over D gives:

$$-\int_D [\nabla \cdot (k\nabla p)]\, v \, \mathrm{d}\mathbf{x} = \int_D f\, v \, \mathrm{d}\mathbf{x}, \tag{4.1}$$

where $\mathrm{d}\mathbf{x}$ denotes the differential element for integration over the domain D. Further, to transform the second-order spatial derivative of p into a first-order derivative of p and v, we apply the *method of integration by parts*, yielding:

$$-\int_D [\nabla \cdot (k\nabla p)]\, v \;\, \mathrm{d}\mathbf{x} = \int_D (k\nabla p) \cdot \nabla v \, \mathrm{d}\mathbf{x} - \int_{\partial D} \left(k \frac{\partial p}{\partial \mathbf{n}} \right) v \, \mathrm{d}\mathbf{s}, \tag{4.2}$$

where ∂D is the boundary of the domain D, and $\mathrm{d}\mathbf{s}$ is the differential element for integration over ∂D. Moreover,

$$\frac{\partial p}{\partial \mathbf{n}} = \nabla p \cdot \mathbf{n} \tag{4.3}$$

is the derivative of p in the outward normal direction $\mathbf{n}$ on the boundary.

Furthermore, the second term in Eq. (4.2) can be evaluated using the information provided by the boundary conditions. To this end, following [28], let us assume that the PDE (1.1) is equipped with multiple Dirichlet, Neumann, and Robin conditions, as follows:

$$p = p_D^i \qquad \text{on } \partial D_D^i, \quad i = 0, 1, \ldots, \tag{4.4}$$

$$-k \frac{\partial p}{\partial \mathbf{n}} = g_i \qquad \text{on } \partial D_N^i, \quad i = 0, 1, \ldots, \tag{4.5}$$

$$-k \frac{\partial p}{\partial \mathbf{n}} = r_i (p - s_i) \qquad \text{on } \partial D_R^i, \quad i = 0, 1, \ldots, \tag{4.6}$$

where p_D^i, g_i, r_i and s_i, $i = 0, 1, \ldots$ are specified functions, and $\partial D_D^i, \partial D_N^i$ and ∂D_R^i, $i = 0, 1, \ldots$ represent the parts of the boundary where the Dirichlet, Neumann and Robin conditions apply, respectively. Note that this is only an example, and other boundary conditions can be included. What is more, Langtangen and Mardal comprehensively explain in [29] why the test function v must vanish on the parts of the boundary where the solution p is known, that is, where Dirichlet boundary conditions are provided.

Further, for the remaining edges, we make use of the Neumann and Robin conditions to

replace the boundary integral in Eq. (4.2) with a sum of integrals which can, in turn, be approximated. Subsequently, Eq. (4.2) becomes:

$$\begin{aligned}\int_D [\nabla \cdot (k\nabla p)]\, v \;\, \mathrm{d}\mathbf{x} &= \int_D k\nabla p \cdot \nabla v \, \mathrm{d}\mathbf{x} - \sum_i \int_{\partial D_N^i} k\frac{\partial p}{\partial \mathbf{n}} \, \mathrm{d}\mathbf{s} - \sum_i \int_{\partial D_R^i} k\frac{\partial p}{\partial \mathbf{n}} \, \mathrm{d}\mathbf{s} \\ &= \int_D k\nabla p \cdot \nabla v \, \mathrm{d}\mathbf{x} + \sum_i \int_{\partial D_N^i} g_i \, \mathrm{d}\mathbf{s} + \sum_i \int_{\partial D_R^i} r_i(p - s_i) \, \mathrm{d}\mathbf{s}. \end{aligned} \tag{4.7}$$

Combining the equation above with Eq. (4.1), we obtain the following variational problem:

$$F = \int_D k\nabla p \cdot \nabla v \;\, \mathrm{d}\mathbf{x} + \sum_i \int_{\partial D_N^i} g_i \, \mathrm{d}\mathbf{s} + \sum_i \int_{\partial D_R^i} r_i(p - s_i) \, \mathrm{d}\mathbf{s} - \int_D f\, v \, \mathrm{d}\mathbf{x} = 0. \tag{4.8}$$

In addition, as noted in [30], another step which needs to be performed by the user before computing the solution to Eq. (4.8) is choosing the finite element trial and test spaces V and $\hat{V}$. In the present case, they are defined as:

$$\begin{aligned} V &= \{v \in H^1(D) : v = p_D \text{ on } \partial D\}, \\ \hat{V} &= \{v \in H^1(D) : v = 0 \text{ on } \partial D\}. \end{aligned} \tag{4.9}$$

Here, $H^1(D)$ denotes the Sobolev space [6] which contains functions v such that v^2 and $|\nabla v|^2$ have finite integrals over D. In particular, this implies that the variational problem (4.8) is a continuous problem. Since our goal is to computationally solve the PDE (1.1), we must replace the infinite-dimensional function spaces V and $\hat{V}$, by discrete, finite-dimensional trial and test spaces $V_h \subset V$ and $\hat{V}_h \subset \hat{V}$. Thence, the discrete variational problem can be formulated as:

$$\text{Find } p \in V_h \subset V \text{ such that } F = 0, \;\; \forall v \in \hat{V}_h \subset \hat{V}. \tag{4.10}$$

This can now be entered into FEniCS, which subsequently solves it automatically. Note that while Neumann and Robin conditions are incorporated into Eq. (4.8), Dirichlet boundary conditions must be defined explicitly inside the program. Thus, Eq. (4.10), together with a suitable definition of V_h and $\hat{V}_h$, provides a unique approximate numerical solution to the PDE (1.1) with the boundary conditions (4.4), (4.5) and (4.6).

Lastly, FEniCS allows the user to specify the stepsize to be applied in the approximation algorithm and the type of basis functions to be used for the spaces V_h and $\hat{V}_h$. While the platform supports the notation defined in the Periodic Table of the Finite Elements [3] for all simplex element families (simplest polytope in a given space), we solely use linear polynomials throughout our programs. An example of a Python implementation which sets up the mesh, function spaces and Dirichlet boundary conditions is given in the `setup_PDE` function in Appendix B.

At last, Figure 2 displays, for illustration purposes, the solution to the PDE (1.1) for an arbitrary sample of the random field k, with $\rho = 0.3$, $\sigma^2 = 2$ and $K = 1400$ (hyperparameters explained in Section 4.2), subject to the following Dirichlet and Neumann boundary conditions:

$$p|_{x_1=0} = 1, \quad p|_{x_1=1} = 0, \tag{4.11}$$

$$\left.\frac{\partial p}{\partial \mathbf{n}}\right|_{x_2=0} = 0, \quad \left.\frac{\partial p}{\partial \mathbf{n}}\right|_{x_2=1} = 0. \tag{4.12}$$

The snapshot in Fig. 2 was created based on The ParaView Guide [44], using ParaView [37]. This is an open-source, multi-platform data analysis and visualisation tool for plotting scalar and vector fields, including those computed by FEniCS.

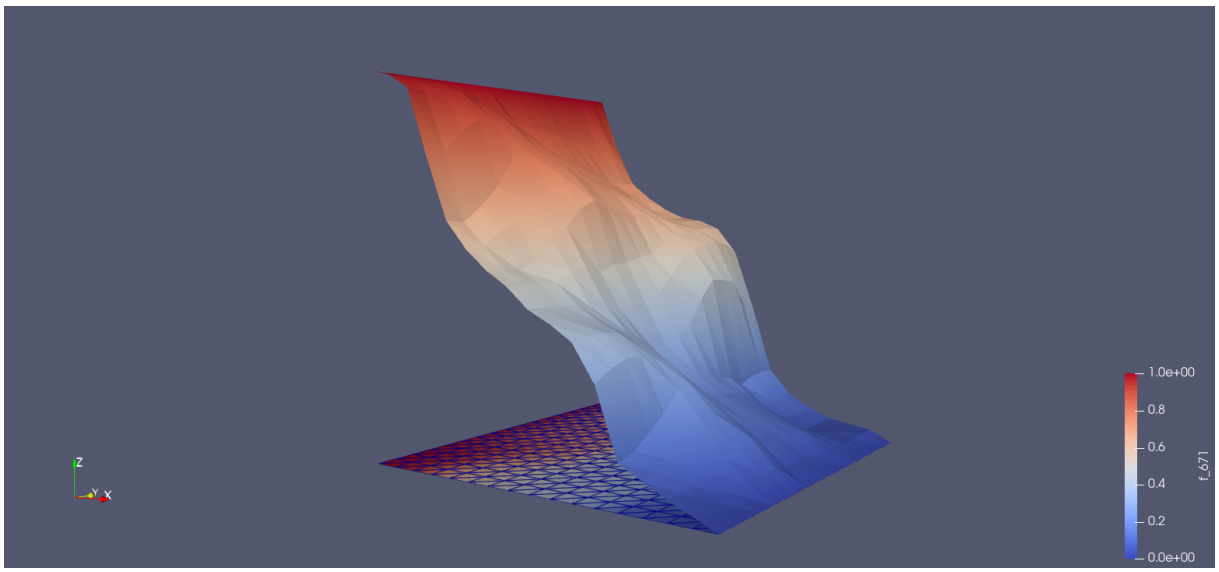


Figure 2: Plot created using ParaView of the mesh and the solution to the PDE (1.1) subject to Dirichlet boundary conditions (4.11) and Neumann conditions (4.12) for an arbitrary sample of the random field $k(\mathbf{x}, \omega)$. The approximation was obtained using FEniCS with a stepsize $h = 1/32$ (2048 triangles), linear polynomials and $f(\mathbf{x}) \equiv 1$.

4.2 Karhunen-Loève expansions

A random field $k(\mathbf{x}, \omega)$ is a function over an arbitrary domain D which takes a random value at each point in D. In other words, it is a generalisation of a stochastic process. However, in the case of a random field, the underlying parameter is no longer real or integer valued, i.e. "time". Rather, k can take values which are multidimensional vectors or points on some manifold [49]. This loose definition is given more rigorously by Tsantili and Hristopulos in [48]:

Definition 4.1 *Let $(\Omega, \mathcal{F}, \mathbb{P})$ denote a complete probability space and $D \subseteq \mathbb{R}$ the spatial domain of interest. Then a random field $k(\mathbf{x}, \omega)$ is a collection of dependent random variables distributed over D. The random field is defined by a mapping from $D \times \Omega$ into the set of real numbers $\mathbb{R}$. Hence, for any fixed $\mathbf{x} \in D$, $k(\mathbf{x}, \omega) \to k(\omega)$ is a random variable $k : \Omega \to \mathbb{R}$. On the other hand, for any sample $\omega \in \Omega$, $k(\mathbf{x}, \omega) \to \boldsymbol{k}(\mathbf{x})$ is a function $\boldsymbol{k} : D \to \mathbb{R}$ of $\mathbf{x}$.*

While they can be used in a variety of applications, such as generation of computer graphics [25] or task-related functional brain imaging [51], here we focus on random fields modelling hydraulic conductivity in subsurface flow. One such model that has been widely studied for $k(\mathbf{x}, \omega)$ is a log-normal distribution, that is, a scalar valued field whose log is Gaussian.

Additionally, the random field k is equipped with a certain mean and covariance structure which is generally determined from the input data. To this end, Hoeksema and Kitanidis suggest in [23] the following covariance function:

$$C(\mathbf{x}, \mathbf{y}) := \sigma^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{y}\|_p}{\rho}\right), \quad \mathbf{x}, \mathbf{y} \in D, \tag{4.13}$$

where $\|\cdot\|_p$ denotes the ℓ_p-norm in $\mathbb{R}^d$. Throughout this thesis we only use $p = 1$, that is:

$$\|\mathbf{x} - \mathbf{y}\|_1 := \sum_{i=1}^{d} |x_i - y_i|, \quad \mathbf{x}, \mathbf{y} \in D \subset \mathbb{R}^d. \tag{4.14}$$

Moreover, the parameters σ^2 and ρ denote the variance and the correlation length of $C(\mathbf{x}, \mathbf{y})$, respectively. In particular, σ^2 relates to the variability of fluctuations in the domain, while ρ measures the extent of the spatial correlation, or, in other words, the distance over which two measurements of the quantity of interest tend to become uncorrelated.

Furthermore, for the mean function, [23] proposes a constant expectation model, specifically:

$$\mathbb{E}[\log k(\mathbf{x}, \cdot)] = \mu, \quad \mu \in \mathbb{R}. \tag{4.15}$$

As aforementioned, the parameters σ^2, ρ and μ are typically inferred from the given measurements. However, with scarce (barely any!) relevant data, we instead resort to using suitable constants. Typical values of interest in subsurface flow applications are $\sigma^2 \geq 1$ and $\rho \leq \text{diam } D = 1$ [7]. In addition, throughout the remainder of this report, we adopt for simplicity $\mu = 0$.

Indeed, this is a relatively simple representation, which accomplishes various objectives. First, it does not significantly escalate the computational complexity of numerically solving (1.1). Second, imperative in the current context, the proposed framework guarantees that $k > 0$ almost surely (a.s.) in Ω. Last, it prompts the range of possible values for the hydraulic conductivity to span multiple orders of magnitude, which is typical in subsurface flow. These features render the log-normal distribution model for $k(\mathbf{x}, \omega)$ feasible for computer simulations and faithful to the physical application.

Once we have established a suitable structure for the random field, the succeeding step is to generate samples of k. Existent techniques which achieve this include Bayesian Compressive Sensing [26] and circulant embedding [20]. Here, we focus on the Karhunen-Loève (KL) expansion [15], which is a powerful tool for simulating random fields with known covariance functions. Accordingly, the remainder of this section largely follows [15], so that the central idea behind KL-expansions is to expand the random process $k(\mathbf{x}, \omega)$ in a series of the Fourier type, such as:

$$k(\mathbf{x}, \omega) = \sum_{n=1}^{\infty} \sqrt{\lambda_n} \xi_n(\omega) f_n(\mathbf{x}), \tag{4.16}$$

where $\{\xi_n(\omega)\}_{n \in \mathbb{N}}$ is a set of random variables to be determined, λ_n is some constant, and $\{f_n(\mathbf{x})\}_{n \in \mathbb{N}}$ is an orthonormal set of deterministic functions.

Specifically, a random field $k(\mathbf{x}, \omega)$ with expected value $\mathbb{E}[k(\mathbf{x}, \cdot)]$ over all possible realisations of the process and covariance function $C(\mathbf{x}_1, \mathbf{x}_2)$ can be expressed as:

$$k(\mathbf{x}, \omega) = \mathbb{E}[k(\mathbf{x}, \cdot)] + \sum_{n=1}^{\infty} \sqrt{\lambda_n} \xi_n(\omega) f_n(\mathbf{x}), \tag{4.17}$$

where $\{\xi_n\}_{n \in \mathbb{N}}$ is a collection of zero-mean, unit standard-deviation, uncorrelated random variables. In other words, they satisfy:

$$\mathbb{E}[\xi_n(\omega)] = 0, \quad \mathbb{E}[\xi_n(\omega)\xi_m(\omega)] = \delta_{nm}, \quad \forall n, m \in \mathbb{N}. \tag{4.18}$$

Here, δ_{nm} is the Kronecker delta function, so that $\delta_{nm} = 1$ when $n = m$, and $\delta_{nm} = 0$, when $n \neq m$. Moreover, $\{\lambda_n\}_{n \in \mathbb{N}}$ and $\{f_n(\mathbf{x})\}_{n \in \mathbb{N}}$ in (4.17) are the eigenvalues and the normalised eigenfunctions of the covariance kernel $C(\mathbf{x}_1, \mathbf{x}_2)$, respectively. In particular, they are the solution to the following integral equation:

$$\int_D C(\mathbf{x}_1, \mathbf{x}_2) f_n(\mathbf{x}) \mathrm{d}\mathbf{x}_1 = \lambda_n f_n(\mathbf{x}_2). \tag{4.19}$$

Additionally, a precise expression for $\xi_n(\omega)$ can be obtained by multiplying the sum in Eq. (4.17) by $f_n(\mathbf{x})$ and subsequently integrating over the domain D. This yields:

$$\xi_n(\omega) = \frac{1}{\lambda_n} \int_D \left(\sum_{m=1}^{\infty} \xi_m(\omega) \sqrt{\lambda_m} f_m(\mathbf{x}) \right) f_n(\mathbf{x}) \mathrm{d}\mathbf{x}. \tag{4.20}$$

However, note that if the random field $k(\mathbf{x}, \omega)$ is Gaussian, the set $\{\xi_n\}_{n \in \mathbb{N}}$ is simply a collection independent standard Gaussian random variables.

Thus, returning to the previous log-normal model for k, let $Z(\mathbf{x}, \omega) := \log k(\mathbf{x}, \omega)$ be a Gaus-

sian random field. With $\mathbb{E}[k(\mathbf{x},\cdot)] = 0 = \mathbb{E}[Z(\mathbf{x},\cdot)]$ and the two-point exponential covariance function (4.13), the KL-expansion of Z is given by:

$$Z(\mathbf{x},\omega) = \sum_{n=1}^{\infty} \sqrt{\lambda_n}\xi_n(\omega) f_n(\mathbf{x}). \tag{4.21}$$

Accordingly, $\{\lambda_n\}_{n\in\mathbb{N}}$ and $\{f_n\}_{n\in\mathbb{N}}$ are the eigenvalues and normalised eigenfunctions of the covariance operator with kernel function $C(\mathbf{x},\mathbf{y})$ defined in Eq. (4.13), while $\{\xi_n\}_{n\in\mathbb{N}}$ is a collection of standard Gaussian random variables, as aforementioned.

Admittedly, the integral equation (4.19) must be solved numerically in most cases. Efficient techniques include quadrature-based methods [21], matrix compression approaches [10], or expansion-based techniques. The latter consists of approximating each eigenfunction by a linear combination of chosen basis functions and minimising the resulting error using, for instance, stochastic collocation methods [52]. Nonetheless, analytical solutions can also be obtained for some relevant kernel functions, which is the case for $C(\mathbf{x},\mathbf{y})$ as defined in (4.13) with $p = 1$.

Therefore, following [14], in the case of the 1-norm $\|\cdot\|_1$, with $d = 1$, $D = (0,1)$ and $\sigma^2 = 1$ the explicit forms for the eigenpairs $\{\lambda_n, f_n\}_{n\in\mathbb{N}}$ of the covariance function (4.13) are given by:

$$\lambda_n^{1\mathrm{D}} = \frac{2\rho}{\rho^2 w_n^2 + 1}, \quad n \in \mathbb{N}, \tag{4.22}$$

$$f_n(x)^{1\mathrm{D}} = C_n \left(\sin(w_n x) + \rho\, w_n \cos(w_n x)\right), \quad n \in \mathbb{N}. \tag{4.23}$$

Here, $\{w_n\}_{n\in\mathbb{N}}$ are the real positive solutions of the following transcendental equation:

$$\tan(w) = \frac{2\rho\, w}{\rho^2 w^2 - 1}. \tag{4.24}$$

Further, the constant C_n in Eq. (4.23) is selected so that $\|f_n\|_{L_2(0,1)} = 1$, where the $L_2(D)$ norm defined over the domain D is defined as:

$$\|f_n\|_{L_2(D)} = \sqrt{\int_D |f_n(x)|^2 \mathrm{d}x}. \tag{4.25}$$

In order to find an explicit expression for C_n, let us first derive the detailed form of $\|f_n\|_{L_2(0,1)}$. To this end, we first substitute $f_n(x)$ with its formula given in Eq. (4.23), subsequently expanding the square inside the integral in (4.25). This yields:

$$\|f_n\|_{L_2(0,1)} = \sqrt{\int_0^1 C_n^2 \left(\sin^2(w_n x) + 2\rho\, w_n \sin(w_n x)\cos(w_n x) + \rho^2 w_n^2 \cos^2(w_n x)\right) \mathrm{d}x}. \tag{4.26}$$

Moreover, splitting the integral into a sum of separate terms produces, in turn, three elementary integrals. These are straightforward to solve, by directly applying the trigonometric double angle formulae for $\sin^2(x)$ and $\cos^2(x)$. On these grounds, the resulting equations for each integral are, consecutively, as follows:

$$\int_0^1 \sin^2(w_n x)\mathrm{d}x = \int_0^1 \left(\frac{1-\cos(2w_n x)}{2}\right) \mathrm{d}x = \frac{w_n}{2} - \frac{\sin(2w_n)}{4}, \tag{4.27}$$

$$\int_0^1 2\rho\, w_n \sin(w_n x)\cos(w_n x)\mathrm{d}x = \rho\, w_n \int_0^1 \sin(2w_n x)\mathrm{d}x = \frac{\rho\, w_n}{2}\left(1 - \cos(2w_n)\right), \tag{4.28}$$

and

$$\int_0^1 \rho^2 w_n^2 \cos^2(w_n x)\mathrm{d}x = \rho^2 w_n^2 \left(\int_0^1 \frac{1+\cos(2w_n x)}{2}\right) \mathrm{d}x = \rho^2 w_n^2 \left(\frac{w_n}{2} + \frac{\sin(w_n)}{4}\right). \tag{4.29}$$

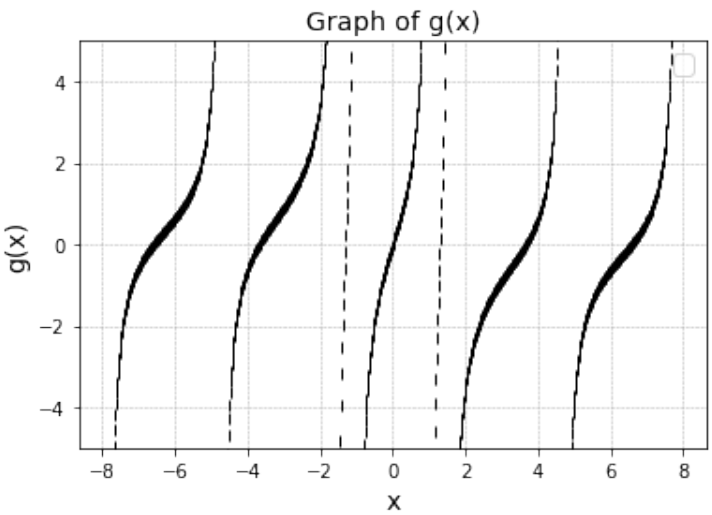


Figure 3: Graph of $g(x)$ as defined in Eq. (4.31) for $x \in \left(-\frac{5\pi}{2}, \frac{5\pi}{2}\right)$ and $\rho = 1$.

Lastly, equating (4.26) to 1 and applying Eq. (4.27), (4.28), and (4.29) prompts the following formula for the constant C_n:

$$C_n = \left(\frac{\sin(2w_n)}{4} \left(\rho^2 w_n - \frac{1}{w_n} \right) - \frac{\rho}{2} \cos(2w_n) + \frac{1}{2} \left(1 + \rho + \rho^2 w_n^2 \right) \right)^{-\frac{1}{2}}. \tag{4.30}$$

For a complete implementation of (4.21), the only missing element is a mechanism for finding the real roots $\{w_n\}_{n\in\mathbb{N}}$. As we cannot rely on analytical solutions to Eq. (4.24), we instead turn to a numerical solver. To this end, we take advantage of the `fsolve` routine available in Python's SciPy library [43], which requires as an argument a starting estimate for the roots of the equation to be solved. Accordingly, we seek the necessary initial guess by re-writing the transcendental equation (4.24) in the form $g(w) = 0$, and plotting g, with $g : \mathbb{R} \to \mathbb{R}$ given by:

$$g(w) = \tan(w) - \frac{2\rho\, w}{\rho^2 w^2 - 1}. \tag{4.31}$$

Correspondingly, its graph is given in Figure 3, based on which we raise a series of remarks. To begin with, due to the presence of the $\tan(w)$ function in the definition of g, we observe vertical asymptotes at each $w = s\pi/2, s \in \mathbb{Z}$. Further, between any two such asymptotes, the graph crosses the x-axis strictly once, generating exactly one real root, as it can be noticed in Figure 3. Additionally, the denominator of the ratio in Eq. (4.31) vanishes at $w = \pm 1/\rho$, yielding two more vertical asymptotes. Adopting the same reasoning as in the previous argument, we can assert that exactly one additional root can be identified in the intervals which include $w = \pm 1/\rho$.

Thus, recall that our aim is to find positive real roots $\{w_n\}_{n\in\mathbb{N}}$ of Eq. (4.24). Exploiting the preceding comments, we can employ the `fsolve` routine using intervals of the form $(-s\pi/2, s\pi/2), s \in \mathbb{N}$ as an offset estimate for the solutions. Moreover, for the interval including $1/\rho$, we can distinguish between the two roots by searching in the first and second half of the interval, successively. As an example, a Python implementation of the described procedure is given in the function `roots_transcendental` in Appendix D. This is based on the MATLAB implementation developed by Teckentrup for the numerical experiments in [7].

Returning to the KL-expansion, it is relatively easy to extend the formulae for the eigenpairs $\{\lambda_n, f_n\}_{n\in\mathbb{N}}$ from the $d = 1$ and $D = (0, 1)$ restrictions in Eq. (4.22) and (4.23) to the $d = 2$ and $D = (0, 1)^2$ case. In particular, according to [14], they can be expressed as:

$$\lambda_n^{2\mathrm{D}} = \lambda_{i_n}^{1\mathrm{D}} \lambda_{j_n}^{1\mathrm{D}}, \quad f_n^{2\mathrm{D}}(\mathbf{x}) = f_{i_n}^{1\mathrm{D}}(x_1) f_{j_n}^{1\mathrm{D}}(x_2), \tag{4.32}$$

for some i_n, $j_n \in \mathbb{N}$.

Furthermore, for values of σ^2 different from 1, in both the 1D and 2D cases, we need only multiply the eigenvalues by σ^2, while the form of the eigenfunctions does not change.

What is more, to be able to implement a KL-expansion for $k(\mathbf{x}, \omega)$ on a computer program, we must also truncate the sum in Eq. (4.17) after a finite number of K terms. In practice, the aforementioned number of KL-modes is chosen so that a sufficient percentage of the variance σ^2 is captured by the first K terms (such as 95%). To this end, let Z_K denote the KL-expansion of Z, as defined in Eq. (4.21), truncated after K terms. Subsequently, following the derivation in [7], notice that the mean value of the $L_2(D)$ error which emerges from replacing the infinite sum in Eq. (4.21) by a finite one can be expressed as follows:

$$\begin{aligned}
\mathbb{E}\left[\|Z - Z_K\|^2_{L_2(D)}\right] &= \mathbb{E}\left[\left\|\sum_{n=1}^{\infty} \sqrt{\lambda_n}\xi_n(\omega) f_n(\mathbf{x}) - \sum_{n=1}^{K} \sqrt{\lambda_n}\xi_n(\omega) f_n(\mathbf{x})\right\|^2_{L_2(D)}\right] \\
&= \mathbb{E}\left[\left\|\sum_{n=K+1}^{\infty} \sqrt{\lambda_n}\xi_n(\omega) f_n(\mathbf{x})\right\|^2_{L_2(D)}\right] \\
&= \mathbb{E}\left[\int_D \left(\sum_{n=K+1}^{\infty} \sqrt{\lambda_n}\xi_n(\omega) f_n(\mathbf{x})\right)^2 d\mathbf{x}\right] \\
&= \mathbb{E}\left[\int_D \left(\sum_{p=K+1}^{\infty}\sum_{q=K+1}^{\infty} \sqrt{\lambda_p}\sqrt{\lambda_q}\xi_p(\omega)\xi_q(\omega) f_p(\mathbf{x}) f_q(\mathbf{x})\right) d\mathbf{x}\right] \qquad \text{(a)} \\
&= \sum_{p=K+1}^{\infty}\sum_{q=K+1}^{\infty} \sqrt{\lambda_p}\sqrt{\lambda_q}\,\mathbb{E}[\xi_p(\omega)]\,\mathbb{E}[\xi_q(\omega)] \int_D f_p(\mathbf{x}) f_q(\mathbf{q}) d\mathbf{x} \qquad \text{(b)} \\
&= \sum_{n=K+1}^{\infty} \lambda_n. \qquad (4.33)
\end{aligned}$$

Here, in (a) we used the independence of $\{\xi_n\}_{n=1}^{\infty}$. Moreover, in (b) we applied, on the one hand, $\mathbb{E}[\xi_n(\omega)] = 1$, $\forall n \in \mathbb{N}$, and on the other hand, the orthonormality of $\{f_n\}_{n\in\mathbb{N}}$, that is:

$$\int_D f_p(\mathbf{x}) f_q(\mathbf{x}) d\mathbf{x} = \begin{cases} 0, & \text{if } p \neq q, \\ 1, & \text{if } p = q. \end{cases} \qquad (4.34)$$

Thus, the accuracy of the truncated KL-expansion Z_K rests plainly on the decay of the eigenvalues λ_n. According to [41], they decline quadratically with respect to n, that is $\lambda_n \lesssim n^{-2}$. In addition, this, together with Eq. (4.33), suggests that the RMSE of Z_K satisfies $e_{L_2(D)}(Z_K) \lesssim K^{-1/2}$, since:

$$e_{L_2(D)}(Z_K) := \sqrt{\mathbb{E}\left[\|Z - Z_K\|^2_{L_2(D)}\right]} = \sqrt{\sum_{n=K+1}^{\infty} \lambda_n} \lesssim K^{-1/2}. \qquad (4.35)$$

In particular, this indicates that in order to reach even small relative errors, such as 10^{-2}, a considerable number of KL-modes is required.

It is important to discern that this poses significant limitations to the improvement that the MLMC algorithm can deliver in comparison to the simple MC approach. First, numerical investigations in [7] show that, for the model currently analysed regarding the random field coefficient $k(\mathbf{x}, \omega)$, the optimal choice for the coarsest mesh size is $h_0 \approx \rho$, that is, $M_0 \approx \rho^{-2}$. Clearly, this limits the number of levels that can be used in a computer program, and, as a consequence, the gains that the MLMC estimator could provide. Second, the computational resources required for generating the KL-expansion are noticeable, especially for small correlation lengths ρ, such as $\rho = 0.1$ or $\rho = 0.01$.

Indeed, a solution to these issues is proposed in [45]. In particular, Teckentrup et al. introduce level dependent (LD) truncations of the KL-expansion of $k(\mathbf{x}, \omega)$ when using MLMC estimators for approximating various quantities of interest related to solutions of PDEs with random coefficients. Accordingly, the leading concept in the aforementioned approach is that a large number of terms needs to be included in the KL-expansion of k on the finest grid exclusively. In other words, we are alternatively able to truncate the sum earlier on coarser levels, incorporating fewer terms and diminishing the overall computational complexity.

The underlying intuition for the above procedure hinges on a relevant property of the eigenvalues and eigenfunctions of the covariance operator with kernel function $C(\mathbf{x}, \mathbf{y})$ defined in Eq. (4.13). Specifically, as noted in [45], if the eigenvalues $\{\lambda_n\}_{n\in\mathbb{N}}$ are arranged in decreasing order of magnitude, the corresponding eigenfunctions $\{f_n\}_{n\in\mathbb{N}}$ will be organised in increasing order of fluctuations over D. Therefore, we are neglecting the contributions of the most oscillatory eigenfunctions by truncating the KL-expansion after $K_l < K$ terms on level l. Moreover, this allows us to choose M_0 autonomous in respect to ρ, since $k_{K_l}(\mathbf{x}, \omega)$ is a more accurate approximation of the random field coefficient than $k_K(\mathbf{x}, \omega)$ on the coarser levels. As a result, the level dependent truncation leads to noteworthy improvements upon the uniformly truncated KL-expansion.

With this in mind, the immediate challenge is identifying a strategy for choosing K_l in relation to the approximation on level l. Teckentrup et al. prove in [45] a series of propositions and corollaries which offer different bounds for K_l depending on the quantity of interest under consideration. In particular, the number of K_l modes to be included on each level is directly connected with the regularity of the preceding functional. Although the theory is compelling and intricate, it is beyond the scope of this thesis.

Alternatively, we adopt a numerical approach to determining a relationship between M_l and K_l. We resort to this procedure mainly due to the high irregularity of the functional considered throughout our numerical experiments, which is the point evaluation of the solution p. Specifically, this is not a bounded functional on the Sobolev space $H_0^1(D)$ in domains with more than one dimension [45].

To this end, note that when integrating the KL-expansion of $k(\mathbf{x}, \omega)$ with the MLMC algorithm, the relative error in (4.35) primarily alters the derivation of the bias in Eq. (3.30). Notably, we can no longer assume that $|\mathbb{E}[Q_M - Q]| \eqsim M^{-\alpha}$, and instead we consider:

$$\begin{aligned}
|\mathbb{E}[Q_{M_l,K_l} - Q_{M_{l-1},K_{l-1}}]| &= |\mathbb{E}[Q_{M_l,K_l} - Q_{M_l,K_{l-1}}] + \mathbb{E}[Q_{M_l,K_{l-1}} - Q_{M_{l-1},K_{l-1}}]| \\
&\leq |\mathbb{E}[Q_{M_l,K_l} - Q_{M_l,K_{l-1}}]| + |\mathbb{E}[Q_{M_l,K_{l-1}} - Q_{M_{l-1},K_{l-1}}]| \\
&= C_\delta K^{-\delta} + C_\alpha M^{-\alpha}, \quad \alpha, \delta > 0,
\end{aligned} \tag{4.36}$$

where in the second step we used the triangle inequality for real numbers, with Q_{M_l,K_l} denoting the approximation of Q on level l with M_l degrees of freedom and K_l modes in the KL-expansion of k. Thus, the crucial remark here is that $K = K_L$ should be elected large enough to not increase the contribution of the bias in Eq. (3.15), and consequently the RMSE of the MLMC estimator.

Further, we make use of the hypothesis in Eq. (4.36) to determine a relationship between M_l and K_l. Particularly, to balance the computational effort invested in constructing a discretisation with M_l degrees of freedom and a truncated KL-expansion with K_l terms, we assume that each term has a contribution of $\sqrt{\epsilon}/4$ to the bias, for given accuracy ϵ. In this case, the overall discretisation and model error is $\epsilon^2/2$, so that, according to Eq. (3.15), the RMSE of the MLMC estimator satisfies $e\left(\hat{Q}_{M,N}^{\text{MLMC}}\right) < \epsilon$. In other words, this suggests the following equality:

$$C_\alpha M_l^{-\alpha} = C_\delta K_l^{-\delta} \iff K_l \eqsim M_l^{\frac{\alpha}{\delta}}, \quad l = 0, \ldots, L. \tag{4.37}$$

This equation can now be used to find K_l for each level $l = 0, \ldots, L$, so that no any additional model errors that decline at a slower rate than the discretisation error are introduced.

Hence, equipped with techniques for approximately solving Eq. (1.1) and generating samples of the random field $k(\mathbf{x}, \omega)$, we now proceed to illustrate various computational experiments. These are aimed at numerically reinforcing the theoretical arguments already established.

5 Numerical Results

In this section, we compare the performance of the MC and MLMC algorithms for computing the expected value of a quantity of interest Q for the PDE (1.1). First, we explore a 1D example, where Q can, in fact, be found analytically. Subsequently, we proceed to a 2D model, where the solution, and consequently the quantity of interest, must be approximated numerically.

Commonly, the statistics of several functionals of the solution are of interest [7], such as the variance of the flow rate at a certain point in the domain, or the average travel time of a particle through the fluid. Here, our main focus is the expected value of the pressure at a specific point in the domain, for instance 3/4 in 1D or $(7/15, 7/15)$ in 2D. In addition, we also consider the expected value of the effective conductivity of the region $D = (0,1)^2$, which is related to the cumulative outflow from D on the boundary $x_1 = 1$. Following [19], this is defined in 2D as:

$$k_{\text{eff}} := -\int_0^1 k \frac{\partial p}{\partial x_1}\bigg|_{x_1=1} dx_2, \tag{5.1}$$

where k denotes the hydraulic conductivity. Correspondingly, our objective is its 1D equivalent, which is simply given by:

$$k_{\text{eff}} = -k \frac{dp}{dx}\bigg|_{x=1}. \tag{5.2}$$

Furthermore, throughout this chapter we use CPU times to quantify the computational cost of the investigated algorithms. These were obtained employing our Python implementation on a 2 GHz Quad-Core Intel Core i5 processor with 16GB of RAM. Even though our code is not optimised, quantifying the cost by the number of FLOPS would be difficult as we make use of an external PDE numerical solver, whose work is not known exactly. In addition, we take advantage of Python's `time` library to measure the CPU time, and we adopt NumPy's [32] Random Generator `default_rng` to sample from various distributions. While all the Python programs used to produce the following results are available in Appendices A-F, throughout the upcoming sections we pinpoint specific snippets of code for illustrative purposes.

5.1 1D example

Let us now turn our attention to the the 1D version of Eq. (1.1):

$$-\frac{d}{dx}\left(k\frac{dp}{dx}(x)\right) = f(x), \quad x \in D, \tag{5.3}$$

where the hydraulic conductivity is modelled as a random variable $k \sim \text{log-normal}(\mu, \sigma^2)$. This is now an ordinary differential equation (ODE) with a random coefficient. We restrict ourselves to the case $D = (0,1)$, and, in order to simulate the natural phenomena, we impose boundary conditions which ensure that the groundwater flow is from $x = 0$ to $x = 1$ (left to right):

$$p(0) = 1 \quad \text{and} \quad p(1) = 0. \tag{5.4}$$

For simplicity, we take $f(x) = a$, where $a \in \mathbb{R}^*$ is a constant. In this case, the ODE (5.3) can be solved explicitly, yielding the solution:

$$p(x) = \frac{a}{2k}x^2 - \left(1 + \frac{a}{2k}\right)x + 1. \tag{5.5}$$

Further, as mentioned at the beginning of this chapter, the quantity of interest we wish to estimate is the mean value of the pressure at a certain point in the domain, say $x = 3/4$. In this simple example, $\mathbb{E}[p(3/4)]$ can be determined analytically and afterwards used to compute the RMSE between the Monte Carlo approximation and the exact value at each simulation. To achieve this, from Eq. (5.5), it is clear that we need only derive $\mathbb{E}[1/k]$. Since $\log(k) \sim N(\mu, \sigma^2)$,

we have that:

$$\log\left(\frac{1}{k}\right) = -\log(k) \sim N(-\mu, \sigma^2) \implies \frac{1}{k} \sim \text{log-normal}(-\mu, \sigma^2). \tag{5.6}$$

Moreover, as stated in [38], the expectation of a log-normal random variable is given by :

$$\mathbb{E}\left[\frac{1}{k}\right] = \exp\left(-\mu + \frac{\sigma^2}{2}\right). \tag{5.7}$$

Accordingly, taking $k \sim \text{log-normal}(0, 1)$ and $a = 1$, the exact value of $\mathbb{E}[Q]$ is:

$$\mathbb{E}\left[p\left(\frac{3}{4}\right)\right] = \frac{a}{2}\left(\frac{3}{4}\right)^2 \exp\left(\frac{1}{2}\right) - \left(1 + \frac{a}{2}\exp\left(\frac{1}{2}\right)\right)\frac{3}{4} + 1 \approx 0.095. \tag{5.8}$$

Similarly, following [38], we can establish the analytical expression of the variance of $1/k$ as:

$$\mathbb{V}\left[\frac{1}{k}\right] = [\exp(\sigma^2) - 1]\exp[(-2\mu) + \sigma^2]), \tag{5.9}$$

With the same hyperparameters as before, the variance of Q is hence given by:

$$\mathbb{V}\left[p\left(\frac{3}{4}\right)\right] = \left(\frac{a}{2}\left(\frac{3}{4}\right)^2 - \frac{a}{2}\cdot\frac{3}{4}\right)^2 [\exp(1) - 1]\exp(1) \approx 0.041. \tag{5.10}$$

This is necessary for estimating the sampling error, and afterwards the RMSE in Eq. (2.13).

As we are using the closed-form solution of the ODE (5.3), it is implausible to employ MLMC here. Thus, we focus only on applying MC for approximating the mean value $\mathbb{E}[p(3/4)]$. The computational times are obtained by running the algorithm R times and averaging the results. In all our experiments below, we let $R = 100$.

Further, to choose the number of samples N, recall the two sources of error in the MC estimator, as shown in Eq. (2.13). Note that, in this case, the only contribution to the RMSE originates from replacing the expected value by a finite sample average. Hence, similar to the derivation at the end of Section 2.2, for a fixed accuracy ϵ, we require:

$$N > \mathbb{V}[Q]\epsilon^{-2} \implies \frac{1}{N}\mathbb{V}[Q] < \epsilon^2. \tag{5.11}$$

Subsequently, to ensure that N is an integer, we can simply choose $N = \lceil \mathbb{V}[Q]\epsilon^{-2} \rceil$. As explained at the end of Section 2.2, we therefore expect:

$$C_\epsilon\left(\widehat{Q}_N^{\text{MC}}\right) \lesssim \epsilon^{-2}. \tag{5.12}$$

Figure 4 shows the results for the case $\mu = 0$ and $\sigma^2 = 1$, so that $k \sim \text{log-normal}(0, 1)$. We use equally spaced RMSE values from $\epsilon = 5\cdot 10^{-3}$ to $\epsilon = 5\cdot 10^{-5}$. Then, assuming $C_\epsilon\left(\widehat{Q}_N^{\text{MC}}\right) = C\cdot\epsilon^\tau$, the value of τ can be recovered by determining the best linear fit through the derived points $\left(\epsilon, C_\epsilon\left(\widehat{Q}_N^{\text{MC}}\right)\right)$ on a log scale. This is because taking logarithms in the above equation gives:

$$\log C_\epsilon\left(\widehat{Q}_N^{\text{MC}}\right) = \log C + \tau\log\epsilon. \tag{5.13}$$

As a result, the slope of the aforementioned regression line yields the value of τ, while the y-intercept gives an estimate for C upon exponentiation. In Python, we exploit NumPy's `polyfit` function, which fits a polynomial of degree *deg* to given points (x, y). Consequently, by choosing *deg*$= 1$, we can take $(x, y) = \left(\log\epsilon, \log C_\epsilon\left(\widehat{Q}_N^{\text{MC}}\right)\right)$ to retrieve the required τ.

Returning to Figure 4, note that, as the variance is rather small in this case, so is the number

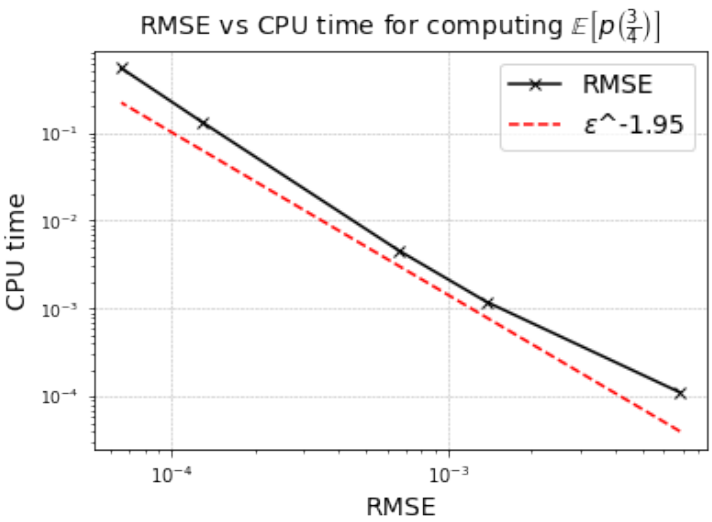


Figure 4: Performance plot on a log scale for $\mu = 0$ and $\sigma^2 = 1$ in 1D using Monte Carlo. The quantity of interest is the mean value of the pressure at $x = 3/4$.

of MC samples, and consequently, the computational time. The longest simulation is naturally for the smallest accuracy, $\epsilon = 5 \cdot 10^{-5}$, when the number of samples is $N = 16,420,691$ and the total time is 0.5 seconds per simulation on average. Even for these modest CPU times, we still observe that the slope of the line for $C_\epsilon\left(\widehat{Q}_N^{\mathrm{MC}}\right)$ is approximately equal to -2, as expected.

Additionally, we can apply the same procedure for the effective conductivity k_{eff}, as defined in Eq. (5.2). Using the closed-form solution in Eq. (5.5), the analytical expression of k_{eff} is:

$$k_{\mathrm{eff}} = k - \frac{a}{2}. \tag{5.14}$$

With $k \sim \text{log-normal}(0, 1)$ and $a = 1$, we get the following values for its expectation and variance:

$$\mathbb{E}[k_{\mathrm{eff}}] = \exp\left(\frac{1}{2}\right) - \frac{1}{2} \approx 1.149, \tag{5.15}$$

$$\mathbb{V}[k_{\mathrm{eff}}] = [\exp(1) - 1]\exp(1) = 4.671. \tag{5.16}$$

Employing the same strategy for choosing N, we apply the MC algorithm to approximate the mean value of the effective conductivity. From Eq. (5.16), we notice that the variance in this case is rather large, so that more samples are required per simulation and, correspondingly, longer CPU times are required. For instance, for an accuracy of $\epsilon = 5 \cdot 10^{-4}$, $N = 18,683,098$ are necessary, leading to a total time of 0.5 seconds per simulation. For comparison, this is similar to the previous experiment, albeit for a RMSE of $\epsilon = 5 \cdot 10^{-5}$, i.e. ten times smaller.

In turn, Figure 5 illustrates the results on a log scale for computing the MC estimate of the expected value of k_{eff} with $\mu = 0$ and $\sigma^2 = 1$. Due to the above reasoning, we use larger RMSE values equally spaced from $5 \cdot 10^{-2}$ to $5 \cdot 10^{-4}$. While the computational times are larger than in the previous experiment, we still observe a slope of the line for the cost roughly equal to -2.

Moreover, Figures 4 and 5 allow us to predict the time necessary for achieving a specific accuracy. For example, to reach the smallest RMSE observed in the first experiment, specifically $\epsilon = 5 \cdot 10^{-5}$, we anticipate that the second script would take approximately 50 seconds per simulation. Indeed, in this case we observed numerically that $N = 1,868,309,709$ samples would be needed, taking approximately 53 seconds per simulation. For another digit of accuracy, we expect a total of 530 seconds, while for $\epsilon = 5 \cdot 10^{-7}$, we predict that the algorithm would take 5300 seconds ≈ 1.5 hours for every simulation.

Finally, note that the Python implementation for both experiments is given in Appendix A.

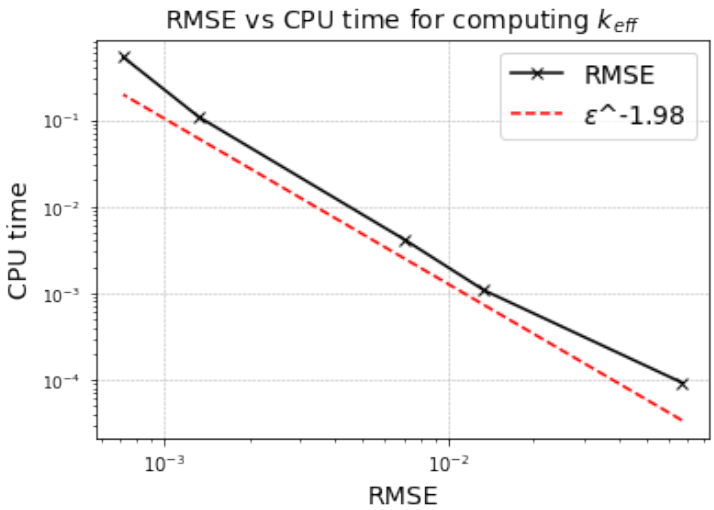


Figure 5: Performance plot on a log scale for $\mu = 0$ and $\sigma^2 = 1$ in 1D using Monte Carlo. The quantity of interest is the mean value of the effective conductivity.

5.2 2D example with random variable coefficient

Let us now consider the original PDE (1.1), with the domain restricted to $D = (0,1)^2$. We take $k \sim$ log-normal(μ, σ^2) again, and $f \equiv a$, with $a \in \mathbb{R}^*$ a constant. This yields a variation of Poisson's equation (since k is essentially a constant and $f \neq 0$):

$$-k\,\nabla^2 p(\mathbf{x}) = f(\mathbf{x}), \quad \mathbf{x} \in D. \tag{5.17}$$

This is now an elliptic PDE with a random coefficient. As in the ODE case, we impose Dirichlet boundary conditions that ensure the groundwater stream is from $x_1 = 0$ to $x_1 = 1$:

$$p|_{x_1=0} = 1, \quad p|_{x_1=1} = 0 \tag{5.18}$$

Additionally, we introduce Neumann boundary conditions which force the flow to remain (vertically) inside the domain, as follows:

$$\left.\frac{\partial p}{\partial \mathbf{n}}\right|_{x_2=0} = 0, \quad \left.\frac{\partial p}{\partial \mathbf{n}}\right|_{x_2=1} = 0. \tag{5.19}$$

Further, we follow the procedure described in Section 4.1 to bring Eq. (5.17) to its variational form, which can then be solved approximately by exploiting the FEniCS platform. In particular, substituting the Neumann conditions (5.19) in Eq. (4.8), we obtain the following equation:

$$F = \int_D k\nabla p \cdot \nabla v d\mathbf{x} - \int_D f\, v\, d\mathbf{x} = 0. \tag{5.20}$$

Since k is simply a random variable, a KL-expansion is not necessary here.

Hence, let us now concentrate on applying Monte Carlo to estimate the mean value of the quantity of interest $\hat{Q} := \mathbb{E}[p(7/15, 7/15)]$. While the MC estimator of $\hat{Q}$ is straightforward to compute, the RMSE, as defined in Eq. (2.13), is rather intricate to evaluate in the absence of an analytical solution. In fact, we are required to perform a number of supplementary steps in order to obtain an estimate for $e\left(\widehat{Q}^{\mathrm{MC}}_{M,N}\right)$.

First, recall the hypothesis at the end of Section 2.2, specifically, that $\mathbb{V}[Q_M]$ is approximately constant independent of M, with Q_M denoting the approximation of Q with M degrees of

freedom. In order to satisfy this assumption, we can compute the sample variance as follows:

$$\mathbb{V}[Q_M] \coloneqq \mathbb{E}\left[(Q_M - \mathbb{E}[Q_M])^2\right] \approx \frac{1}{N-1}\sum_{i=1}^{N}\left(Q_M^{(i)} - \frac{1}{N}\sum_{i=1}^{N} Q_M^{(i)}\right)^2, \tag{5.21}$$

where the division by $N-1$ instead of N in the first fraction is known as Bessel's correction [27]. Moreover, both N and M must be chosen large enough to ensure that the approximation in Eq. (5.21) is essentially exact, and, consequently, that the variance $\mathbb{V}[Q_M]$ is almost constant. The Python function which achieves this is `var_est_fun` given in Appendix B.

Second, the bias $(\mathbb{E}[Q_M - Q])^2$ in Eq. (2.13) cannot be calculated exactly either, since Q is an inaccessible random variable. In order to estimate it, let us suppose further that the descent in $|\mathbb{E}[Q_M - Q]|$ is monotonic when $M \geq M'$, for some integer $M' \in \mathbb{N}$, and that it satisfies:

$$|\mathbb{E}[Q_M - Q]| = C\,M^{-\alpha}, \quad \alpha > 0. \tag{5.22}$$

Following the derivation in (3.30) and using Q_{4M} in the extrapolation step yields:

$$\begin{aligned} |\mathbb{E}\left[Q_M - Q_{4M}\right]| &= |\mathbb{E}\left[Q_M - Q\right] + \mathbb{E}\left[Q - Q_{4M}\right]| \\ &= |CM^{-\alpha} - C(4M)^{-\alpha}| \\ &= |CM^{-\alpha}|\,|1 - 4^{-\alpha}| \\ &= |\mathbb{E}\left[Q_M - Q\right]|\,|1 - 4^{-\alpha}|. \end{aligned} \tag{5.23}$$

This can be further re-arranged to attain:

$$|\mathbb{E}\left[Q_M - Q\right]| = \left|\frac{\mathbb{E}\left[Q_M - Q_{4M}\right]}{1 - 4^{-\alpha}}\right|. \tag{5.24}$$

Since the numerator is now a computable quantity, we need only find α in Eq. (5.22) for a complete program. This can be achieved by estimating $|\mathbb{E}[Q_M - Q_{4M}]|$ on consecutively finer grids, that is, with progressively larger M, since, from Eq. (5.23):

$$|\mathbb{E}\left[Q_M - Q_{4M}\right]| = |C_\alpha| M^{-\alpha}, \tag{5.25}$$

where $C_\alpha = C(1 - 4^{-\alpha})$. Afterwards, both C_α and α can be recovered by determining the best linear fit through the aforementioned approximations on a log scale. An example of a Python script which accomplishes this is the `compute_alpha` function provided in Appendix B.

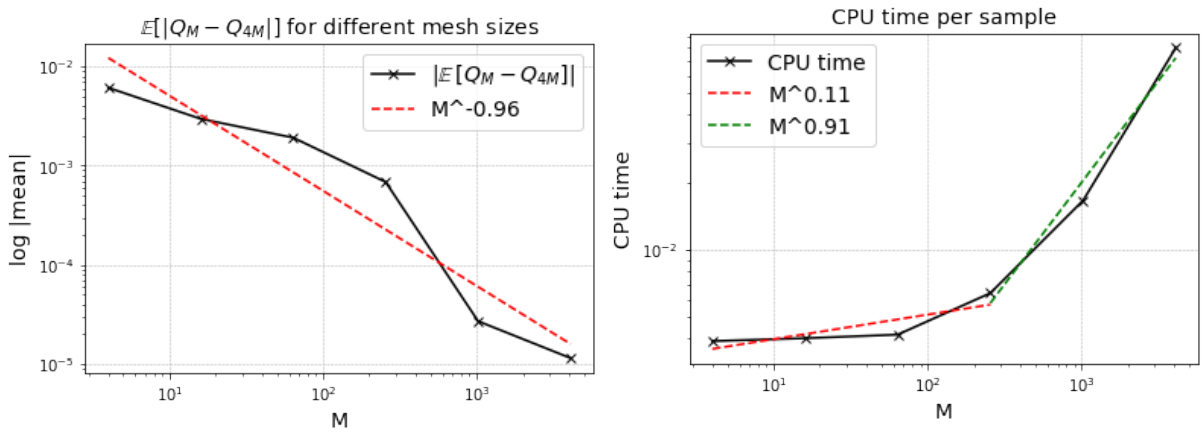
Additionally, recall from Section 2.2 that we anticipate the cost of achieving a RMSE of $\mathcal{O}(\epsilon)$ to reach $C_\epsilon\left(\hat{Q}_{M,N}^{\mathrm{MC}}\right) \lesssim \epsilon^{-2-\gamma/\alpha}$. Here, γ emerges from the definition of the cost of computing solely one sample $Q_M^{(i)}$ of Q_M, which satisfies $C\left(Q_M^{(i)}\right) \leq C_\gamma M^\gamma, \gamma > 0$. Hence, following the same reasoning as above, both γ and C_γ can be identified by measuring the cost of computing $Q_M^{(i)}$ for successively greater M and determining the line of best fit through the collected estimates on a log scale. In Python, this is implemented in the `compute_gamma` function in Appendix B.

For the problem at hand, we take $f(\mathbf{x}) \equiv 1$ in (5.17), and $\mu = 0$ and $\sigma^2 = 1$ for the log-normal distribution of k, analogous to the 1D case in Section 5.1. Further, we take $M = (2^n)^2$, $n = \{1, \ldots, 6\}$, so that the extrapolation values are $4M = \left(2^{n+1}\right)^2$, $n = \{1, \ldots, 6\}$. The quantity $|E[Q_M - Q_{4M}]|$ is then approximated with simple MC using a fixed number of samples N as:

$$|E[Q_M - Q_{4M}]| = \frac{1}{N}\left|\sum_{i=1}^{N} Q_M^{(i)} - Q_{4M}^{(i)}\right|, \tag{5.26}$$

where $Q_M^{(i)}$ and $Q_{4M}^{(i)}$ are computed with the same underlying sample of the random variable k.

Thus, Figure 6a shows the decay in $\log|\mathbb{E}[Q_M - Q_{4M}]|$, so that the resulting line has a slope of roughly -0.96, indicating $\alpha \approx 0.96$, while the y-intercept is around -3.07, suggesting $C_\alpha \approx 0.05$.



(a) $\log|\mathbb{E}[Q_M - Q_{4M}]|$ and its best linear fit.

(b) $C\left(Q_M^{(i)}\right)$ and its best linear fit.

Figure 6: Plots of α and γ on a log scale for $\mu = 0$ and $\sigma^2 = 1$ in 2D using Monte Carlo. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$.

Also in this framework, we quantify the cost of computing one sample $Q_M^{(i)}$ of Q_M by measuring the CPU time taken to estimate $Q_M^{(i)}$ over multiple executions of the program, subsequently averaging the results. This is to prevent erroneous outcomes emerging from just one realisation.

Accordingly, Figure 6b shows the growth of $C\left(Q_M^{(i)}\right)$ with M. Due to the shape of the graph, we fit two lines rather than solely one, initially through the first four points and afterwards through the last three. To recover a value for γ, we aggregate the two slopes by taking their mean, yielding $\gamma \approx 0.51$. Analogously, we identify $C_\gamma \approx 0.0013$.

Moreover, with the same hyperparameters as before and using $N = 10^6$ samples with $M = 64^2$ degrees of freedom, we obtain an estimate for the sample variance of about 0.062. Consequently, the values for α, C_α and $\mathbb{V}[Q_M]$ can now be used to determine approximate values for the constants N and M required to achieve a RMSE of ϵ, for given ϵ, by applying Eq. (2.15) and (2.16).

Correspondingly, Figure 7 displays the results on a log scale for $k \sim$ log-normal$(0, 1)$ and $f(\mathbf{x}) \equiv 1$. We adopt equally spaced RMSE values from $\epsilon = 10^{-1}$ to $\epsilon = 2.5 \cdot 10^{-3}$, for which the largest emerging value of M is approximately 49 (it is for this reason that we only use $M = (2^n)^2$, $n \in \{1, \ldots, 6\}$ when determining α and γ). For each such value of ϵ, we evaluate the CPU time taken to achieve $e\left(\hat{Q}_{M,N}^{\mathrm{MC}}\right) < \epsilon$. Subsequently, we determine the line of best fit on a log scale through the derived points $\left(\epsilon, C_\epsilon\left(\hat{Q}_{M,N}^{\mathrm{MC}}\right)\right)$. The slope of the aforementioned line should be at most $-2 - \gamma/\alpha$, so that $C_\epsilon\left(\hat{Q}_{M,N}^{\mathrm{MC}}\right) \lesssim \epsilon^{-2-\gamma/\alpha}$ as theoretically determined in Section 2.2. Indeed, with $\gamma \approx 0.51$ and $\alpha \approx 0.96$, we have $\gamma/\alpha = 0.53125$, which is precisely what features in Figure 7. An example of a Python function which generates the MC estimate of $\mathbb{E}[p(7/15, 7/15)]$ with $e\left(\hat{Q}_{M,N}^{\mathrm{MC}}\right) < \epsilon$ for given ϵ is `MC_simulation` given in Appendix B.

Let us now focus on computing the MLMC estimate of $\mathbb{E}[Q]$. In order to achieve this, the first step is selecting the approximation levels so that they satisfy Eq. (3.13). Here, we simply choose $M_l = 4M_l$, $l = 1, \ldots, 7$. Next, as mentioned in Section 3.2, M_0 must be chosen large enough to yield a sufficient amount of detail on the behaviour of the PDE solution. Since k is simply a log-normal random variable, the solution to Eq. (5.17) does not exhibit large oscillations. Thus, we take the coarsest mesh to have $M_0 = 2^2$, so that $M_l = \left(2 \cdot 2^l\right)^2$, $l = 0, \ldots 7$.

Further, Q_M and Q_{4M} in Eq. (5.23) are identical to Q_{M_L} and $Q_{M_{L-1}}$ in Eq. (3.30). This can be seen by simply applying the definition of the MLMC levels above. As a result, the procedures for finding α in the MC and MLMC methods are analogous, and, moreover, the value of α is not altered. The only difference is that here we include two additional values for M, $M = (2^7)^2$ and $M = (2^8)^2$, which should not, and, indeed, does not affect the convergence rate, fact illustrated

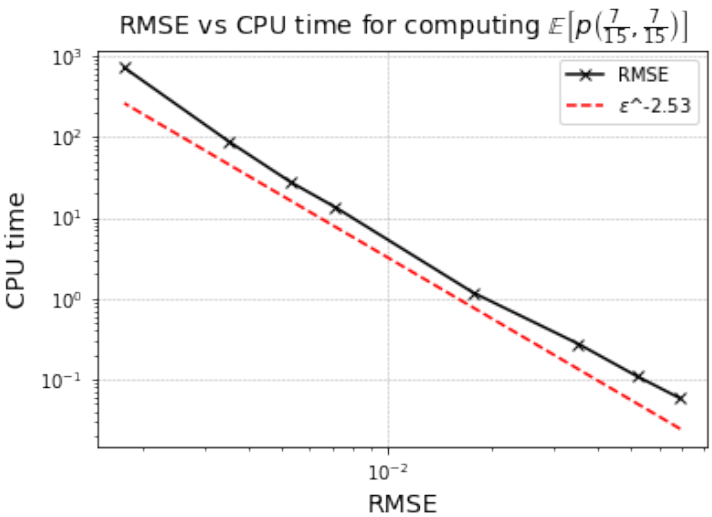


Figure 7: Performance plot on a log scale for $\mu = 0$ and $\sigma^2 = 1$ in 2D using Monte Carlo. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$.

in Figure 8a. Note that the value of α is 0.97, approximately equal to $\alpha \approx 0.96$ in the MC case. Additionally, we observe that the two graphs in Figures 6a and 8a exhibit the same behaviour - slow convergence at the beginning, followed by faster convergence as M increases.

What is more, we can determine the theoretical bound for the computational complexity of the MLMC algorithm applied to the current application by exploiting Theorem 3.1. To this end, we must also numerically determine the values of β and γ, as defined in the preceding theorem.

We shall initially study β, which is the convergence rate of the sample variance of Y_l. First, we assume that the decay in $\mathbb{V}[Y_l]$ is monotonic when $M_l \geq M'$, for some $M' \in \mathbb{N}$, and satisfies:

$$\mathbb{V}[Y_l] = C_\beta M_l^\beta \iff \mathbb{V}[Q_l - Q_{l-1}] = C_\beta M_l^\beta. \tag{5.27}$$

To compute the sample variance $\mathbb{V}[Y_l]$, we apply the same formula as in Eq. (5.21), yielding:

$$\begin{aligned}
\mathbb{V}[Y_l] &:= \mathbb{E}\left[(Y_l - \mathbb{E}[Y_l])^2\right] \\
&\approx \frac{1}{N_l - 1} \sum_{i=1}^{N_l} \left(Y_l^{(i)} - \frac{1}{N_l} \sum_{i=1}^{N_l} Y_l^{(i)} \right)^2 \\
&= \frac{1}{N_l - 1} \sum_{i=1}^{N_l} \left(\left(Q_l^{(i)} - Q_{l-1}^{(i)} \right) - \frac{1}{N_l} \sum_{i=1}^{N_l} \left(Q_l^{(i)} - Q_{l-1}^{(i)} \right) \right)^2,
\end{aligned} \tag{5.28}$$

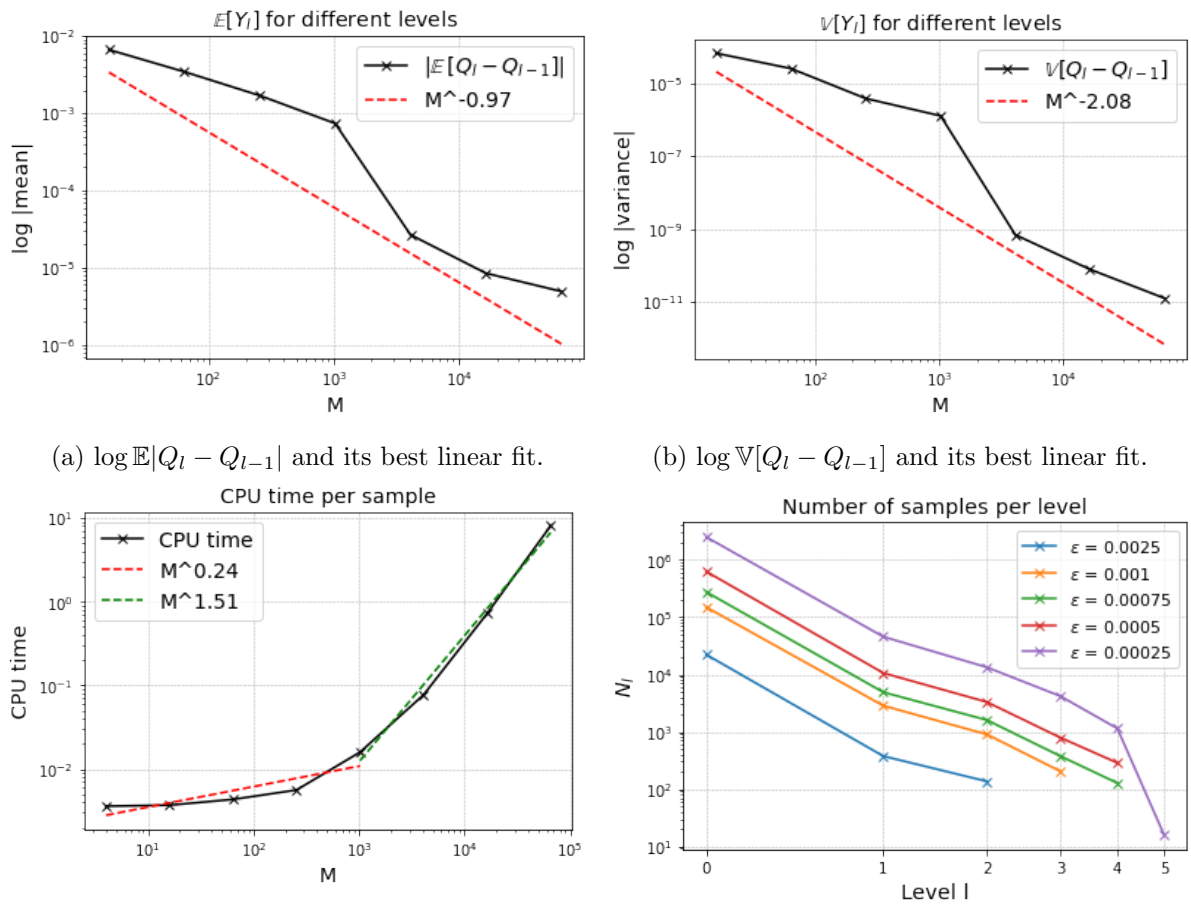
where $Q_l^{(i)}$ and $Q_{l-1}^{(i)}$ are generated using the same underlying sample of k.

Hence, following the same procedure used for approximating α, we estimate $\mathbb{V}[Y_l]$ for $l = 1, \ldots, 7$, and subsequently determine the line of best fit through $(M_l, \mathbb{V}[Y_l])$ on a log scale, whose slope and intercept yield β and C_β, respectively. A Python function which implements the described procedure for approximating β is `compute_beta` in Appendix C.

On this account, Figure 8b illustrates the decline in $\log \mathbb{V}[Y_l]$ with level l, $l = 1, \ldots, 7$. Here, the slope of the resulting line is roughly -2.08, indicating $\beta \approx 2.08$. Moreover, note that the values of $\mathbb{V}[Y_l]$ are nearly twice the expectation estimates in Figure 8a, which leads to $\beta \approx 2\alpha$.

In the same manner, let us now focus on γ. In particular, to quantify the cost of computing one sample $Q_{M_l}^{(i)}$ on level l, we assume that:

$$C\left(Q_{M_l}^{(i)}\right) = C_\gamma M_l^\gamma. \tag{5.29}$$



(a) $\log \mathbb{E}|Q_l - Q_{l-1}|$ and its best linear fit.

(b) $\log \mathbb{V}[Q_l - Q_{l-1}]$ and its best linear fit.

(c) $C\left(Y_l^{(i)}\right)$ and its best linear fit.

(d) Number of samples for different ϵ.

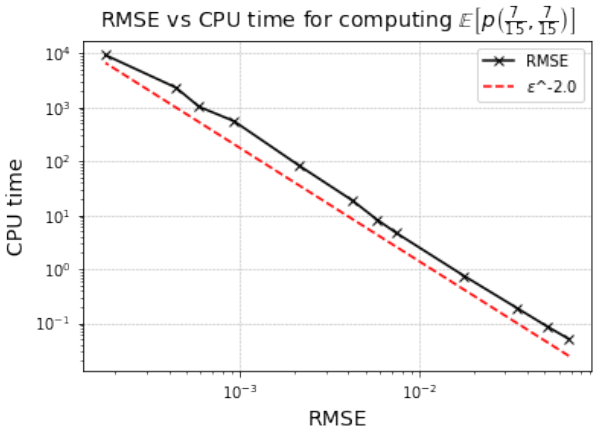
Figure 8: Plots of α, β and γ on a log scale, and of number of samples N_l necessary per level for $\mu = 0$ and $\sigma^2 = 1$ in 2D using Multilevel Monte Carlo with $l = 0, \ldots, 7$. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$.

Hence, for each M_l, $l = 0, \ldots, 7$, we measure the CPU time taken to estimate $Q_l^{(i)}$. Similarly as before, we do this over multiple executions of the algorithm and then average the results. Afterwards, we find the line of best fit through the points $\left(M_l, C\left(Q_{M_l}^{(i)}\right)\right)$ on a log scale, whose slope and intercept offer an estimate for γ and C_γ, respectively.

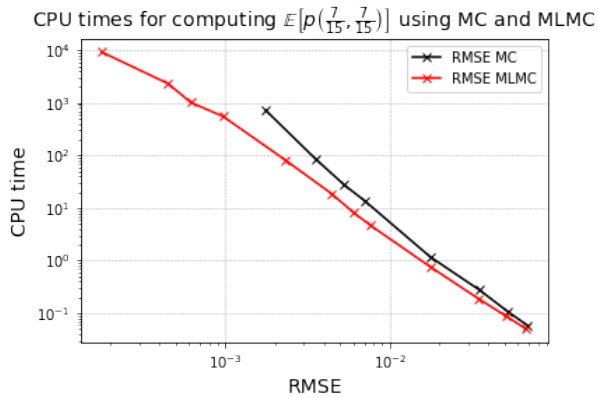
With this purpose, Figure 8c depicts the increase in $C\left(Q_{M_l}^{(i)}\right)$ with level l, $l = 0, \ldots, 7$. Analogous to the MC case, due to the shape of the graph we fit two lines: one through the first five points and, subsequently, one through the last four. The former has a slope of 0.24, while the latter features a slope of 1.51. Averaging the two values, we obtain $\gamma \approx 0.875$.

Unlike α, notice that the value of γ is, in fact, altered by including two larger values for M, namely $M = (2^7)^2$ and $M = (2^8)^2$. Specifically, it increases from $\gamma \approx 0.51$ in the MC example to $\gamma \approx 0.875$ here. This is to be expected since the computational complexity of generating one sample $Q_{M_l}^{(i)}$ on level l originates mainly from numerically solving the PDE (5.17) with M_l degrees of freedom. On relatively coarse grids, for instance with $M = 2^2$ or even $M = 32^2$ degrees of freedom, the cost for approximating the pressure p at the prescribed gridpoints, which is essentially the cost of solving an $M \times M$ linear system, is rather small. Naturally, the computational complexity increases much more rapidly if smoother approximations are intended, such as with $M = 128^2$ or $M = 256^2$ degrees of freedom.

Finally, Figure 8d illustrates the number of samples N_l needed per level l for different accuracies ϵ. For the RMSE, we use equally spaced values between $\epsilon = 2.5 \cdot 10^{-3}$ and $\epsilon = 2.5 \cdot 10^{-4}$. Further, N_l is determined using the formula in Eq. (3.23), where the cost C_l of estimating one sample $Y_l^{(i)}$ is required. Since $Y_l^{(i)} := Q_l^{(i)} - Q_{l-1}^{(i)}$ and we have already established the cost of



(a) Performance plot for MLMC.



(b) Performance plot for MLMC compared to MC.

Figure 9: Performance plots on a log scale for $\mu = 0$ and $\sigma^2 = 1$ in 2D using Multilevel Monte Carlo and Monte Carlo. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$.

approximating one sample $Q_{M_l}^{(i)}$, we calculate C_l as follows:

$$C_l = C_\gamma M_l^\gamma + C_\gamma M_{l-1}^\gamma. \tag{5.30}$$

Additionally, Figure 8d also emphasizes that most of the computational effort is clustered around the cheaper levels. For instance, to achieve an accuracy of $\epsilon = 10^{-3}$ in comparison to an RMSE of $\epsilon = 2.5 \cdot 10^{-3}$, a further level is indeed required. However, the number of samples on level zero increases nearly sevenfold, from $N_0 = 22,140$ to $N_0 = 149,280$, while the numbers of samples on the last level for both values of ϵ are nearly equal. Moreover, consider now the number of samples necessary for $e\left(\hat{Q}_M^{\text{MLMC}}\right) < 2.5 \cdot 10^{-4}$. Specifically, this starts at $N_0 = 2,500,721$ on level zero, and reaches $N_5 = 16$ on the final level. This is, in fact, a feature of the problem considered here. As we shall see in Section 5.3, the decline in the number of samples is not as considerable if k is modelled as a random field rather than a random variable.

Further, it is straightforward to see that $\gamma < \beta$ in this case. Then, according to Theorem 3.1, the theoretical cost of the MLMC algorithm should satisfy $C\left(\hat{Q}_M^{\text{MLMC}}\right) \lesssim \epsilon^{-2}$. Accordingly, Figure 9a shows on a log scale the CPU time taken to reach a RMSE less than ϵ, for given ϵ, using the MLMC estimator. We adopt again $k \sim \text{log-normal}(0, 1)$ and $f(x) \equiv 1$, and we exploit the same approximation levels as above. Additionally, we let ϵ take equally spaced values from 10^{-1} to $2.5 \cdot 10^{-4}$, so that we cover a slightly wider spectrum than the one explored with simple MC. As a result, the regression line through the points $\left(\log \epsilon, \log C_\epsilon\left(\hat{Q}_M^{\text{MLMC}}\right)\right)$ has a slope approximately equal to -2, suggesting $C_\epsilon\left(\hat{Q}_M^{\text{MLMC}}\right) \lesssim \epsilon^{-2}$, as anticipated. A Python program which generates an MLMC estimate of $\mathbb{E}[Q]$ is `MLMC_simulation` in Apendix C.

Before proceeding to the next section, let us compare the performance of the MC and MLMC algorithms. To this end, in Figure 9b we superimpose the graphs showing the computational complexity of the two methods. The resulting figure indicates that not only does the MLMC approach lead to significant cost savings, but also that the cost associated with the standard MC technique increases more swiftly than the one of the MLMC estimator as $\epsilon \to 0$. Even though the line corresponding to the MC algorithm ceases at $\epsilon = 2.5 \cdot 10^{-3}$, we can still use it to predict the time it would take to compute a MC estimate with RMSE less than $\epsilon = 2.5 \cdot 10^{-4}$. In particular, $N = 198,075$ samples with $M = 140^2$ degrees of freedom would be required. With roughly 1.3 seconds necessary per sample, the total CPU time would be approximately 72 hours, compared to around 2.6 hours necessary for the MLMC algorithm. Given the analysis above, this is to be expected and it only reinforces the advantages of MLMC over simple MC.

5.3 2D example with random field coefficient

Let us return to the PDE (1.1) on the same domain as above, $D = (0,1)^2$. However, let us now consider $k(\mathbf{x}, \omega)$ to be a random field whose covariance function is given as in Eq. (4.13) with correlation length ρ and variance σ^2. In other words, we study the following equation:

$$-\nabla \cdot (k(\mathbf{x}, \omega)\nabla p(\mathbf{x})) = f(\mathbf{x}), \quad \mathbf{x} \in D. \tag{5.31}$$

This is still an elliptic PDE with a random coefficient. Additionally, we let $f(\mathbf{x}) \equiv a$, with $a \in \mathbb{R}^*$ a constant, and impose the same Dirichlet and Neumann conditions (5.18) and (5.19) as in Section 5.3. As a result, this does not change the variational formulation (5.20).

To implement a truncated KL-expansion of the random field coefficient $k(\mathbf{x}, \omega)$ with a given number of modes K, we follow the steps in Section 4.2. First, we compute K roots of Eq. (4.24) $\{w_n\}_{n=1}^K$. Second, we use these to determine K eigenvalues $\{\lambda_n\}_{n=1}^K$ and eigenfunctions $\{f_n(\mathbf{x})\}_{n=1}^K$ of the covariance operator with kernel function (4.13), using the formulae in Eq. (4.22) and (4.23). Last, we build an $(m+1)^2 \times K$ matrix A, whose entries are as follows:

$$A_{i,j} = \sqrt{\lambda_j} f_j(\mathbf{x}_i), \quad i = 1, \dots (m+1)^2;\ j = 1, \dots, K, \tag{5.32}$$

where m denotes the mesh size used in discretising the equation (5.31) and $\{\mathbf{x}_i\}_{i=1}^{(m+1)^2}$ are the grid points of the discretisation mesh (in Python, these can be recovered by exploiting the `coordinates` function in FEniCS). Subsequently, to generate a sample from $k(\mathbf{x}, \omega)$, we need only sample K IID standard Gaussian random variables $\{\xi_n\}_{n=1}^K$, so that:

$$\mathbf{k} = \exp(A\boldsymbol{\xi}) \tag{5.33}$$

is a $(m+1)^2$ column vector whose entries represent the value of the random field $k(\mathbf{x}, \omega)$ at each grid point $\{\mathbf{x}_i\}_{i=1}^{(m+1)^2}$. Here, $\boldsymbol{\xi} \in \mathbb{R}^K$ is a column vector whose elements are the former random variables $\{\xi_n\}_{n=1}^K$. This can now be integrated with the discretisation of the variational problem (5.20) to obtain a complete solution algorithm. An example of constructing the truncated KL-expansion and solving the resulting PDE is given in Appendix D. This is based on the MATLAB implementation developed by Teckentrup for the numerical experiments in [7].

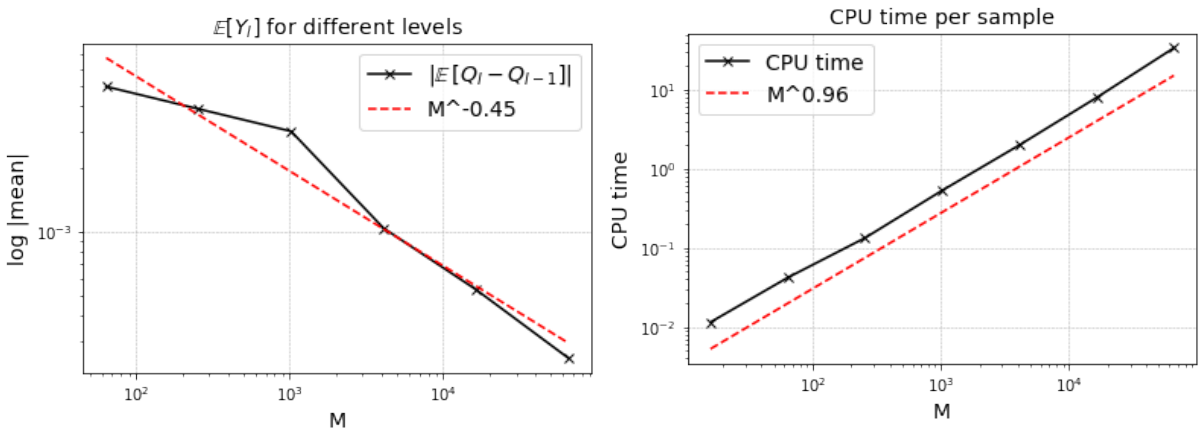
5.3.1 Covariance function with correlation length 0.3 and variance 1

With this in mind, let us now investigate how both the MC and MLMC estimators perform when applied to the above problem. We retain $f(\mathbf{x}) \equiv 1$ and the same quantity of interest as before. In addition, we use $\rho = 0.3$ and $\sigma^2 = 1$ for the correlation length and variance of the covariance function of $k(\mathbf{x}, \omega)$, respectively. At the same time, we exploit the observations in Section 4.2 to obtain $K = 1,400$ modes necessary for the truncated KL-expansion of k and $M_0 = 4^2$ degrees of freedom on the first MLMC level, slightly larger than $(1/\rho)^2$, as required.

An aspect worth mentioning is that the procedures for determining $\mathbb{V}[Q_M]$, α, β, and γ are identical to those described in the previous section. Consequently, we omit specifying most of the implementational details and directly outline the results.

Thus, Figure 10a shows the decline in $\log|\mathbb{E}[Q_l - Q_{l-1}]|$, whose slope is around -0.45, suggesting $\alpha \approx 0.45$. Conversely, Figure 10b illustrates on a log scale the growth of the cost of computing one sample $C\left(Q_{M_l}^{(i)}\right)$ with level l. Here, the slope of the regression line is 0.96, indicating $\gamma \approx 0.96$. It is important to note that the values of M employed in both the MC and MLMC methods for estimating α and γ are the same. Consequently, the approximations of α and γ are almost equal for both algorithms, albeit with different constants C_α and C_γ. For this reason, we avoid including the graphs which portray the decay in $\log|\mathbb{E}[Q_M - Q_{4M}]|$ and increase in $\log C\left(Q_M^{(i)}\right)$ corresponding to the MC estimator, too.

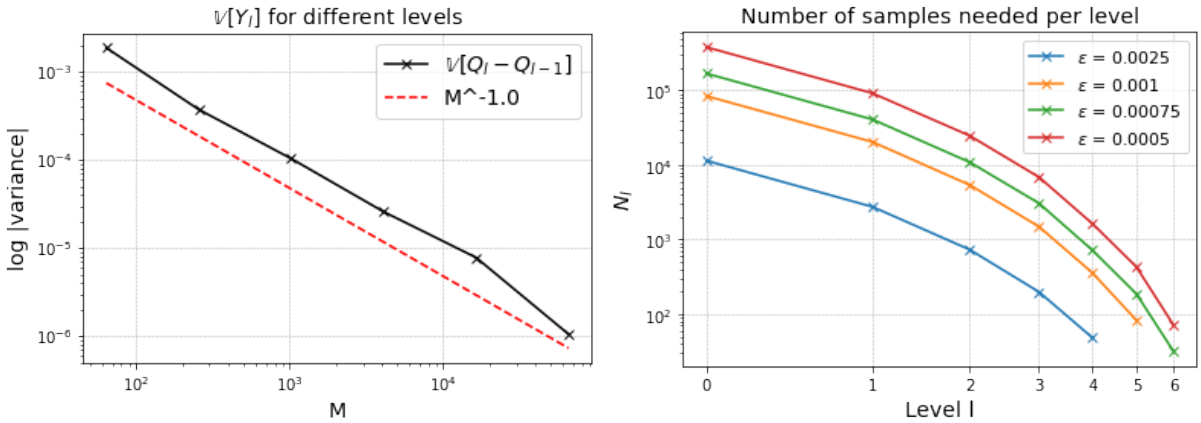
Next, Figure 11 is exclusively related to the MLMC simulations. In particular, Figure 11a highlights the reduction in $\mathbb{V}[Y_l]$ with level l on a log scale. From the example in Section 5.2,



(a) $\log|\mathbb{E}[Q_l - Q_{l-1}]|$ and its best linear fit.

(b) $C\left(Q_{M_l}^{(i)}\right)$ and its best linear fit.

Figure 10: Plots of α and γ on a log scale for $\rho = 0.3$ and $\sigma^2 = 1$ in 2D using Multilevel Monte Carlo with $l = 0, \ldots, 6$. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$.



(a) $\log \mathbb{V}[Q_l - Q_{l-1}]$ and its best linear fit.

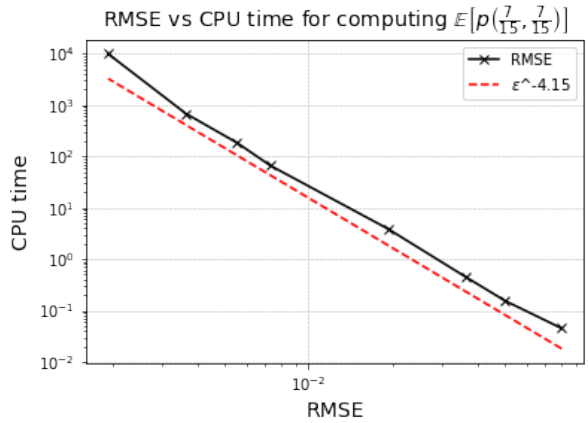
(b) Number of samples per level for different ϵ.

Figure 11: Plots of β on a log scale, and of the number of samples N_l necessary per level for $\rho = 0.3$ and $\sigma^2 = 1$ in 2D using Multilevel Monte Carlo with $l = 0, \ldots, 6$. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$.

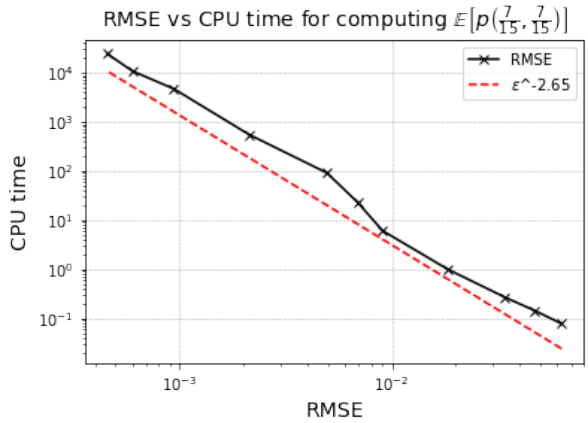
we might expect to find $\beta \approx 2\alpha$, and, indeed, the slope of the line for $\log \mathbb{V}[Y_l]$ is roughly -1, implying $\beta \approx 1 \approx 2\alpha$. On the other hand, the number of samples N_l needed per level for different accuracies ϵ is depicted in Figure 11b. Here, we use four equally spaced values for ϵ, from $\epsilon = 2.5 \cdot 10^{-3}$ to $\epsilon = 5 \cdot 10^{-4}$. While significantly more samples are required on the first level compared to the last one, the decline from N_L to N_0 is not as dramatic as in the example in the prior section. For instance, to achieve a RMSE less than $7.5 \cdot 10^{-4}$, all six levels are required, with $N_0 = 168,850$ samples on the first level and $N_6 = 32$ samples on the final level. This is, in fact, a feature of the current problem. Particularly, $\gamma \approx \beta$ here suggests that the cost of computing one sample $Q_{M_l}^{(i)}$ increases as rapidly as the variance $\mathbb{V}[Y_l]$ decreases, so that the computational effort is uniformly spread across the levels of the MLMC estimator.

Finally, recall that the sample variance $\mathbb{V}[Q_M]$ is required for estimating the constants N and M which ensure that the MC estimator attains a RMSE smaller than ϵ, for given ϵ. Here, we use $N = 10^4$ samples with $M = 64^2$ degrees of freedom, yielding $\mathbb{V}[Q_M] \approx 0.0042$.

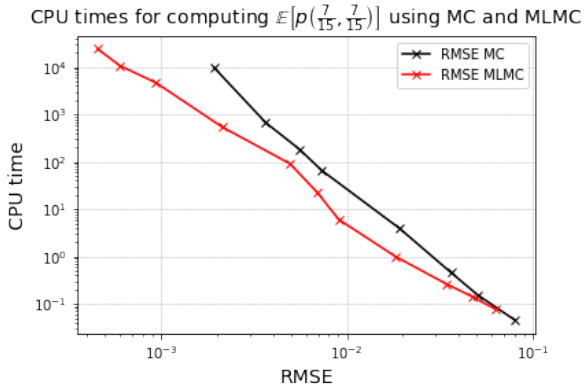
Having determined all the necessary parameters, let us now study the performance of the two methods. To this end, Figure 12a illustrates the CPU time necessary for the MC estimator to reach a RMSE less than ϵ, where we select equally spaced values for ϵ, between $\epsilon = 10^{-1}$



(a) Performance plot for MC.



(b) Performance plot for MLMC.



(c) Performance plot for MC compared to MLMC.

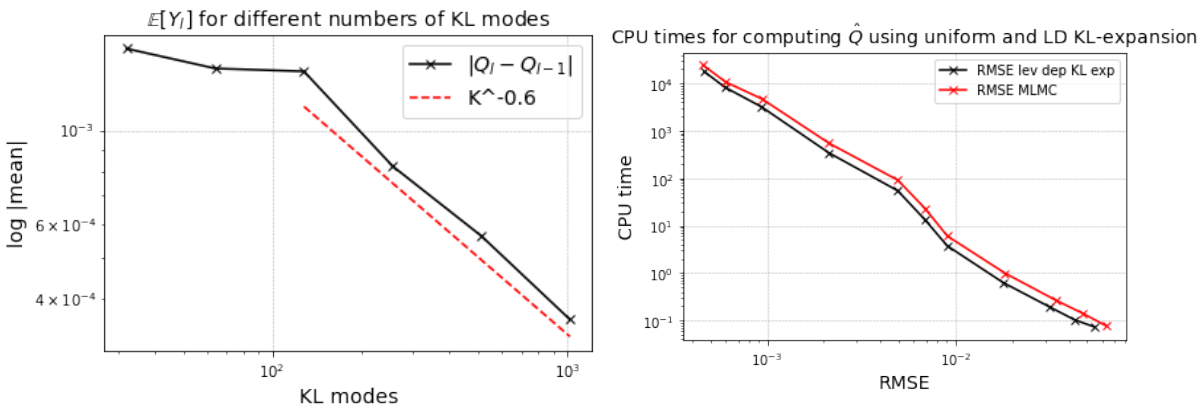
Figure 12: Performance plots on a log scale for $\rho = 0.3$ and $\sigma^2 = 1$ in 2D using Monte Carlo and Multilevel Monte Carlo. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$.

and $\epsilon = 2.5 \cdot 10^{-3}$. Accordingly, the regression line through the resulting points $\left(\epsilon, C_\epsilon\left(\widehat{Q}^{\mathrm{MC}}_{M,N}\right)\right)$ has a slope of roughly -4.15. From Eq. (2.19), this is precisely what we would expect, since $\gamma/\alpha \approx 2.13$ and so, theoretically, $C_\epsilon\left(\widehat{Q}^{\mathrm{MC}}_{M,N}\right) \lesssim \epsilon^{-4.13}$.

In turn, Figure 12b highlights the performance of the MLMC method, for a slightly wider range of ϵ. In particular, we use RMSE values equally spaced from $\epsilon = 10^{-1}$ to $\epsilon = 5 \cdot 10^{-4}$ and quantify the CPU time required for the MLMC estimator to reach $e\left(\hat{Q}^{\mathrm{MLMC}}_M\right) < \epsilon$. As it can be seen in Figure 12b, the slope of the resulting regression line (obtained as in the MC case) is approximately -2.65. Again, this is what Theorem 3.1 indicates. Specifically, since we are in the $\beta \approx \gamma$ case, we anticipate $C\left(\hat{Q}^{\mathrm{MLMC}}_M\right) \lesssim \epsilon^{-2}(\log \epsilon)^2$, which is roughly what we observe.

Finally, Figure 12c emphasises the disparity between the efficiency of the simple MC and MLMC estimators. Clearly, the cost savings prompted by the latter approach are more compelling than in the previous example. For comparison, the MC method requires about 10^4 seconds to reach an accuracy close to $2 \cdot 10^{-3}$, while the MLMC technique takes just over $2 \cdot 10^4$ seconds to achieve an error almost fivefold smaller. Further, to reach an RMSE as small as $5 \cdot 10^4$, MLMC takes roughly $3 \cdot 10^4$ seconds, while standard MC involves computing $N = 33,379$ samples with $M = 314^2$ degrees of freedom. The largest M that FEniCS can tolerate is $M = 256^2$, and hence we cannot measure the CPU time exactly. Nonetheless, using the slope and intercept of the regression line in Figure 12a, we estimate that this would require around 145 hours.

Let us now compare the performance of the MLMC estimator with a uniformly truncated KL-expansion of the random field coefficient, offered above, and with a level dependent truncation of the KL-expansion of $k(\mathbf{x}, \omega)$. To this end, we must first establish a relationship between K_l



(a) $\log|\mathbb{E}[Q_{M_l,K_l} - Q_{M_l,K_{l-1}}]|$ and its best linear fit.

(b) Uniform and LD truncated KL-expansion.

Figure 13: Plot of δ on a log scale and performance plot of uniform versus level dependent truncated KL-expansion of random field coefficient for $\rho = 0.3$ and $\sigma^2 = 1$ in 2D using Multilevel Monte Carlo with $l = 0, \ldots, 6$. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$.

and M_l, as described in Section 4.2. In particular, recall Eq. (4.37), which suggests that we need only determine α, δ, and their corresponding constants, so that, subsequently:

$$K_l = \frac{C_\alpha}{C_\delta} M_l^{\frac{\alpha}{\delta}}, \quad l = 0, \ldots, L. \tag{5.34}$$

Accordingly, we make use of Eq. (4.36), which relates the mean error of the quantity of interest estimated on two consecutive levels with the rates of convergence α and δ. Since there are two terms contributing to the error, we must distinguish between them by successively fixing one to be large and varying the other. For instance, we select $K_l = 1,400$ and vary M_l, which generates an estimate for α, and then set $M_l = 128$ and vary K_l, yielding an approximate value for δ.

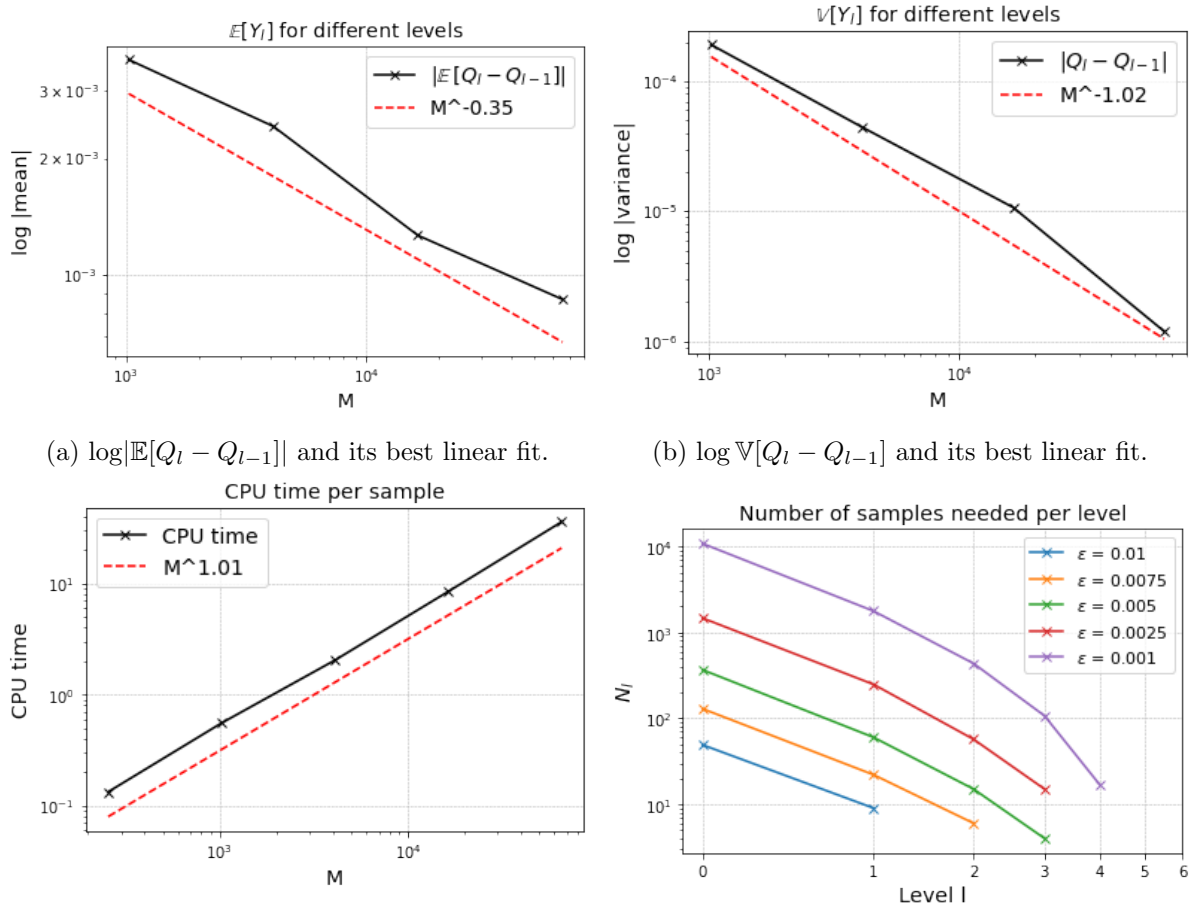
Correspondingly, Figure 13a shows the decay of $\log|\mathbb{E}[Q_{M_l,K_l} - Q_{M_l,K_{l-1}}]|$. As pointed in [7], the eigenvalues of the covariance function of $k(\mathbf{x}, \omega)$ exhibit a pre-asymptotic stage (whose exact length depends on ρ). This corresponds to the pre-asymptotic phase we observe in Figure 13a. Once we include more than 100 modes in the KL-expansion of k, we notice that the mean error decreases at rate $\delta \approx 0.6$. Specifically, we recover the value of δ by estimating the slope of the regression line through $\left(K_l, |\mathbb{E}[Q_{M_l,K_l} - Q_{M_l,K_{l-1}}]|\right)$ on a log scale.

Further, the graph corresponding to α is identical to the one in Figure 10a. This is because the latter was also constructed including $K = 1,400$ KL-modes and ranging M_l between $M_0 = 4^2$ and $M_L = 256^2$. Since $\mathbb{E}[Y_l]$ does not change with this choice of α and δ, the value of β is not altered either. However, γ is decreases to 0.93, which could be a result of using less KL-modes per level, consequently reducing the cost of computing one sample $Q^{(i)}_{M_l,K_l}$ on levels $l = 0, \ldots, L-1$.

The above comments are confirmed in Figure 13b. In particular, we note that the slope of the MLMC cost is not modified by employing the level dependent truncation, in other words $C_\epsilon\left(\hat{Q}^{\text{MLMC}}_{M,K}\right) \approx C \cdot \epsilon^{-2.65}$. However, the constant C is smaller, leading to lower costs that grow at the same rate. To gain an understanding of the cost savings achieved, suppose, for instance, we wish to reach an accuracy of $\epsilon = 5 \cdot 10^{-4}$. The MLMC algorithm with $K = 1,400$ across all levels takes 6.75 hours, while using the level dependent truncation accomplishes this in 5 hours.

5.3.2 Covariance function with correlation length 0.1 and variance 1

Let us now consider a computationally more onerous example. Specifically, we explore the case in which the covariance function of the random field coefficient has the same variance as before, $\sigma^2 = 1$, albeit a smaller correlation length, $\rho = 0.1$. The essential adjustment this prompts is that we must now adopt a considerably larger degree of approximation on the coarsest level



(a) $\log|\mathbb{E}[Q_l - Q_{l-1}]|$ and its best linear fit.

(b) $\log \mathbb{V}[Q_l - Q_{l-1}]$ and its best linear fit.

(c) $C\left(Q_{M_l}^{(i)}\right)$ and its best linear fit

(d) Number of samples for different ϵ.

Figure 14: Plots of α, β and γ on a log scale, and the number of samples N_l necessary per level for $\rho = 0.1$ and $\sigma^2 = 1$ in 2D using Multilevel Monte Carlo with $l = 0, \ldots, 4$. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$.

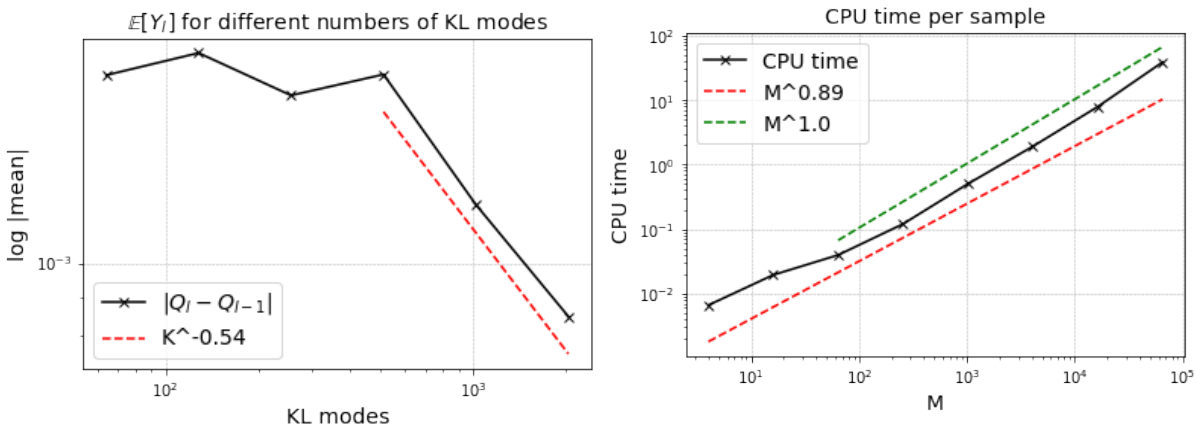
when applying MLMC, namely $M_0 = 16^2$. Further, we now use $K = 2,500$ modes in the KL-expansion of $k(\mathbf{x}, \omega)$, so that 95% of the variance is captured, according to Eq. (4.33).

A direct consequence of employing a random field with a smaller correlation length is a lower mean convergence rate of the approximation of Q to its true value. Correspondingly, Figure 14a shows the decay in $|\mathbb{E}[Q_l - Q_{l-1}]|$ on a log scale with level l. As illustrated, the slope of the resulting regression line through $(M_l, \mathbb{E}[Y_l])$ is approximately -0.35, so that $\alpha \approx 0.35$. This is the same for the MC simulations, as well as for the MLMC algorithm with a LD truncation of the KL-expansion of $k(\mathbf{x}, \omega)$.

In similar fashion, Figure 14b illustrates the decline in $\mathbb{V}[Q_l - Q_{l-1}]$ on a log scale. The line of best fit through the generated points $(M_l, \mathbb{V}[Y_l])$, has a slope of -1.02, suggesting $\beta \approx 1.02$. Clearly, we no longer have $\beta \approx 2\alpha$, which could be a consequence of the low regularity of the functional considered here, especially for the small correlation length $\rho = 0.1$. Again, β is not altered when using a level dependent truncation of the KL-expansion of $k(\mathbf{x}, \omega)$.

In addition, the growth of the cost of computing one sample $Q_{M_l}^{(i)}$ with $K = 2,500$ KL-modes is displayed in Figure 14c. Here, the best linear fit through $\left(M_l, C\left(Q_{M_l}^{(i)}\right)\right)$ has a slope of 1.01 on a log scale, indicating $\gamma \approx 1.01$, which is roughly equal to $\gamma \approx 0.96$ in the $\rho = 0.3$ case with $K = 1,400$ KL-modes. This is to be expected, since the main contribution to the cost of computing $Q_{M_l}^{(i)}$ originates from solving the PDE (5.31), rather than sampling from $k(\mathbf{x}, \omega)$. The same value of γ is established for the standard MC simulations.

Equally important, Figure 14d highlights the number of samples N_l needed per level l for different accuracies ϵ. Certainly, we observe the same behaviour as in the prior experiments,



(a) $\log|\mathbb{E}[Q_{M_l,K_l} - Q_{M_l,K_{l-1}}]|$ and its best linear fit.

(b) $C\left(Q^{(i)}_{M_l,K_l}\right)$ and its best linear fit.

Figure 15: Plots of δ and γ on a log scale for $\rho = 0.1$ and $\sigma^2 = 1$ in 2D using Multilevel Monte Carlo with a level dependent truncation of KL-expansion of $k(\mathbf{x}, \omega)$ and $l = 0, \ldots, 7$. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$.

namely a significant dip in N_l as l increases. However, we are restricted in the savings that this could induce, due to the rather large value of M necessary on level zero. For instance, to achieve an accuracy of only 10^{-3}, the MLMC algorithm employs all five levels, albeit with a relatively small number of samples on each level, taking around 80 minutes to execute.

In a similar manner to Section 5.3.1, the potential benefits that the MLMC estimator can accomplish are enhanced by exploiting the level dependent truncation of the KL-expansion of $k(\mathbf{x}, \omega)$. Accordingly, Figure 15a depicts the reduction in $|\mathbb{E}[Q_{M_l,K_l} - Q_{M_l,K_{l-1}}]|$, where we notice again the pre-asymptotic phase corresponding to the one of the eigenvalues (4.32). Afterwards, we estimate δ by determining the slope of the regression line through $\left(K_l, |\mathbb{E}[Q_{M_l,K_l} - Q_{M_l,K_{l-1}}]|\right)$ on a log scale. This yields $\delta \approx 0.54$, so that $K_l \eqsim M_l^{0.35/0.54}$ from Eq. (5.34).

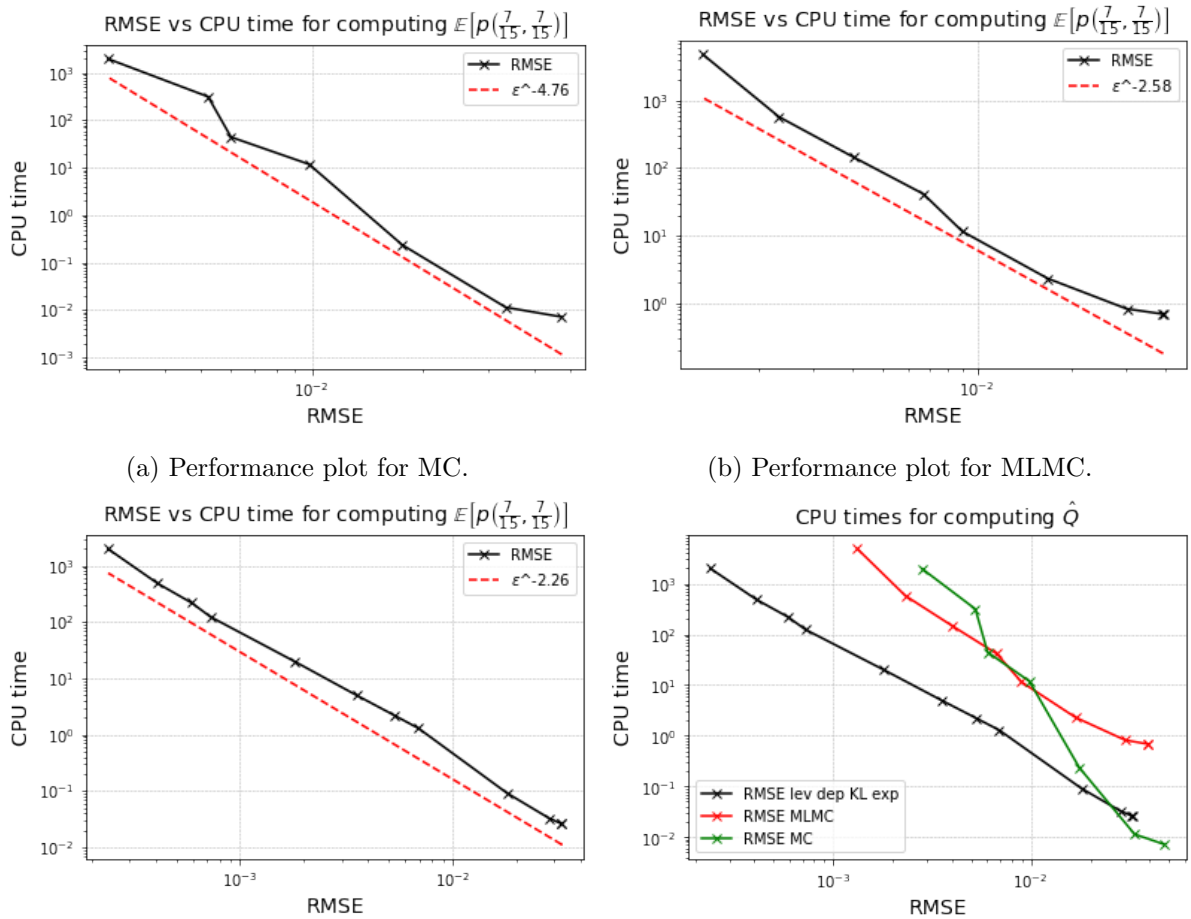
Further, with the LD truncation of the KL-expansion of the random field coefficient, the choice of M_0 is unrestricted. For instance, taking $M_0 = 2^2$ leads not only to three additional (cheaper) MLMC levels, but also to a diminished computational complexity. Correspondingly, Figure 15b shows the increase in the cost of computing one sample $Q^{(i)}_{M_l,K_l}$ with level l. Here, the red line matches the line of best fit through $\left(M_l, C\left(Q^{(i)}_{M_l,K_l}\right)\right)$ for $l = 0, \ldots, 7$, which has a slope of 0.89 on a log scale. This suggests $\gamma \approx 0.89$, smaller than $\gamma \approx 1.01$ in the previous case, where $K = 2,500$ is constant across all levels. This is a direct consequence of engaging the coarser levels $l = 0, 1, 2$ with $M_l = 2^2, 4^2, 8^2$, respectively. In fact, if we only consider the cost for $l = 4, \ldots, 7$, even with $K_l < K$ KL-modes on each level, the resulting regression line, drawn in green in Figure 15b, has a slope of 1.0, almost equal to the preceding $\gamma \approx 1.01$, as expected.

Finally, Figure 16 illustrates the efficiency of the three surveyed algorithms, namely MC and MLMC with a uniform and level dependent truncation of the KL-expansion of $k(\mathbf{x}, \omega)$, for estimating the mean value of the quantity of interest.

First, Figure 16a highlights that $C_\epsilon\left(\hat{Q}^{\mathrm{MC}}_{M,N}\right) \lesssim \epsilon^{-4.76}$, which is to be expected since $\gamma/\alpha \approx 2.84$. Further, the value -4.76 is established analogously to the prior experiments, specifically by estimating the slope of the regression line through $\left(\epsilon, C_\epsilon\left(\hat{Q}^{\mathrm{MC}}_{M,N}\right)\right)$, where ϵ takes linearly spaced values between $7.5 \cdot 10^{-2}$ and $2.5 \cdot 10^{-3}$.

Second, in Figure 16b we observe in a likewise manner that the MLMC method with $K = 2,500$ KL-modes across all levels has a similar computational complexity to the $\rho = 0.3$ case in Section 5.3.1, namely $C_\epsilon\left(\hat{Q}^{\mathrm{MLMC}}_{M}\right) \lesssim \epsilon^{-2.58}$. This corresponds to the second case in Theorem 3.1, since, in this instance, we recognise that $\beta \approx \gamma$, too.

Third, the performance plot of the MLMC technique with a LD truncation of the KL-expansion of $k(\mathbf{x}, \omega)$ is given in Figure 16c. Here, we notice a slightly smaller slope of the



(a) Performance plot for MC.

(b) Performance plot for MLMC.

(c) Performance plot for LD KL-expansion.

(d) MC vs uniform MLMC vs LD MLMC.

Figure 16: Performance plots on a log scale for $\rho = 0.1$ and $\sigma^2 = 1$ in 2D using Monte Carlo and Multilevel Monte Carlo with a uniform and level dependent truncation of the KL-expansion of $k(\mathbf{x}, \omega)$. The quantity of interest is the mean value of the pressure at $(x, y) = (7/15, 7/15)$.

line of best fit through $\left(\epsilon, C_\epsilon\left(\hat{Q}^{\text{MLMC}}_{M,K}\right)\right)$, particularly we identify $C_\epsilon\left(\hat{Q}^{\text{MLMC}}_{M,K}\right) \lesssim \epsilon^{-2.26}$. As aforementioned, this is a direct consequence of employing three additional MLMC levels, which are coarser, and thus, computationally cheaper. Despite this, we still expect $C_\epsilon\left(\hat{Q}^{\text{MLMC}}_{M,K}\right) \to C_\epsilon\left(\hat{Q}^{\text{MLMC}}_{M}\right)$ as $\epsilon \to 0$, or, equivalently, as $l \to \infty$, where $\hat{Q}^{\text{MLMC}}_{M,K}$ denotes the MLMC estimator with a LD truncation of the KL-expansion of k. The corresponding reason is that, in this case, the dominant contribution to the cost stems from the finest level, where M_L and K_L in the level dependent algorithm are equal to M_L and K in the uniform truncation approach.

The fourth and last plot is provided in Figure 16d, which exposes the efficiency of the three procedures above in relation to one another. In particular, we observe that for very large accuracies, such as $\epsilon = 0.075$ or $\epsilon = 0.05$, MC outperforms both MLMC methods. This is because the former directly applies the number of degrees of freedom M and number of samples N necessary to achieve $e\left(\hat{Q}^{\text{MC}}_{M,N}\right) < \epsilon$, while the latter must employ at least two levels, which, in this example, is clearly more expensive. Additionally, notice the significant computational savings the MLMC approach with a level-dependent KL-expansion of $k(\mathbf{x}, \omega)$ prompts compared to the other two techniques. For instance, to achieve an accuracy of $\epsilon = 2.5 \cdot 10^{-3}$, the standard MC simulation requires approximately $1,965$ seconds, while the MLMC estimator with a uniformly truncated KL-expansion takes roughly 574 seconds. Further, only 20 seconds are necessary here for the MLMC algorithm with a level dependent truncation to achieve a RMSE less than ϵ.

6 Conclusions and Further Work

In this thesis, we have explored the functionality of Monte Carlo and Multilevel Monte Carlo methods in the context of quantifying uncertainty and assessing risk in partial differential equation models. In particular, we focused on uncertainty and risk arising from modelling groundwater flow (which, for instance, carries contaminated substances leaking from radioactive waste containers to the human environment) as a PDE with a random coefficient, namely the hydraulic conductivity. To this end, we adopted a probabilistic approach, so that the uncertainty in the aforementioned PDE parameter is described in terms of statistical quantities, such as probability density functions or covariance functions. In turn, we determined statistical estimates, such as expected values or variances, related to the uncertainty in a functional of the PDE solution.

Within this framework, we showed that Multilevel Monte Carlo simulations can be computationally more efficient than the standard Monte Carlo approach. To gain an understanding of where the cost savings originate from, in Chapters 2 and 3 we discussed the theoretical foundation of the two techniques. We identified that, for the application under consideration, the simple Monte Carlo method can achieve a given accuracy with a cost which is contingent upon two factors. The first aspect is the cost of computing one Monte Carlo sample, which involves generating a sample from the random PDE coefficient, solving the resulting equation and afterwards estimating the desired functional. The second component is the order of convergence in mean of the quantity's of interest approximation to its true value. On the other hand, the Multilevel Monte Carlo approach hinges on estimating the desired functional on a relatively coarse grid, subsequently adding "correction" terms approximated on successively finer meshes. Thus, we avoid evaluating the quantity of interest directly on the finest mesh, prompting significant reductions in computational cost compared to Monte Carlo simulations.

In Chapter 4, we briefly reviewed the two ingredients we need to be able to generate one MC sample. Specifically, we investigated the necessary procedure for determining the variational form of the PDE under consideration, which can, in turn, be approximately solved by FEniCS. Additionally, in Section 4.2 we examined the application of the Karhunen-Loève expansion for sampling from the random field coefficient of the PDE.

The theoretical ideas established in Chapters 2-4 were verified numerically in Chapter 5. Particularly, in Section 5.1 we showed that in the simple 1D case with the ODE coefficient modelled as a random variable, where the exact solution is known, Monte Carlo achieves its best computational cost, independent of the quantity of interest considered. Next, in Sections 5.2 and 5.2 we confirmed that Multilevel Monte Carlo is preferable to simple Monte Carlo, focusing on a 2D example with a random variable and random field coefficient, respectively.

The main addition this report brought to the already vast literature concerning Multilevel Monte Carlo methods is the study of the particular functional considered throughout our numerical experiments in Chapter 5. The predominant quantity of interest we analysed was the point evaluation of the PDE solution. What makes this functional problematic is that it is neither linear, nor bounded in the Sobolev space $H_0^1(D)$. While this makes some of the presented graphs look somewhat erratic, we could still numerically confirm the theoretical results related to the computational complexity of the Monte Carlo and Multilevel Monte Carlo algorithms. Moreover, there are limited papers exploring the level dependent truncation of the KL-expansion of the random field PDE coefficient, especially for the aforementioned functional. This thesis examined the preceding approach in Section 5.3 from a purely numerical perspective and established its benefits in terms of cost savings over the uniformly truncated KL-expansion.

Future work Elaborating on the previous discussion, an immediate next step would be to consider the point evaluation of the PDE solution in 1D. This would indicate how weak the convergence properties for this functional are in more than one dimension. Further, following the level dependent truncation of the KL-expansion, a possible development would be to implement level dependent selection of the covariance function parameters ρ and σ^2. This could allow us to efficiently resolve more realistic models with smaller correlation length or larger variance. Finally, throughout this thesis we adopted simple Monte Carlo in the Multilevel approach. An alternative would be employing a different estimator, such as randomised-Monte Carlo [13, 19].

References

[1] A. Abdulle, A. Barth, and C. Schwab. Multilevel monte carlo methods for stochastic elliptic multiscale pdes. *Society for Industrial and Applied Mathematics*, 11(4):1033–1070, 2013.

[2] A. A. Ali, E. Ullmann, and M. Hinze. Multilevel monte carlo analysis for optimal control of elliptic pdes with random coefficients. *Society for Industrial and Applied Mathematics*, 5(1):466–492, 2017.

[3] D. N. Arnold and A. Logg. Periodic table of the finite elements. *SIAM News, http://www-users.math.umn.edu/~arnold/femtable/*, 47, November 2014. [Online; accessed: 28-July-2021].

[4] A. Barth, C. Schwab, and N. Zollinger. Multi-level monte carlo finite element method for elliptic pdes with stochastic coefficients. *Numerische Mathematik*, 119:123–161, 09 2011.

[5] D. Belomestny, F. Dickmann, and T. Nagapetyan. Pricing bermudan options via multilevel approximation methods. *SIAM Journal on Financial Mathematics*, 6(1):448–466, 2015.

[6] H. Brezis. *Functional Analysis, Sobolev Spaces and Partial Differential Equations*. Springer New York Dordrecht Heidelberg London, 2011.

[7] K. A. Cliffe, M. B. Giles, R. Scheichl, and A. L. Teckentrup. Multilevel monte carlo methods and applications to elliptic pdes with random coefficients. *Computing and Visualization in Science*, 14(3), 2011.

[8] G. De Marsily. Quantitative hydrogeology. *https://www.osti.gov/biblio/6784827*, 1 1986.

[9] J. Delhomme. Spatial variability and uncertainty in groundwater flow parameters: A geostatistical approach. *Water Resources Research*, 15:269–280, 1979.

[10] M. EIERMANN, O. G. ERNST, and E. ULLMANN. Computational aspects of the stochastic finite element method. *Computing and Visualization in Science*, 10:3–15, 2007.

[11] C. Fefferman, J. Robinson, and J. Rodrigo, editors. *Partial Differential Equations in Fluid Mechanics*. London Mathematical Society Lecture Note Series. Cambridge University Press, 2018.

[12] FEniCS. Fenics project. `https://fenicsproject.org`, 2019. [Online; accessed: 15-July-2021].

[13] T. Gerstner and M. Noll. Randomized multilevel quasi-monte carlo path simulation. *Recent Developments in Computational Finance*, pages 349–369, 2013.

[14] R. G. Ghanem and P. D. Spanos. *Stochastic Finite Elements: A Spectral Approach*. Springer, New York, 1991.

[15] R. G. Ghanem and P. D. Spanos. Spectral techniques for stochastic finite elements. *Archives of Computational Methods in Engineering*, 4:63–100, 1997.

[16] M. B. Giles. Multilevel monte carlo path simulation. *Operations Research*, 56(3):607–617, 2008.

[17] M. B. Giles. Multilevel monte carlo methods. In J. Dick, F. Y. Kuo, G. W. Peters, and I. H. Sloan, editors, *Monte Carlo and Quasi-Monte Carlo Methods 2012*, pages 83–103, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.

[18] M. B. Giles. Multilevel monte carlo methods. *Acta Numerica*, 24:259–328, 2015.

[19] I. Graham, F. Kuo, D. Nuyens, R. Scheichl, and I. Sloan. Quasi-monte carlo methods for elliptic pdes with random coefficients and applications. *Journal of Computational Physics*, 230(10):3668–3694, 2011.

[20] I. Graham, F. Kuo, D. Nuyens, R. Scheichl, and I. Sloan. Analysis of circulant embedding methods for sampling stationary random fields. *SIAM Journal on Numerical Analysis (SINUM)*, 56(3):1871–1895, Dec. 2018.

[21] E. Hairer and G. Wanner. A theory for nyström methods. *Numerische Mathematik*, 25:383–400, 1975.

[22] S. Heinrich. Multilevel monte carlo methods. In S. Margenov, J. Waśniewski, and P. Yalamov, editors, *Large-Scale Scientific Computing*, pages 58–67, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.

[23] R. J. Hoeksema and P. K. Kitanidis. Analysis of the spatial structure of properties of selected aquifers. *Water Resources Research*, 21(4):563–572, 1985.

[24] R. Howe. Quantum mechanics and partial differential equations. *Journal of Functional Analysis*, 38(2):188–254, 1980.

[25] L. Hunger, B. Cosenza, S. Kimeswenger, and T. Fahringer. Random fields generation on the gpu with the spectral turning bands method. In F. Silva, I. Dutra, and V. Santos Costa, editors, *Euro-Par 2014 Parallel Processing*, pages 656–667, Cham, 2014. Springer International Publishing.

[26] S. Ji, Y. Xue, and L. Carin. Bayesian compressive sensing. *IEEE Transactions on Signal Processing*, 56(6):2346–2356, 2008.

[27] M. H. Kalos and P. A. Whitlock. *Monte Carlo Methods.* Wiley-VCH, 2nd edition, 2008.

[28] H. Langtangen and A. Logg. *Solving PDEs in Python - The FEniCS Tutorial Volume I.* Springer, 2017.

[29] H. P. Langtangen and K.-A. Mardal. Introduction to numerical methods for variational problems. In *Texts in Computational Science and Engineering*, volume 21. Springer International Publishing, 1st edition, 2019.

[30] M. G. Larson and F. Bengzon. The finite element method: Theory, implementation, and applications. In *Texts in Computational Science and Engineering*, volume 10. Springer-Verlag Berlin Heidelberg, 1st edition, 2013.

[31] G. Marsily, F. Delay, J. Goncalves, P. Renard, V. Teles, and S. Violette. Dealing with spatial heterogeneity. *Hydrogeology Journal*, 13, 03 2005.

[32] NumPy.org. Numpy library. `https://numpy.org`, 2021. [Online; accessed: 02-August-2021].

[33] U. S. D. of Energy. Waste isolation pilot plant. `https://www.wipp.energy.gov`, 2019. [Online; accessed: 13-August-2021].

[34] A. B. Owen. Monte carlo, quasi-monte carlo, and randomized quasi-monte carlo. In H. Niederreiter and J. Spanier, editors, *Monte-Carlo and Quasi-Monte Carlo Methods 1998*, pages 86–97, Berlin, Heidelberg, 2000. Springer Berlin Heidelberg.

[35] A. B. Owen. *Monte Carlo theory, methods and examples*, chapter 1-2. `https://statweb.stanford.edu/~owen/mc/`, 2013.

[36] A. B. Owen. *Monte Carlo theory, methods and examples*, chapter 8. `https://statweb.stanford.edu/~owen/mc/`, 2013.

[37] ParaView. Paraview platform. `https://www.paraview.org`, 2007. [Online; accessed: 16-July-2021].

[38] J.-M. M. Rendu. Normal and lognormal estimation. *Journal of the International Association for Mathematical Geology*, 11:407–422, 1979.

[39] P. Révész. *The Laws of Large Numbers*. New York Academic Press, New York, 1st edition, 1968.

[40] M. Rosin, L. Ricketson, A. Dimits, R. Caflisch, and B. Cohen. Multilevel monte carlo simulation of coulomb collisions. *Journal of Computational Physics*, 274:140–157, 2014.

[41] C. Schwab and R.-A. Todor. Karhunen–loève approximation of random fields by generalized fast multipole methods. *Journal of Computational Physics*, 217:100–122, 09 2006.

[42] B. Schweizer. Darcy's law and groundwater flow modelling. *https://publications.mfo.de/handle/mfo/450*, 01 2015.

[43] Scipy.org. Scipy library. `https://scipy.org`, 2021. [Online; accessed: 30-July-2021].

[44] A. H. Squillacote and D. E. DeMarle. *The ParaView Guide*. Kitware, 2007.

[45] A. Teckentrup, R. Scheichl, M. Giles, and E. Ullmann. Further analysis of multilevel monte carlo methods for elliptic pdes with random coefficients. *Numerische Mathematik*, 125, 04 2012.

[46] A. L. Teckentrup. *Multilevel Monte Carlo methods and uncertainty quantification*. PhD thesis, University of Bath, 6 2013.

[47] N. E. Tovmasyan, G. Zakaryan, and G. L.Z. *Boundary value problems for partial differential equations and applications in electrodynamics*. World Scientific Publishing Co. Pte. Ltd., 1994.

[48] I. C. Tsantili and D. T. Hristopulos. Karhunen–loève expansion of spartan spatial random fields. *Probabilistic Engineering Mechanics*, 43:132–147, 2016.

[49] E. Vanmarcke. *Random Fields - Analysis and Synthesis*. World Scientific, July 2010.

[50] D. Wilson and R. E. Baker. Multi-level methods and approximating distribution functions. *AIP Advances*, 6(7), 2016.

[51] K. J. Worsley, A. C. Evans, S. Marrett, and P. Neelin. A three-dimensional statistical analysis for cbf activation studies in human brain. *Journal of Cerebral Blood Flow & Metabolism*, 12(6):900–918, 1992. PMID: 1400644.

[52] D. Xiu. Stochastic collocation methods: A survey. In R. Ghanem, D. Higdon, and H. Owhadi, editors, *Handbook of Uncertainty Quantification*, pages 1–18, Cham, 2016. Springer International Publishing.

Appendices

A Monte Carlo implementation for 1D example

```python
import numpy as np
import matplotlib.pyplot as plt
from numpy.random import default_rng
import time

def p(x, a, k):
    """
    Computes the exact solution of the ODE -d/dx (k * dp/dx) = a, a != 0.
    :param x: the point at which to compute the solution.
    :param a: the RHS constant.
    :param k: the value of the random variable.
    :return: the value of the solution at the given point x.
    """
    return (a/(2*k))*(x**2)-(1+a/(2*k))*x+1 # the value of the solution at x

def exp_true_fun(x, a, k_mean, k_sigma):
    """
    Computes the exact expectation of the solution of the ODE -d/dx(k*dp/dx)=a.
    :param x: the point at which to compute the expectation.
    :param a: the RHS constant.
    :param k_mean: the mean of the log-normal distribution of k.
    :param k_sigma: the standard deviation of the log-normal distribution of k.
    :return: the expectation of the solution at the given point x.
    """
    exp_k = np.exp(-k_mean + k_sigma**2 / 2) # expectation of 1/k
    return (a/2)*exp_k*(x**2)-(1+(a/2)*exp_k)*x+1 # exact value of E[p(x)]

def var_p_fun(x, a, k_mean, k_sigma):
    """
    Computes the exact variance of the solution of the ODE -d/dx(k*dp/dx)=a.
    :param x: the point at which to compute the variance.
    :param a: the RHS constant.
    :param k_mean: the mean of the log-normal distribution of k.
    :param k_sigma: the standard deviation of the log-normal distribution k.
    :return: the variance of the the solution at the given point x.
    """
    var_k=(np.exp(k_sigma**2)-1)*np.exp(-2*k_mean+k_sigma**2) # variance of 1/k
    return (a/2*x**2-a/2*x)**2*var_k # variance of the solution at x

def k_eff(a, k):
    """
    Computes the quantity of interest -k * dp/dx | x = 1.
    :param a: the RHS constant.
    :param k: the value of the random variable.
    :return: the quantity of interest k_eff_val = -k * dp/dx | x = 1.
    """
    return k - a/2 # the exact value of the quantity of interest

def exp_k_eff_fun(a, k_mean, k_sigma):
    """
    Computes the exact expectation of -k * dp/dx | x = 1.
    :param a: the RHS constant.
    :param k_mean: the mean of the log-normal distribution of k.
    :param k_sigma: the standard deviation of the log-normal distribution of k.
    :return: the exact expectation of the quantity of interest.
    """
    return np.exp(k_mean+k_sigma**2/2)-a/2 # the exact expectation of k_eff

def var_k_eff_fun(k_mean, k_sigma):
    """
    Computes the variance -k * dp/dx | x = 1.
```

```
    :param k_mean: the mean of the log-normal distribution of k.
    :param k_sigma: the standard deviation of the log-normal distribution of k.
    :return: the variance of the quantity of interest.
    """
    return (np.exp(k_sigma**2)-1)*np.exp(2*k_mean+k_sigma**2) # exact variance

def MC_p_simulation(N, x, a, k_mean, k_sigma):
    """
    Performs a standard Monte Carlo simulation for computing E[p(x)].
    :param N: number of samples.
    :param x: point at which to compute E[p(x)]
    :param a: the RHS constant.
    :param k_mean: the mean of the log-normal distribution of k.
    :param k_sigma: the standard deviation of the log-normal distribution of k.
    :return: the root mean squared error between approximation and true value.
    """
    k = rng.lognormal(mean=k_mean, sigma=k_sigma, size=N) # N samples of k
    p_hat = p(x, a, k) # for each k, compute quantity of interest p(x)
    exp_p_est = np.average(p_hat) # Monte Carlo estimate of E[p(x)]
    exp_p_true = exp_true_fun(x, a, k_mean, k_sigma) # true value of E[p(x)]
    disc_error_p = (exp_p_est - exp_p_true) ** 2 # discretisation error
    var_p = var_p_fun(x, a, k_mean, k_sigma) # sample error
    return np.sqrt(1 / N * var_p + disc_error_p) # RMSE of E[p(x)]

def MC_k_eff_simulation(N, a, k_mean, k_sigma):
    """
    Performs a standard Monte Carlo simulation for computing E[k_eff].
    :param N: number of samples.
    :param a: the RHS constant.
    :param k_mean: the mean of the log-normal distribution of k.
    :param k_sigma: the standard deviation of the log-normal distribution of k.
    :return: the root mean squared error between approximation and true value.
    """
    k = rng.lognormal(mean=k_mean, sigma=k_sigma, size=N) # N samples of k
    k_eff_hat = k_eff(a, k) # for each k, compute quantity of interest k_eff
    k_eff_est = np.average(k_eff_hat) # Monte Carlo estimate of k_eff
    k_eff_true = exp_k_eff_fun(a, k_mean, k_sigma) # true value of k_eff
    disc_error_k_eff = (k_eff_est - k_eff_true) ** 2 # discretisation error
    var_k_eff = var_k_eff_fun(k_mean, k_sigma) # sample error
    return np.sqrt(1 / N * var_k_eff + disc_error_k_eff) # RMSE of k_eff

rng = default_rng() # generator for sampling from the distribution of k
x_val = 3 / 4 # the value at which to compute E[p(x)]
a_val = 1 # RHS constant for the ODE
k_mean_val = 0 # the mean of log-normal distribution of k.
k_sigma_val = 1 # the standard deviation of log-normal distribution of k.
R = 100 # number of times to run experiment
epsilons = [0.05, 0.01, 0.005, 0.001, 0.0005] # list of accuracies
# variance values for computing N
var_p_val = var_p_fun(x_val, a_val, k_mean_val, k_sigma_val)
var_k_eff_val = var_k_eff_fun(k_mean_val, k_sigma_val)
rmse_p_vec = np.zeros(R)
time_p_vec = np.zeros(R)
rmse_k_eff_vec = np.zeros(R)
time_k_eff_vec = np.zeros(R)
rmse_p_vec_avg = []
time_p_vec_avg = []
rmse_k_eff_vec_avg = []
time_k_eff_vec_avg = []

for epsilon in epsilons: # run for different values of N
    N_p_val = int(np.ceil(var_p_val * epsilon**(-2))) # N for E[p(3/4)]
    N_k_eff_val = int(np.ceil(var_k_eff_val * epsilon**(-2))) # N for E[k_eff]
    for j in range(R): # average RMSE and time over multiple runs
        t0 = time.time()
        rmse_p_val = \
```

```
            MC_p_simulation(N_p_val, x_val, a_val, k_mean_val, k_sigma_val)
        t1 = time.time()
        rmse_p_vec[j] = rmse_p_val
        time_p_vec[j] = t1 - t0 # time taken for computing E[p(x)]
        t0 = time.time()
        rmse_k_eff_val = \
            MC_k_eff_simulation(N_k_eff_val, a_val, k_mean_val, k_sigma_val)
        t1 = time.time()
        rmse_k_eff_vec[j] = rmse_k_eff_val
        time_k_eff_vec[j] = t1 - t0 # time taken for computing k_eff
    # average results over R runs
    rmse_p_vec_avg.append(np.average(rmse_p_vec))
    time_p_vec_avg.append(np.average(time_p_vec))
    rmse_k_eff_vec_avg.append(np.average(rmse_k_eff_vec))
    time_k_eff_vec_avg.append(np.average(time_k_eff_vec))
```

B Monte Carlo implementation for 2D example with random variable

```
from fenics import *
from numpy.random import default_rng
import matplotlib.pyplot as plt
import numpy as np
import math
import logging
import time

# suppress information messages (printing takes more time)
set_log_active(False)
logging.getLogger('FFC').setLevel(logging.WARNING)

def setup_PDE(mesh_size, pol_degree, a):
    """
    Set up the mesh, function space and boundary conditions for FEM solution of
    the PDE $- \nabla \cdot (k \nabla p) = a$, where a is a real constant.
    :param mesh_size: the stepsize to create FEM grid.
    :param pol_degree: the degree of the polynomial used in FEM approximation.
    :param a: the RHS constant of the PDE.
    :return:
        V - the finite element function space.
        f - the RHS.
        bc - the Dirichlet boundary conditions.
        k - Constant (C++ object) for random coefficient.
    """
    mesh = UnitSquareMesh(mesh_size, mesh_size) # discretisation mesh
    V = FunctionSpace(mesh, 'P', pol_degree) # FEM function space
    # Dirichlet boundary conditions
    high_pressure = 'near(x[0], 0)'
    low_pressure = 'near(x[0], 1)'
    bc_hp = DirichletBC(V, Constant(1), high_pressure)
    bc_lp = DirichletBC(V, Constant(0), low_pressure)
    bc = [bc_hp, bc_lp]
    f = Constant(a) # RHS
    k = Constant(1) # this is for FEniCS - define k as a Constant object
    return V, f, bc, k

def compute_alpha(N, x, y, a, k_mean=0, k_sigma=1):
    """
    Computes alpha in E[Q_M - Q] = C * M ^ (-alpha).
    :param N: the number of samples to be used in the Monte Carlo simulations.
    :param x: the x point at which to compute E[p(x,y)].
    :param y: the y point at which to compute E[p(x,y)].
    :param a: the RHS constant of the PDE.
    :param k_mean (default 0): the mean of the log-normal distribution of k.
    :param k_sigma (default 1): the standard deviation of the distribution of k.
```

```
    :return:
        Ms - different grid sizes used in computing approximations.
        p_hats - ndarray of approximations on different grids (Q_M - Q_{4M}).
        alpha - the slope of the log of the expectations.
        C_alpha - the constant C from E[Q_M - Q] = C * M ^ (-alpha).
    """
    p_hats = np.zeros(6)
    Ms = np.zeros(6)
    for n in range(1, 7):
        p_sol = 0
        # FEM setup for 2 different mesh sizes
        V, f, bc, k_const = \
            setup_PDE(mesh_size=2**n, pol_degree=1, a=a)
        V_2, f_2, bc_2, k_const = \
            setup_PDE(mesh_size=2*(2**n), pol_degree=1, a=a)
        ks = rng.lognormal(mean=k_mean, sigma=k_sigma, size=N) # N samples of k
        for i in range(N): # MC loop - sample k, solve PDE and average results
            k_const.assign(Constant(ks[i]))
            # compute solution for initial mesh-size (M)
            u = Function(V)
            v = TestFunction(V)
            F = k_const * dot(grad(u), grad(v)) * dx - f * v * dx
            solve(F == 0, u, bc)
            # compute solution for finer mesh (4*M)
            u_2 = Function(V_2)
            v_2 = TestFunction(V_2)
            F_2 = k_const * dot(grad(u_2), grad(v_2)) * dx - f_2 * v_2 * dx
            solve(F_2 == 0, u_2, bc_2)
            p_sol += (u(x, y) - u_2(x, y)) # Q_M - Q_4M
        p_hat = p_sol / N # avergage - Monte Carlo estimate
        p_hats[n-1] = np.abs(p_hat)
        Ms[n-1] = 4 ** n
    # compute best linear fit on log scale to find alpha and C
    alpha, C_alpha = np.polyfit(x=np.log(Ms), y=np.log(p_hats), deg=1)
    return Ms, p_hats, -alpha, C_alpha

def compute_gamma(N, a, k_mean=0, k_sigma=1):
    """
    Computes gamma in Cost = C * M ^ gamma.
    :param N: the number of times to run one Monte Carlo simulation.
    :param a: the RHS constant of the PDE.
    :param k_mean (default 0): the mean of the log-normal distribution of k.
    :param k_sigma (default 1): the standard deviation of the distribution of k.
    :return:
        Ms - different grid sizes used in computing approximations.
        avg_times - ndarray of time taken on average for one simulation.
        gamma - the slope on the log-log scale of times.
        C_gamma -  the constant in Cost = C * M^gamma.
    """
    Ms = np.zeros(6)
    avg_times = np.zeros(6)
    for n in range(1, 7):
        times = np.zeros(N)
        ks = rng.lognormal(mean=k_mean, sigma=k_sigma, size=N) # N samples of k
        # FEM setup for current mesh size
        V, f, bc, k_const = setup_PDE(mesh_size=2**n, pol_degree=1, a=a)
        for i in range(N): # MC loop - sample k and solve PDE
            k_const.assign(Constant(ks[i]))
            t0 = time.time()
            # compute solution with current k and mesh size
            u = Function(V)
            v = TestFunction(V)
            F = k_const * dot(grad(u), grad(v)) * dx - f * v * dx
            solve(F == 0, u, bc)
            t1 = time.time()
            times[i] = t1-t0
```

```
        Ms[n-1] = 4 ** n
        avg_times[n-1] = np.average(times)
    # compute gamma and C by finding best linear fit on log-log scale
    gamma, C_gamma = np.polyfit(x=np.log(Ms), y=np.log(avg_times), deg=1)
    return Ms, avg_times, gamma, C_gamma

def var_est_fun(N, x, y, a, mesh_size=16, pol_degree=1, k_mean=0, k_sigma=1):
    '''
    Computes the sample variance V[Q].
    :param N: number of samples to be used.
    :param x: the x value at which to compute E[p(x,y)].
    :param y: the y value at which to compute E[p(x,y)].
    :param a: the RHS constant.
    :param mesh_size (default 16): the stepsize to be used in approximation.
    :param pol_degree (default 1): the degree of the polynomials for FEM.
    :param k_mean (default 0): the mean of the log-normal distribution of k.
    :param k_sigma (default 1): the standard deviation of the distribution of k.
    :return:
        var - the MC estimate of the sample variance.
    '''
    p_sol = np.zeros(N)
    ks = rng.lognormal(mean=k_mean, sigma=k_sigma, size=N) # N samples of k
    V, f, bc, k_const = \
        setup_PDE(mesh_size=mesh_size, pol_degree=pol_degree, a=a) # FEM setup
    for i in range(N):  # MC loop - solve PDE for each k and average results
        k_const.assign(Constant(ks[i]))
        # compute solution with current k and mesh size
        u = Function(V)
        v = TestFunction(V)
        F = k_const * dot(grad(u), grad(v)) * dx - f * v * dx
        solve(F == 0, u, bc)
        p_sol[i] = u(x, y)
    p_hat = np.average(p_sol) # MC estimate
    return 1 / (N-1) * np.sum((p_sol - p_hat) ** 2) # sample variance

def MC_simulation(N, x, y, a, alpha, var, mesh_size, pol_degree=1, k_mean=0,
    k_sigma=1):
    '''
    Performs a standard Monte Carlo simulation for computing E[p(x,y)].
    :param N: the number of samples to use per simulation.
    :param x: the x point at which to compute E[p(x,y)].
    :param y: the y point at which to compute E[p(x,y)].
    :param a: the RHS constant of the PDE.
    :param alpha: constant for computing E[Q_M - Q].
    :param var: the sample variance used in computing RMSE.
    :param mesh_size: the step size to be used for computing FEM approximation.
    :param pol_degree (default 1): degree of polynomial used in FEM.
    :param k_mean (default 0): the mean of the log-normal distribution of k.
    :param k_sigma (default 1): the standard deviation of the distribution of k.
    :return:
        rmse_p - root mean squared error between approximation and exact value.
        time - time taken to compute MC estimate.
    '''
    p_hat = 0
    p_hat_2 = 0
    ks = rng.lognormal(mean=k_mean, sigma=k_sigma, size=N) # N samples of k
    t0 = time.time()
    # FEM setup on two different grids
    V, f, bc, k_const = \
        setup_PDE(mesh_size=mesh_size, pol_degree=pol_degree, a=a)
    V_2, f_2, bc_2, k_const = \
        setup_PDE(mesh_size=2*mesh_size, pol_degree=pol_degree, a=a)
    for k in ks: # Monte Carlo loop
        k_const.assign(Constant(k))
        # Compute solution on current grid with current k
        u = Function(V)
```

```
        v = TestFunction(V)
        F = k_const * dot(grad(u), grad(v)) * dx - f * v * dx
        solve(F == 0, u, bc)
        p_hat += u(x, y)
    t1 = time.time() # time taken to complete simulation
    total_time = t1-t0
    for k in ks: # Monte Carlo loop for computing extrapolation value
        k_const.assign(Constant(k))
        # compute solution on current grid with current k (same k's as before)
        u_2 = Function(V_2)
        v_2 = TestFunction(V_2)
        F_2 = k_const * dot(grad(u_2), grad(v_2)) * dx - f_2 * v_2 * dx
        solve(F_2 == 0, u_2, bc_2)
        p_hat_2 += u_2(x, y)
    exp_est = p_hat/N # MC estimate of E[p(x, y)]
    exp_est_2 = p_hat_2/N # extrapolation value
    disc_error = (exp_est - exp_est_2) / (1 - 4 ** (-alpha))
    rmse = np.sqrt(1 / N * var + disc_error**2) # RMSE of E[p(x,y)]
    return rmse, total_time

rng = default_rng() # generator for sampling from the distribution of k
x_val = 7 / 15 # the x value at which to compute E[p(x,y)]
y_val = 7 / 15 # the y value at which to compute E[p(x,y)]
a_val = 1 # RHS constant for the PDE
k_mean_val = 0 # mean of log-normal distribution of k.
k_sigma_val = 1 # standard deviation of log-normal distribution of k.
pol_degree_val = 1 # polynomial degree for computing FEM approximations
epsilons = [0.1, 0.075, 0.05, 0.025, 0.01, 0.0075, 0.005, 0.0025] # accuracies
N_sim = len(epsilons) # total number of simulations
rmse_vec = np.zeros(N_sim)
time_vec = np.zeros(N_sim)

M_vec, p_hat_vec, alpha_val, C_alpha = \
    compute_alpha(100, x_val, y_val, a_val, k_mean_val, k_sigma_val)
M_vec, avg_times_vec, gamma_val, C_gamma = \
    compute_gamma(100, a_val, k_mean_val, k_sigma_val)
var_val = var_est_fun(10**5, x_val, y_val, a_val, 16, pol_degree_val, k_mean_val
    , k_sigma_val)

for i in range(N_sim): # run MC simulations with different accuracies
    epsilon = epsilons[i]
    N_val = math.ceil(2*epsilon ** (-2) * var_val) # N for current epsilon
    mesh_size_val = math.ceil((math.sqrt(2)*np.abs(C_alpha)/epsilon)**(1/(2*
    alpha_val))) # M for current epsilon
    rmse_val, time_val = MC_simulation(N_val, x_val, y_val, a_val, alpha_val,
    var_val, mesh_size_val, pol_degree_val, k_mean_val, k_sigma_val)
    rmse_vec[i] = rmse_val
    time_vec [i] = time_val
```

C Multilevel Monte Carlo implementation for 2D example with random variable

```
from fenics import *
from numpy.random import default_rng
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import numpy as np
import math
import time
import logging

# suppress information messages (printing takes more time)
set_log_active(False)
logging.getLogger('FFC').setLevel(logging.WARNING)

```

```python
def setup_PDE(mesh_size, pol_degree, a):
    """
    Set up the mesh, function space and boundary conditions for FEM solution of
    the PDE $- \nabla \cdot (k \nabla p) = a$, where a is a real constant.
    :param mesh_size: the stepsize to to create FEM grid.
    :param pol_degree: the degree of the polynomial for FEM approximation.
    :param a: the RHS constant of the PDE.
    :return:
        V - the finite element function space.
        f - the RHS.
        bc - the Dirichlet boundary conditions.
        k - Constant (C++ object) for random coefficient.
    """
    mesh = UnitSquareMesh(mesh_size, mesh_size) # discretisation mesh
    V = FunctionSpace(mesh, 'P', pol_degree) # FEM function space
    # Dirichlet boundary conditions
    high_pressure = 'near(x[0], 0)'
    low_pressure = 'near(x[0], 1)'
    bc_hp = DirichletBC(V, Constant(1), high_pressure)
    bc_lp = DirichletBC(V, Constant(0), low_pressure)
    bc = [bc_hp, bc_lp]
    f = Constant(a) # RHS
    k = Constant(1) # this is for FEniCS - define k as a Constant object
    return V, f, bc, k

def Y_l_est_fun(N, x, y, a, l, m_0, pol_degree=1, k_mean=0, k_sigma=1):
    '''
    Computes the MC estimate of E[Y_l] and V[Y_l] for a given level.
    :param N: number of samples to be used.
    :param x: the x value at which to compute E[p(x,y)].
    :param y: the y value at which to compute E[p(x,y)].
    :param a: the RHS constant.
    :param l: the level for which to compute E and V.
    :param m_0: the mesh size on level zero.
    :param pol_degree (default 1): the degree of the polynomials used in FEM.
    :param k_mean (default 0): the mean of the log-normal distribution of k.
    :param k_sigma (default 1): the standard deviation of the distribution of k.
    :return:
        Y_hat - the expectation E[Y_l].
        Y_hat_var - the variance V[Y_l].
        p_hats - the samples Y_l used to compute approximations.
    '''
    p_hats = np.zeros(N)
    m_l = 2**l*m_0 # mesh size on current level
    # FEM setup for the 2 levels (except for level 0)
    if (l != 0):
        V, f, bc, k_const = setup_PDE(m_l//2, pol_degree, a)
    V_2, f_2, bc_2, k_const = setup_PDE(m_l, pol_degree, a)
    ks = rng.lognormal(mean=k_mean, sigma=k_sigma, size=N) # N samples of k
    for i in range(N): # MC loop - solve PDE for each k and compute average
        k_const.assign(Constant(ks[i]))
        # compute solution on the two different grids (except for level 0)
        if (l != 0):
            u = Function(V)
            v = TestFunction(V)
            F = k_const * dot(grad(u), grad(v)) * dx - f * v * dx
            solve(F == 0, u, bc)
        u_2 = Function(V_2)
        v_2 = TestFunction(V_2)
        F_2 = k_const * dot(grad(u_2), grad(v_2)) * dx - f_2 * v_2 * dx
        solve(F_2 == 0, u_2, bc_2)
        if (l != 0):
            p_hats[i] += (u_2(x, y) - u(x, y))
        else:
            p_hats[i] += u_2(x, y)
    Y_hat = np.average(p_hats)  # MC estimate
```

```
    Y_hat_var = 1 / (N-1) * np.sum((p_hats - Y_hat) ** 2) # MC sample variance
    return Y_hat, Y_hat_var, p_hats

def compute_alpha_beta_gamma(x, y, a, m_0, pol_degree=1, k_mean=0, k_sigma=1):
    '''
    Computes alpha in E[Q_M - Q] = C * M ^ (-alpha), beta in V[Q_M-Q] = C * M ^ (-beta) and gamma in Cost = C * M ^ gamma.
    :param x: the x point at which to compute E[p(x,y)].
    :param y: the y point at which to compute E[p(x,y)].
    :param a: the RHS constant of the PDE.
    :param m_0: the mesh size on level zero.
    :param pol_degree (default 1): the degree of the polynomials used in FEM.
    :param k_mean (default 0): the mean of the log-normal distribution of k.
    :param k_sigma (default 1): the standard deviation of the distribution of k.
    :return:
        Ms - different grid sizes used in computing approximations.
        Y_hats - E[Q_l-Q_{l-1}] for different levels l.
        alpha - slope of Y_hats line on log-log scale.
        C_alpha - constant in E[Q_M - Q] = C * M ^ (-alpha).
        Y_hat_vars - V[Q_l-Q_{l-1}] for different levels l.
        beta - slope of Y_hat_vars line on log-log scale.
        C_beta - contant in V[Q_M-Q] = C * M ^ (-beta).
        Y_ls - samples used to compute alpha.
        avg_times - ndarray of time taken on average for one simulation.
        gamma - the slope on the log-log scale of times.
        C_gamma -  the constant in Cost = C * M^gamma.
    '''
    Y_hats = np.zeros(8)
    Y_hat_vars = np.zeros(8)
    Ms = np.zeros(8)
    Y_ls = []
    avg_times = np.zeros(8)
    Ns = [10000, 1000, 1000, 500, 500, 100, 50, 10] # no of samples per level
    for l in range(8):
        t0 = time.time()
        # for each level, compute expectation and variance
        Y_hat, Y_hat_var, Y_l = \
            Y_l_est_fun(Ns[l], x, y, a, l, m_0, pol_degree, k_mean, k_sigma)
        t1 = time.time()
        Ms[l] = 4 ** l
        Y_hats[l] = np.abs(Y_hat)
        Y_hat_vars[l] = Y_hat_var
        Y_ls.append(Y_l)
        avg_times[l] = (t1-t0)/Ns[l]
    # compute alpha and C_alpha by finding best linear fit through Y_hats
    alpha, C_alpha = np.polyfit(x=np.log(Ms[1:]), y=np.log(Y_hats[1:]), deg=1)
    # compute beta and C_beta by finding best linear fit through Y_hat_vars
    beta, C_beta = np.polyfit(x=np.log(Ms[1:]), y=np.log(Y_hat_vars[1:]), deg=1)
    # compute gamma and C_gamma by finding the best linear fit through avg_times
    gamma, C_gamma = np.polyfit(x=np.log(Ms), y=np.log(avg_times), deg=1)
    return Ms, Y_hats, -alpha, C_alpha, Y_hat_vars, -beta, C_beta, Y_ls, avg_times, gamma, C_gamma

def MLMC_simulation(x, y, a, m_0, epsilon, alpha, gamma, C_gamma, pol_degree=1, k_mean=0, k_sigma=1):
    '''
    Runs one simulation of MLMC for computing E[p(x,y)]. In particular, implements MLMC algorithm in Cliffe (2011).
    :param x: the x value at which to compute E[p(x,y)].
    :param y: the y value at which to compute E[p(x,y)].
    :param a: the RHS constant.
    :param m_0: the mesh size on level zero.
    :param epsilon: the required accuracy.
    :param alpha: the value of alpha required for Richardson extrapolation.
    :param gamma: the slope of the cost required per simulation.
    :param C_gamma: the constant in Cost = C * M ^ gamma.
```

```
    :param pol_degree (default 1): the degree of the polynomials used in FEM.
    :param k_mean (default 0): the mean of the log-normal distribution of k.
    :param k_sigma (default 1): the standard deviation of the distribution of k.
    :return:
        rmse - the final RMSE.
        total_time - time taken for one simulation.
        exp_est - the final expectation estimate.
        N_ls - ndarray with number of samples needed for each level.
    '''
    Ms = np.array([(m_0*(2**l))**2 for l in range(8)]) # deg of freedom per lev
    # the cost per level
    C_ls = np.array([(-C_gamma)*(Ms[l]**gamma) + (-C_gamma)*(Ms[l-1]**gamma) for l in range(1,8)])
    C_ls = np.insert(C_ls, 0, (-C_gamma)*(Ms[0]**gamma))
    Ns = [10000, 1000, 1000, 500, 500, 100, 50, 10] # no of samples per level
    V_ls = np.zeros(8)
    exp_ls = np.zeros(8)
    Y_hats = []
    ##### STEP 1 - Start with L = 0
    L = 0
    total_time_t0 = time.time()
    while(True):
        ##### STEP 2 - Estimate variance using initial number of samples
        Y_hat, Y_hat_var, Y_l = \
            Y_l_est_fun(Ns[L], x, y, a, L, m_0, pol_degree, k_mean, k_sigma)
        V_ls[L] = Y_hat_var
        exp_ls[L] = Y_hat
        Y_hats.append(Y_l)
        ##### STEP 3 - Calculate optimal number of samples
        N_ls = np.array([math.ceil(np.sum(np.sqrt(C_ls[:L+1]*V_ls[:L+1])) * 2 * epsilon ** (-2) * np.sqrt(V_ls[l] / C_ls[l])) for l in range(L+1)])
        ##### STEP 4 - Evaluate extra samples at each level
        for l in range(L+1):
            Y_hat_l = Y_hats[l]
            N_old = len(Y_hat_l)
            if(N_ls[l] > N_old): # if we need more samples
                Y_hat_l = np.resize(Y_hat_l, N_ls[l])
                N_new_samples = N_ls[l] - N_old
                _, _, Y_hat_extra = Y_l_est_fun(N_new_samples, x, y, a, l, pol_degree, k_mean, k_sigma) # compute extra samples needed
                Y_hat_l[N_old:] = Y_hat_extra
                # compute new expectation and variance
                exp_l = np.average(Y_hat_l)
                var_l = 1 / (N_ls[l]-1) * np.sum((Y_hat_l - exp_l) ** 2)
                V_ls[l] = var_l
                exp_ls[l] = exp_l
                Y_hats[l] = Y_hat_l
        ### STEP 5 - Test for convergence if L >= 1
        sample_var = np.sum(1 / N_ls * V_ls[:L+1]) # sample var across all levs
        if (L >= 1): # only test for converges if we have at least 2 levels
            disc_error = (exp_ls[L] / (1-4 ** alpha))**2 # discretisation error
            rmse = np.sqrt(sample_var + disc_error) # RMSE (for refernce)
        ### STEP 6 - If not converged, set L = L+1
        if (L == 0):
            L += 1
        elif (disc_error > epsilon**2/2):
            L += 1
        else:
            break # break the loop once we have converged
    exp_est = np.sum(exp_ls[:L+1]) # MLMC estimate
    total_time_t1 = time.time()
    total_time = total_time_t1-total_time_t0 # total time taken
    return rmse, total_time, exp_est, N_ls

rng = default_rng() # generator for sampling from the distribution of k
x_val = 7 / 15 # the x value at which to compute E[p(x,y)]
```

```
y_val = 7 / 15 # the y value at which to compute E[p(x,y)]
a_val = 1 # RHS constant for the PDE
k_mean_val = 0 # mean of log-normal distribution of k.
k_sigma_val = 1 # standard deviation of log-normal distribution of k.
pol_degree_val = 1 # polynomial degree for computing FEM approximations
m_0_val = 2 # size of mesh on level zero
epsilons = [0.1, 0.075, 0.05, 0.025, 0.01, 0.0075, 0.005, 0.0025, 0.001,
    0.00075, 0.0005, 0.00025] # list of different accuracies
N_sim = len(epsilons) # total number of simulations
rmse_vec = np.zeros(N_sim)
time_vec = np.zeros(N_sim)
exp_vec = np.zeros(N_sim)
N_ls_vec = []

M_vec, Y_hat_vec, alpha_val, C_alpha, Y_hat_var_vec, beta_val, C_beta, Y_ls_vec,
     avg_times_vec, gamma_val, C_gamma = compute_alpha_beta_gamma(x_val, y_val,
    a_val, m_0_val, pol_degree_val, k_mean_val, k_sigma_val)

for i in range(N_sim): # run MLMC simulations with different accuracies
    epsilon = epsilons[i]
    rmse_val, total_time_val, exp_est_val, N_ls_val = \
        MLMC_simulation(x_val, y_val, a_val, m_0_val, epsilon, alpha_val,
    gamma_val, C_gamma, pol_degree_val, k_mean_val, k_sigma_val)
    rmse_vec[i]= rmse_val
    time_vec[i] = total_time_val
    exp_vec[i] = exp_est_val
    N_ls_vec.append(N_ls_val)
```

D Monte Carlo implementation for 2D example with random field

```
from fenics import *
from numpy.random import default_rng
import matplotlib.pyplot as plt
import numpy as np
import math
import time
import logging
from scipy import optimize

# suppress information messages (printing takes more time)
set_log_active(False)
logging.getLogger('FFC').setLevel(logging.WARNING)

def roots_transcendental(rho, M):
    '''
    Finds M roots of the equation tan w = (2*rho*w)/(rho^2*w^2-1) for w.
    :param rho: the correlation length of the covariance function.
    :param M: the number of roots to find.
    :return:
        roots - (ndarray) M roots of the transcendental equation.
    '''
    c = 1/rho # where we look for roots depends on the value of 1/rho
    g = lambda x: (np.tan(x) - (2*c*x)/(x**2-c**2)) # re-write eq as g(w) = 0
    roots = []
    for i in range(M+1): # search for M+1 roots as we discard the null root
        # search in the intervals of the form (k pi/2, (k+1) pi/2)
        wmin = (i - 0.4999) * np.pi
        wmax = (i + 0.4999) * np.pi
        if ((wmin <= c) and (c <= wmax)): # we have 2 roots if 1/rho in interval
            if (wmin > 0): # search towards the left of the interval
                roots.append(optimize.fsolve(g, (c+wmin)/2)[0])
            # always search towards the right of the interval
            roots.append(optimize.fsolve(g, (c+wmax)/2)[0])
        elif (wmin > 0): # we have 1 root if 1/rho not in interval
```

```
            roots.append(optimize.fsolve(g, np.array([wmin, wmax]))[0])
    roots = np.array(roots[:M]) # discard w = 0
    return roots

def eigenfunctions(omega, rho, x):
    '''
    Computes M eigenfunctions of the two-point exponential covariance function using the roots of the transcendetal equation.
    :param omega: the roots of the transcendental equation.
    :param rho: the correlation length of the covariance function.
    :param x: the point at which to compute the eigenfunctions.
    :return:
        b_n - (ndarray) M eigenfunctions of the covariance operator with covariance function above.
    '''
    A_n = 1/(np.sqrt(np.sin(2*omega)*(0.25*(rho**2*omega-1/omega))-(0.5*rho * np.cos(2*omega))+(0.5*(1+rho+rho**2*omega**2)))) # normalising coefficient
    b_n=A_n*(np.sin(x*omega) + rho*omega*np.cos(x*omega)) # eigenfunctions
    return b_n

def setup_randomField(xs, m, m_KL, rho=0.3, sigma=1):
    '''
    Set up the matrix A, so that k = e^(A*xi).
    :param xs: the gridpoints of the mesh of the discretisation.
    :param m: the mesh size used in the discretisation.
    :param m_KL: the number of modes to include in the KL-expansion.
    :param rho (default 0.3): the correlation length of the covariance function.
    :param sigma (default 1): the variance of the covariance function.
    :return:
        A - (ndarray) the matrix A used to find k = e^(A*xi).
    '''
    m_KL_1D = math.ceil(m_KL/2 +1) # number of 1D eigenvalues and eigenfunctions
    omegas = roots_transcendental(rho, M=m_KL_1D) # roots of the trans equation
    theta = sigma*2*rho/(rho**2 * omegas**2 +1) # 1D eigenvalues
    theta = theta.reshape(1, m_KL_1D) # re-shape for matrix multiplication
    theta = np.transpose(theta) @ theta # theta_2D  = theta_1D^T * theta_1D
    theta = np.reshape(theta, (m_KL_1D**2)) # re-shape as row vector
    # the indices of the eigenvalues sorted in descending order
    indices = np.array([i[0] for i in \
        sorted(enumerate(theta), key=lambda k: k[1], reverse=True)]) + 1
    theta[::-1].sort() # sort eigenvalues in descending order
    theta = theta[:m_KL] # only keep largest m_KL eigenvalues
    # indices for selecting 1D eigenfunctions to compute 2D eigenfunctions
    indexi = np.ceil(indices[:m_KL]/m_KL_1D).astype('int')
    indexj = indices[:m_KL] - (indexi-1)*m_KL_1D
    b_ns = np.zeros((m_KL_1D, m+1)) # 1D eigenfunctions at every gridpoint
    for i in range(m+1):
        b_ns[:,i] = eigenfunctions(omegas, rho, xs[i])
    A = np.zeros(((m+1)**2, m_KL))
    for i in range(1,m+2): # build matrix A with A_ij = sqrt(theta_2D) * b_n_2D
        for j in range(1,m+2):
            A[(j-1)*(m+1)+i-1,:] = \
                np.sqrt(theta) * b_ns[indexi-1,i-1] * b_ns[indexj-1,j-1]
    return A

class k_RF(UserExpression):
    '''
    New type of User Expression for selecting the value of the random field at a specific grid point (for building discretisation system).
    :param xi: standard Gaussian random variables.
    :param A: random field matrix.
    :param m_KL: number of modes included in KL-expansion.
    :param m: mesh size used in discretisation.
    :return:
        values[0] - the value of the random field at specific point x[0],x[1]
    '''
```

```python
    def __init__(self, xi, A, m_KL, m, **kwargs):
        super().__init__(**kwargs)
        self.xi = xi
        self.A = A
        self.m = m
        self.m_KL = m_KL
    def eval(self,values,x):
        # select the correct row in the matrix
        eigenf = self.A[int(x[0]*self.m+(self.m+1)*x[1]*self.m),:]
        eigenf = np.reshape(eigenf,(1, self.m_KL)) # shape for multiplication
        k = eigenf @ self.xi # multiply by standard Gaussian RV vector
        values[0] = np.exp(k.item()) # k = e^(A*xi)
    def value_shape(self):
        return()

def setup_PDE(m, pol_degree, a):
    """
    Set up the mesh, function space and boundary conditions for FEM solution of
    the PDE $- \nabla \cdot (k \nabla p) = a$, where a is a real constant.
    :param m: the mesh size to use in x and y direction to create finite
    element grid.
    :param pol_degree: the degree of the polynomial to be used in FEM
    approximation.
    :param a: the RHS constant of the PDE.
    :return:
        V - the finite element function space.
        f - the RHS.
        bc - the Dirichlet boundary conditions.
        mesh - Python object representing the discretisation grid.
    """
    mesh = UnitSquareMesh(m, m) # discretisation mesh
    V = FunctionSpace(mesh, 'P', pol_degree) # FEM function space
    # Dirichlet boundary conditions
    high_pressure = 'near(x[0], 0)'
    low_pressure = 'near(x[0], 1)'
    bc_hp = DirichletBC(V, Constant(1), high_pressure)
    bc_lp = DirichletBC(V, Constant(0), low_pressure)
    bc = [bc_hp, bc_lp]
    f = Constant(a) # RHS
    return V, f, bc, mesh

def compute_alpha(N, m_KL, x, y, a, rho=0.3, sigma=1):
    """
    Computes alpha in E[Q_M - Q] = C * M ^ (-alpha).
    :param N: the number of samples to be used in the Monte Carlo simulations.
    :param m_KL: number of modes to include in KL-expansion of k.
    :param x: the x point at which to compute E[p(x,y)].
    :param y: the y point at which to compute E[p(x,y)].
    :param a: the RHS constant of the PDE.
    :param rho (default 0.3): correlation length of the covariance function.
    :param sigma (default 1): variance of the covariance function.
    :return:
        Ms - different grid sizes used in computing approximations.
        p_hats - ndarray of approximations on different grids (Q_M - Q_{4M}).
        alpha - the slope of the log of the expectations.
        C_alpha - the constant C from E[Q_M - Q] = C * M ^ (-alpha).
    """
    p_hats = np.zeros(6)
    Ms = np.zeros(6)
    for n in range(2, 8):
        p_sol = 0
        # FEM setup for 2 different mesh sizes
        V, f, bc, mesh = setup_PDE(m=2**n, pol_degree=1, a=a)
        mesh_points = mesh.coordinates()
        xs = np.unique(mesh_points)
        A = setup_randomField(xs, 2**n, m_KL, rho, sigma)
```

```
        V_2, f_2, bc_2, mesh_2 = setup_PDE(m=2*(2**n), pol_degree=1, a=a)
        mesh_points_2 = mesh_2.coordinates()
        xs_2 = np.unique(mesh_points_2)
        A_2 = setup_randomField(xs_2, 2*(2**n), m_KL, rho, sigma)
        for i in range(N): # MC loop - sample k, solve PDE and average results
            xi = rng.standard_normal(size=m_KL) # m_KL samples of xi
            # compute solution for initial mesh-size (M)
            k = k_RF(xi, A, m_KL, 2**n)
            u = Function(V)
            v = TestFunction(V)
            F = k * dot(grad(u), grad(v)) * dx - f * v * dx
            solve(F == 0, u, bc)
            # compute solution for finer mesh (4*M)
            k_2 = k_RF(xi, A_2, m_KL, 2*(2**n))
            u_2 = Function(V_2)
            v_2 = TestFunction(V_2)
            F_2 = k_2 * dot(grad(u_2), grad(v_2)) * dx - f_2 * v_2 * dx
            solve(F_2 == 0, u_2, bc_2)
            p_sol += u(x,y) - u_2(x, y) # |Q_M - Q_4M|
        p_hat = p_sol / N  # Monte Carlo estimate
        p_hats[n-2] = np.abs(p_hat)
        Ms[n-2] = 4 ** n
    # compute best linear fit on log scale to find alpha and C
    alpha, C_alpha = np.polyfit(x=np.log(Ms), y=np.log(p_hats), deg=1)
    return Ms, p_hats, -alpha, C_alpha

def compute_gamma(N, m_KL, a, rho=0.3, sigma=1):
    """
    Computes gamma in Cost = C * M ^ gamma.
    :param N: the number of times to run one Monte Carlo simulation.
    :param m_KL: number of modes to include in KL-expansion of random field.
    :param a: the RHS constant of the PDE.
    :param rho (default 0.3): correlation length of the covariance function.
    :param sigma (default 1): variance of the covariance function.
    :return:
        Ms - different grid sizes used in computing approximations.
        avg_times - ndarray of time taken on average for one simulation.
        gamma - the slope on the log-log scale of times.
        C_gamma -  the constant in Cost = C * M^gamma.
    """
    Ms = np.zeros(7)
    avg_times = np.zeros(7)
    for n in range(2, 9):
        times = np.zeros(N)
        # FEM setup for current mesh size
        V, f, bc, mesh = setup_PDE(m=2**n, pol_degree=1, a=a)
        mesh_points = mesh.coordinates()
        xs = np.unique(mesh_points)
        A = setup_randomField(xs, 2**n, m_KL, rho, sigma)
        for i in range(N): # MC loop - sample k and solve PDE
            xi = rng.standard_normal(size=m_KL) # m_KL samples of xi
            t0 = time.time()
            # compute solution for initial mesh-size (M)
            k = k_RF(xi, A, m_KL, 2**n)
            u = Function(V)
            v = TestFunction(V)
            F = k * dot(grad(u), grad(v)) * dx - f * v * dx
            solve(F == 0, u, bc)
            t1 = time.time()
            times[i] = t1-t0
        Ms[n-2] = 4 ** n
        avg_times[n-2] = np.average(times)
    # compute gamma and C by finding best linear fit on log-log scale
    gamma, C_gamma = np.polyfit(x=np.log(Ms), y=np.log(avg_times), deg=1)
    return Ms, avg_times, gamma, C_gamma
```

```python
def var_est_fun(N, m_KL, x, y, a, mesh_size=16, pol_degree=1, rho=0.3, sigma=1):
    '''
    Computes the sample variance V[Q].
    :param N: number of samples to be used.
    :param m_KL: number of modes to be included in KL-expansion of k.
    :param x: the x value at which to compute E[p(x,y)].
    :param y: the y value at which to compute E[p(x,y)].
    :param a: the RHS constant.
    :param mesh_size (default 16): the stepsize to be used in approximation.
    :param pol_degree (defualt 1): the degree of the polynomials used in FEM.
    :param rho (default 0.3): correlation length of the covariance function.
    :param sigma (default 1): variance of the covariance function.
    :return:
        var - the sample variance.
    '''
    p_sol = np.zeros(N)
    V, f, bc, mesh = setup_PDE(mesh_size, pol_degree, a=a) # setup FEM
    mesh_points = mesh.coordinates()
    xs = np.unique(mesh_points)
    A = setup_randomField(xs, mesh_size, m_KL, rho, sigma)
    for i in range(N): # MC loop - solve PDE for each k and average results
        xi = rng.standard_normal(size=m_KL) # m_KL samples of xi
        # compute solution with current k and mesh size
        k = k_RF(xi, A, m_KL, mesh_size)
        u = Function(V)
        v = TestFunction(V)
        F = k * dot(grad(u), grad(v)) * dx - f * v * dx
        solve(F == 0, u, bc)
        p_sol[i] = u(x, y)
    p_hat = np.average(p_sol) # MC estimate
    return 1 / (N-1) * np.sum((p_sol - p_hat) ** 2) # sample variance

def MC_simulation(N, m_KL, x, y, a, alpha, var, mesh_size=8, pol_degree=1,
    rho=0.3, sigma=1):
    '''
    Performs a standard Monte Carlo simulation for computing E[p(x,y)].
    :param N: the number of samples to use per simulation.
    :param m_KL: number of KL modes to include in KL-expansion.
    :param x: the x point at which to compute E[p(x,y)].
    :param y: the y point at which to compute E[p(x,y)].
    :param a: the RHS constant of the PDE.
    :param alpha: constant for computing E[Q_M - Q].
    :param var: the sample variance used in computing RMSE.
    :param mesh_size (default 8): the step size for FEM approximation.
    :param pol_degree (default 1): degree of polynomial to be used in FEM.
    :param rho (default 0.3): correlation length of the covariance function.
    :param sigma (default 1): variance of the covariance function.
    :return:
        rmse - root mean squared error between approximation and exact value.
        total_time - time taken to compute MC estimate.
    '''
    p_hat = 0
    p_hat_2 = 0
    total_time = 0 # time the duration of simulation
    # FEM setup on two different grids
    V, f, bc, mesh = setup_PDE(m=mesh_size, pol_degree=pol_degree, a=a)
    mesh_points = mesh.coordinates()
    xs = np.unique(mesh_points)
    A = setup_randomField(xs, mesh_size, m_KL, rho, sigma)
    V_2, f_2, bc_2, mesh_2=setup_PDE(m=2*mesh_size, pol_degree=pol_degree, a=a)
    mesh_points_2 = mesh_2.coordinates()
    xs_2 = np.unique(mesh_points_2)
    A_2 = setup_randomField(xs_2, 2*mesh_size, m_KL, rho, sigma)
    for i in range(N): # Monte Carlo loop
        xi = rng.standard_normal(size=m_KL) # m_KL samples of xi
        t0 = time.time()
```

```
        # compute solution on current grid with current k
        k = k_RF(xi, A, m_KL, mesh_size)
        u = Function(V)
        v = TestFunction(V)
        F = k * dot(grad(u), grad(v)) * dx - f * v * dx
        solve(F == 0, u, bc)
        p_hat += u(x, y)
        t1 = time.time()
        total_time += t1-t0
        # compute solution for finer mesh (4*M)
        k_2 = k_RF(xi, A_2, m_KL, 2*mesh_size)
        u_2 = Function(V_2)
        v_2 = TestFunction(V_2)
        F_2 = k_2 * dot(grad(u_2), grad(v_2)) * dx - f_2 * v_2 * dx
        solve(F_2 == 0, u_2, bc_2)
        p_hat_2 += u_2(x, y)
    exp_est = p_hat/N # MC estimates of E[p(x, y)]
    exp_est_2 = p_hat_2/N # extrapolation value
    disc_error = (exp_est - exp_est_2) / (1 - 2 ** (-2 * alpha))
    rmse = np.sqrt(1 / N * var + disc_error**2) # RMSE of E[p(x,y)]
    return rmse, total_time

rng = default_rng() # generator for sampling from the distribution of k
x_val = 7 / 15 # the x value at which to compute E[p(x,y)]
y_val = 7 / 15 # the y value at which to compute E[p(x,y)]
a_val = 1 # RHS constant for the PDE
pol_degree_val = 1 # polynomial degree for computing FEM approximations
sigma = 1 # variance of covariance function of k
rho = 0.3 # correlation length of covariance function of k
m_0_val = 4 # size of mesh on level zero
m_KL = 1400 # number of modes to include in KL-expansion of k
epsilons = [0.1, 0.075, 0.05, 0.025, 0.01, 0.0075, 0.005, 0.0025] # accuracies
N_sim = len(epsilons) # total number of simulations
rmse_vec = np.zeros(N_sim)
time_vec = np.zeros(N_sim)

M_vec, p_hat_vec, alpha_val, C_alpha = \
    compute_alpha(100, m_KL, x_val, y_val, a_val, rho, sigma)
M_vec, avg_times_vec, gamma_val, C_gamma = \
    compute_gamma(10, m_KL, a_val, rho, sigma)
var_val = \
    var_est_fun(10**4,m_KL,x_val, y_val, a_val, 32, pol_degree_val, rho, sigma)

for i in range(N_sim): # MC simulations with different accuracies
    epsilon = epsilons[i]
    N_val = math.ceil(2*epsilon ** (-2) * var_val) # N for current epsilon
    mesh_size_val = math.ceil((math.sqrt(2)*np.abs(C_alpha)/epsilon)**(1/(2*
    alpha_val))) # M for current epsilon
    rmse_val, time_val = MC_simulation(N_val, m_KL, x_val, y_val, a_val,
    alpha_val, var_val, mesh_size_val, pol_degree_val, rho, sigma)
    rmse_vec[i] = rmse_val
    time_vec[i] = time_val
```

E Multilevel Monte Carlo implementation for 2D example with random field

```
from fenics import *
from numpy.random import default_rng
import matplotlib.pyplot as plt
import numpy as np
import math
import time
import logging
from scipy import optimize

```

```python
# suppress information messages (printing takes more time)
set_log_active(False)
logging.getLogger('FFC').setLevel(logging.WARNING)

def roots_transcendental(rho, M):
    '''
    Finds M roots of the equation tan w = (2*rho*w)/(rho^2*w^2-1) for w.
    :param rho: the correlation length of the covariance function.
    :param M: the number of roots to find.
    :return:
        roots - (ndarray) M roots of the transcendental equation.
    '''
    c = 1/rho # where we look for roots depends on the value of 1/rho
    g = lambda x: (np.tan(x) - (2*c*x)/(x**2-c**2)) # re-write eq as g(w) = 0
    roots = []
    for i in range(M+1): # search for M+1 roots as we discard the null root
        # search in the intervals of the form (k pi/2, (k+1) pi/2)
        wmin = (i - 0.4999) * np.pi
        wmax = (i + 0.4999) * np.pi
        if ((wmin <= c) and (c <= wmax)): # we have 2 roots if 1/rho in interval
            if (wmin > 0): # search towards the left of the interval
                roots.append(optimize.fsolve(g, (c+wmin)/2)[0])
            # always search towards the right of the interval
            roots.append(optimize.fsolve(g, (c+wmax)/2)[0])
        elif (wmin > 0): # we have 1 root if 1/rho not in interval
            roots.append(optimize.fsolve(g, np.array([wmin, wmax]))[0])
    roots = np.array(roots[:M]) # discard w = 0
    return roots

def eigenfunctions(omega, rho, x):
    '''
    Computes M eigenfunctions of the two-point exponential covariance function using the roots of the transcendetal equation.
    :param omega: the roots of the transcendental equation.
    :param rho: the correlation length of the covariance function.
    :param x: the point at which to compute the eigenfunctions.
    :return:
        b_n - (ndarray) M eigenfunctions of the covariance operator with covariance function above.
    '''
    A_n = 1/(np.sqrt(np.sin(2*omega)*(0.25*(rho**2*omega-1/omega))-(0.5*rho * np.cos(2*omega))+(0.5*(1+rho+rho**2*omega**2)))) # normalising coefficient
    b_n=A_n*(np.sin(x*omega) + rho*omega*np.cos(x*omega)) # eigenfunctions
    return b_n

def setup_randomField(xs, m, m_KL, rho=0.3, sigma=1):
    '''
    Set up the matrix A, so that k = e^(A*xi).
    :param xs: the gridpoints of the mesh of the discretisation.
    :param m: the mesh size used in the discretisation.
    :param m_KL: the number of modes to include in the KL-expansion.
    :param rho (default 0.3): the correlation length of the covariance function.
    :param sigma (default 1): the variance of the covariance function.
    :return:
        A - (ndarray) the matrix A used to find k = e^(A*xi).
    '''
    m_KL_1D = math.ceil(m_KL/2 +1) # number of 1D eigenvalues and eigenfunctions
    omegas = roots_transcendental(rho, M=m_KL_1D) # roots of the trans equation
    theta = sigma*2*rho/(rho**2 * omegas**2 +1) # 1D eigenvalues
    theta = theta.reshape(1, m_KL_1D) # re-shape for matrix multiplication
    theta = np.transpose(theta) @ theta # theta_2D  = theta_1D^T * theta_1D
    theta = np.reshape(theta, (m_KL_1D**2)) # re-shape as row vector
    # the indices of the eigenvalues sorted in descending order
    indices = np.array([i[0] for i in \
        sorted(enumerate(theta), key=lambda k: k[1], reverse=True)]) + 1
    theta[::-1].sort() # sort eigenvalues in descending order
```

```
    theta = theta[:m_KL] # only keep largest m_KL eigenvalues
    # indices for selecting 1D eigenfunctions to compute 2D eigenfunctions
    indexi = np.ceil(indices[:m_KL]/m_KL_1D).astype('int')
    indexj = indices[:m_KL] - (indexi-1)*m_KL_1D
    b_ns = np.zeros((m_KL_1D, m+1)) # 1D eigenfunctions at every gridpoint
    for i in range(m+1):
        b_ns[:,i] = eigenfunctions(omegas, rho, xs[i])
    A = np.zeros(((m+1)**2, m_KL))
    for i in range(1,m+2): # build matrix A with A_ij = sqrt(theta_2D) * b_n_2D
        for j in range(1,m+2):
            A[(j-1)*(m+1)+i-1,:] = \
                np.sqrt(theta) * b_ns[indexi-1,i-1] * b_ns[indexj-1,j-1]
    return A

class k_RF(UserExpression):
    '''
    New type of User Expression for selecting the value of the random field at a specific grid point (for building discretisation system).
    :param xi: standard Gaussian random variables.
    :param A: random field matrix.
    :param m_KL: number of modes included in KL-expansion.
    :param m: mesh size used in discretisation.
    :return:
        values[0] - the value of the random field at specific point x[0],x[1]
    '''
    def __init__(self, xi, A, m_KL, m, **kwargs):
        super().__init__(**kwargs)
        self.xi = xi
        self.A = A
        self.m = m
        self.m_KL = m_KL
    def eval(self,values,x):
        # select the correct row in the matrix
        eigenf = self.A[int(x[0]*self.m+(self.m+1)*x[1]*self.m),:]
        eigenf = np.reshape(eigenf,(1, self.m_KL)) # shape for multiplication
        k = eigenf @ self.xi # multiply by standard Gaussian RV vector
        values[0] = np.exp(k.item()) # k = e^(A*xi)
    def value_shape(self):
        return()

def setup_PDE(m, pol_degree, a):
    """
    Set up the mesh, function space and boundary conditions for FEM solution of
    the PDE $- \nabla \cdot (k \nabla p) = a$, where a is a real constant.
    :param m: the mesh size to use in x and y direction to create finite
    element grid.
    :param pol_degree: the degree of the polynomial to be used in FEM approximation.
    :param a: the RHS constant of the PDE.
    :return:
        V - the finite element function space.
        f - the RHS.
        bc - the Dirichlet boundary conditions.
        mesh - Python object representing the discretisation grid.
    """
    mesh = UnitSquareMesh(m, m) # discretisation mesh
    V = FunctionSpace(mesh, 'P', pol_degree) # FEM function space
    # Dirichlet boundary conditions
    high_pressure = 'near(x[0], 0)'
    low_pressure = 'near(x[0], 1)'
    bc_hp = DirichletBC(V, Constant(1), high_pressure)
    bc_lp = DirichletBC(V, Constant(0), low_pressure)
    bc = [bc_hp, bc_lp]
    f = Constant(a) # RHS
    return V, f, bc, mesh
```

```python
def Y_l_est_fun(N, m_KL, x, y, a, l, m_0, pol_degree=1, rho=0.3, sigma=1):
    '''
    Computes the MC estimate of the expectation E[Y_l] and the variance V[Y_l] for a given level.
    :param N: number of samples to be used.
    :param m_KL: number of modes to be included in KL-expansion.
    :param x: the x value at which to compute E[p(x,y)].
    :param y: the y value at which to compute E[p(x,y)].
    :param a: the RHS constant.
    :param l: the level for which to compute E and V.
    :param m_0: the mesh size on level zero.
    :param pol_degree (default 1): the degree of the polynomials used in FEM approximation.
    :param rho (default 0.3): correlation length of the covariance function.
    :param sigma (default 1): variance of the covariance function.
    :return:
        Y_hat - the expectation E[Y_l].
        Y_hat_var - the variance V[Y_l].
        p_hats - the samples Y_l used to compute approximations.
    '''
    p_hats = np.zeros(N)
    m_l = 2**l*m_0 # mesh size on current level
    # FEM setup for the 2 levels (except for level zero)
    if (l != 0):
        V, f, bc, mesh = setup_PDE(m_l//2, pol_degree, a)
        mesh_points = mesh.coordinates()
        xs = np.unique(mesh_points)
        A = setup_randomField(xs, m_l//2, m_KL, rho, sigma)
    V_2, f_2, bc_2, mesh_2 = setup_PDE(m_l, pol_degree, a)
    mesh_points_2 = mesh_2.coordinates()
    xs_2 = np.unique(mesh_points_2)
    A_2 = setup_randomField(xs_2, m_l, m_KL, rho, sigma)
    for i in range(N): # MC loop - solve PDE for each k and compute average
        xi = rng.standard_normal(size=m_KL) # m_KL samples of xi
        # compute solution on the two different grids (except for level 0)
        if (l != 0):
            k = k_RF(xi, A, m_KL, m_l//2)
            u = Function(V)
            v = TestFunction(V)
            F = k * dot(grad(u), grad(v)) * dx - f * v * dx
            solve(F == 0, u, bc)
        k_2 = k_RF(xi, A_2, m_KL, m_l)
        u_2 = Function(V_2)
        v_2 = TestFunction(V_2)
        F_2 = k_2 * dot(grad(u_2), grad(v_2)) * dx - f_2 * v_2 * dx
        solve(F_2 == 0, u_2, bc_2)
        if (l != 0):
            p_hats[i] += (u_2(x, y) - u(x, y))
        else:
            p_hats[i] += u_2(x, y)
    Y_hat = np.average(p_hats) # MC estimate
    Y_hat_var = 1 / (N-1) * np.sum((p_hats - Y_hat) ** 2) # MC sample variance
    return Y_hat, Y_hat_var, p_hats

def compute_alpha_beta_gamma(m_KL,x,y, a, m_0, pol_degree=1, rho=0.3, sigma=1):
    '''
    Computes alpha in E[Q_M - Q] = C * M ^ (-alpha), beta in V[Q_M-Q] = C * M ^ (-beta) and gamma in Cost = C * M ^ gamma .
    :param m_KL: number of KL modes to be included in KL-expansion.
    :param x: the x point at which to compute E[p(x,y)].
    :param y: the y point at which to compute E[p(x,y)].
    :param a: the RHS constant of the PDE.
    :param m_0: mesh size on level zero.
    :param pol_degree (default 1): the degree of the polynomials used in FEM.
    :param rho (default 0.3): correlation length of the covariance function.
    :param sigma (default 1): variance of the covariance function.
```

```
    :return:
        Ms - different grid sizes used in computing approximations.
        Y_hats - E[Q_l-Q_{l-1}] for different levels l.
        alpha - slope of Y_hats line on log-log scale.
        C_alpha - constant in E[Q_M - Q] = C * M ^ (-alpha).
        Y_hat_vars - V[Q_l-Q_{l-1}] for different levels l.
        beta - slope of Y_hat_vars line on log-log scale.
        C_beta - contant in V[Q_M-Q] = C * M ^ (-beta).
        Y_ls - samples of Q_l - Q_l-1 used in approximation.
        avg_times - ndarray of time taken on average for one simulation.
        gamma - the slope on the log-log scale of times.
        C_gamma -  the constant in Cost = C * M^gamma.
    '''
    Y_hats = np.zeros(7)
    Y_hat_vars = np.zeros(7)
    Ms = np.zeros(7)
    Y_ls = []
    avg_times = np.zeros(7)
    Ns = [10000, 7500, 3500, 500, 150, 75, 50] # no of samples per level
    for l in range(7):
        t0 = time.time()
        # for each level, compute expectation and variance
        Y_hat, Y_hat_var, Y_l =  \
            Y_l_est_fun(Ns[l], m_KL, x, y, a, l, m_0, pol_degree, rho, sigma)
        t1 = time.time()
        Ms[l] = (m_0 * 2**l) ** 2
        Y_hats[l] = np.abs(Y_hat)
        Y_hat_vars[l] = Y_hat_var
        Y_ls.append(Y_l)
        avg_times[l] = (t1-t0)/Ns[l]
    # compute alpha and C_alpha by finding best linear fit through Y_hats
    alpha, C_alpha = np.polyfit(x=np.log(Ms[1:]), y=np.log(Y_hats[1:]), deg=1)
    # compute beta and C_beta by finding best linear fit through Y_hat_vars
    beta, C_beta = np.polyfit(x=np.log(Ms[1:]), y=np.log(Y_hat_vars[1:]), deg=1)
    # compute gamma and C_gamma by finding the best linear fit through avg_times
    gamma, C_gamma = np.polyfit(x=np.log(Ms), y=np.log(avg_times), deg=1)
    return Ms, Y_hats, -alpha, C_alpha, Y_hat_vars, -beta, C_beta, Y_ls,
    avg_times, gamma, C_gamma

def MLMC_simulation(x, y, a, epsilon, alpha, gamma, C_gamma, m_KL, m_0=4,
    pol_degree=1, rho=0.3, sigma=1):
    '''
    Runs one simulation of MLMC for computing E[p(x,y)]. In particular,
    implements MLMC algorithm in Cliffe (2011).
    :param x: the x value at which to compute E[p(x,y)].
    :param y: the y value at which to compute E[p(x,y)].
    :param a: the RHS constant.
    :param epsilon: the required accuracy.
    :param alpha: the value of alpha required for Richardson extrapolation.
    :param gamma: the slope of the cost required per simulation.
    :param C_gamma: the constant in Cost = C * M ^ gamma.
    :param m_KL: number of modes to be included in KL-expansion of k.
    :param m_0 (default 4): the mesh size on the coarsest level.
    :param pol_degree (default 1): the degree of the polynomials used in FEM.
    :param rho (default 0.3): correlation length of the covariance function.
    :param sigma (default 1): variance of the covariance function.
    :return:
        rmse - the final RMSE.
        total_time - time taken for one simulation.
        exp_est - the final expectation estimate.
        N_ls - ndarray with number of samples needed for each level.
    '''
    Ms = np.array([(m_0*(2**l))**2 for l in range(7)]) # deg of freedom per lev
    # the cost per level
    C_ls = np.array([(-C_gamma)*(Ms[l]**gamma) + (-C_gamma)*(Ms[l-1]**gamma) for
     l in range(1,7)])
```

```
    C_ls = np.insert(C_ls, 0, (-C_gamma)*(Ms[0]**gamma))
    Ns = [10000, 7500, 3500, 500, 150, 75, 50] # initial no of samples per lev
    V_ls = np.zeros(7)
    exp_ls = np.zeros(7)
    Y_hats = []
    ##### STEP 1 - Start with L = 0
    L = 0
    total_time_t0 = time.time()
    while(True):
        ##### STEP 2 - Estimate variance using initial number of samples
        Y_hat, Y_hat_var, Y_l = \
            Y_l_est_fun(Ns[L],m_KL, x, y, a, L, m_0, pol_degree, rho, sigma)
        V_ls[L] = Y_hat_var
        exp_ls[L] = Y_hat
        Y_hats.append(Y_l)
        ##### STEP 3 - Calculate optimal number of samples
        N_ls = np.array([math.ceil(np.sum(np.sqrt(C_ls[:L+1]*V_ls[:L+1])) * 2 * epsilon ** (-2) * np.sqrt(V_ls[l]/C_ls[l])) for l in range(L+1)])
        ##### STEP 4 - Evaluate extra samples at each level
        for l in range(L+1):
            Y_hat_l = Y_hats[l]
            N_old = len(Y_hat_l)
            if(N_ls[l] > N_old): # if we need more samples
                Y_hat_l = np.resize(Y_hat_l, N_ls[l])
                N_new_samples = N_ls[l] - N_old
                _, _, Y_hat_extra = Y_l_est_fun(N_new_samples, m_KL, x, y, a, l, m_0, pol_degree, rho, sigma) # extra samples needed
                Y_hat_l[N_old:] = Y_hat_extra
                # new expectation and variance
                exp_l = np.average(Y_hat_l)
                var_l = 1 / (N_ls[l]-1) * np.sum((Y_hat_l - exp_l) ** 2)
                V_ls[l] = var_l
                exp_ls[l] = exp_l
                Y_hats[l] = Y_hat_l
        ### STEP 5 - Test for convergence if L >= 1
        sample_var = np.sum(1 / N_ls * V_ls[:L+1]) # sample var across all levs
        if (L >= 1): # only test for converges if we have at least 2 levels
            disc_error = (exp_ls[L] / (1-4 ** alpha))**2 # discretisation error
            rmse = np.sqrt(sample_var + disc_error) # RMSE (for refernce)
        ### STEP 6 - If not converged, set L = L+1
        if (L == 0):
            L += 1
        elif (disc_error > epsilon**2/2):
            L += 1
        else:
            break # break the loop once we have converged
    exp_est = np.sum(exp_ls[:L+1]) # MLMC expectation estimate
    total_time_t1 = time.time()
    total_time = total_time_t1-total_time_t0 # total time taken
    return rmse, total_time, exp_est, N_ls

rng = default_rng() # generator for sampling from the distribution of k
x_val = 7 / 15 # the x value at which to compute E[p(x,y)]
y_val = 7 / 15 # the y value at which to compute E[p(x,y)]
a_val = 1 # RHS constant for the PDE
pol_degree_val = 1 # polynomial degree for computing FEM approximations
sigma = 1 # variance of covariance function of k
rho = 0.3 # correlation length of covariance function of k
m_0_val = 4 # size of mesh on level zero
m_KL = 1400 # number of modes to include in KL-expansion of k
epsilons = [0.1, 0.075, 0.05, 0.025, 0.01, 0.0075, 0.005, 0.0025, 0.001, 0.00075, 0.0005] # list of accuracies
N_sim = len(epsilons) # total number of simulations
rmse_vec = np.zeros(N_sim)
time_vec = np.zeros(N_sim)
exp_vec = np.zeros(N_sim)
```

```
N_ls_vec = []

M_vec, Y_hat_vec, alpha_val, C_alpha, Y_hat_var_vec, beta_val, C_beta, Y_ls_vec, avg_times_vec, gamma_val, C_gamma = \
    compute_alpha_beta_gamma(m_KL,x_val,y_val, a_val, m_0_val, pol_degree_val, rho, sigma)

for i in range(N_sim): # MLMC simulations for each epsilon
    epsilon = epsilons[i]
    rmse_val, total_time_val, exp_est_val, N_ls_val = \
        MLMC_simulation(x_val, y_val, a_val, epsilon, alpha_val, gamma_val, C_gamma, m_KL, m_0_val, pol_degree_val, rho, sigma)
    rmse_vec[i]= rmse_val
    time_vec[i] = total_time_val
    exp_vec[i] = exp_est_val
    N_ls_vec.append(N_ls_val)
```

F Multilevel Monte Carlo implementation for 2D example with level dependent truncation of random field KL-expansion

```
from fenics import *
from numpy.random import default_rng
import matplotlib.pyplot as plt
import numpy as np
import math
import time
import logging
from scipy import optimize

# suppress information messages (printing takes more time)
set_log_active(False)
logging.getLogger('FFC').setLevel(logging.WARNING)

def roots_transcendental(rho, M):
    '''
    Finds M roots of the equation tan w = (2*rho*w)/(rho^2*w^2-1) for w.
    :param rho: the correlation length of the covariance function.
    :param M: the number of roots to find.
    :return:
        roots - (ndarray) M roots of the transcendental equation.
    '''
    c = 1/rho # where we look for roots depends on the value of 1/rho
    g = lambda x: (np.tan(x) - (2*c*x)/(x**2-c**2)) # re-write eq as g(w) = 0
    roots = []
    for i in range(M+1): # search for M+1 roots as we discard the null root
        # search in the intervals of the form (k pi/2, (k+1) pi/2)
        wmin = (i - 0.4999) * np.pi
        wmax = (i + 0.4999) * np.pi
        if ((wmin <= c) and (c <= wmax)): # we have 2 roots if 1/rho in interval
            if (wmin > 0): # search towards the left of the interval
                roots.append(optimize.fsolve(g, (c+wmin)/2)[0])
            # always search towards the right of the interval
            roots.append(optimize.fsolve(g, (c+wmax)/2)[0])
        elif (wmin > 0): # we have 1 root if 1/rho not in interval
            roots.append(optimize.fsolve(g, np.array([wmin, wmax]))[0])
    roots = np.array(roots[:M]) # discard w = 0
    return roots

def eigenfunctions(omega, rho, x):
    '''
    Computes M eigenfunctions of the two-point exponential covariance function using the roots of the transcendetal equation.
    :param omega: the roots of the transcendental equation.
    :param rho: the correlation length of the covariance function.
    :param x: the point at which to compute the eigenfunctions.
```

```
    :return:
        b_n - (ndarray) M eigenfunctions of the covariance operator with covariance function above.
    '''
    A_n = 1/(np.sqrt(np.sin(2*omega)*(0.25*(rho**2*omega-1/omega))-(0.5*rho * np.cos(2*omega))+(0.5*(1+rho+rho**2*omega**2)))) # normalising coefficient
    b_n=A_n*(np.sin(x*omega) + rho*omega*np.cos(x*omega)) # eigenfunctions
    return b_n

def setup_randomField(xs, m, m_KL, rho=0.3, sigma=1):
    '''
    Set up the matrix A, so that k = e^(A*xi).
    :param xs: the gridpoints of the mesh of the discretisation.
    :param m: the mesh size used in the discretisation.
    :param m_KL: the number of modes to include in the KL-expansion.
    :param rho (default 0.3): the correlation length of the covariance function.
    :param sigma (default 1): the variance of the covariance function.
    :return:
        A - (ndarray) the matrix A used to find k = e^(A*xi).
    '''
    m_KL_1D = math.ceil(m_KL/2 +1) # number of 1D eigenvalues and eigenfunctions
    omegas = roots_transcendental(rho, M=m_KL_1D) # roots of the trans equation
    theta = sigma*2*rho/(rho**2 * omegas**2 +1) # 1D eigenvalues
    theta = theta.reshape(1, m_KL_1D) # re-shape for matrix multiplication
    theta = np.transpose(theta) @ theta # theta_2D  = theta_1D^T * theta_1D
    theta = np.reshape(theta, (m_KL_1D**2)) # re-shape as row vector
    # the indices of the eigenvalues sorted in descending order
    indices = np.array([i[0] for i in \
        sorted(enumerate(theta), key=lambda k: k[1], reverse=True)]) + 1
    theta[::-1].sort() # sort eigenvalues in descending order
    theta = theta[:m_KL] # only keep largest m_KL eigenvalues
    # indices for selecting 1D eigenfunctions to compute 2D eigenfunctions
    indexi = np.ceil(indices[:m_KL]/m_KL_1D).astype('int')
    indexj = indices[:m_KL] - (indexi-1)*m_KL_1D
    b_ns = np.zeros((m_KL_1D, m+1)) # 1D eigenfunctions at every gridpoint
    for i in range(m+1):
        b_ns[:,i] = eigenfunctions(omegas, rho, xs[i])
    A = np.zeros(((m+1)**2, m_KL))
    for i in range(1,m+2): # build matrix A with A_ij = sqrt(theta_2D) * b_n_2D
        for j in range(1,m+2):
            A[(j-1)*(m+1)+i-1,:] = \
                np.sqrt(theta) * b_ns[indexi-1,i-1] * b_ns[indexj-1,j-1]
    return A

class k_RF(UserExpression):
    '''
    New type of User Expression for selecting the value of the random field at a specific grid point (for building discretisation system).
    :param xi: standard Gaussian random variables.
    :param A: random field matrix.
    :param m_KL: number of modes included in KL-expansion.
    :param m: mesh size used in discretisation.
    :return:
        values[0] - the value of the random field at specific point x[0],x[1]
    '''
    def __init__(self, xi, A, m_KL, m, **kwargs):
        super().__init__(**kwargs)
        self.xi = xi
        self.A = A
        self.m = m
        self.m_KL = m_KL
    def eval(self,values,x):
        # select the correct row in the matrix
        eigenf = self.A[int(x[0]*self.m+(self.m+1)*x[1]*self.m),:]
        eigenf = np.reshape(eigenf,(1, self.m_KL)) # shape for multiplication
        k = eigenf @ self.xi # multiply by standard Gaussian RV vector
```

```
        values[0] = np.exp(k.item()) # k = e^(A*xi)
    def value_shape(self):
        return()

def setup_PDE(m, pol_degree, a):
    """
    Set up the mesh, function space and boundary conditions for FEM solution of
    the PDE $- \nabla \cdot (k \nabla p) = a$, where a is a real constant.
    :param m: the mesh size to use in x and y direction to create finite
    element grid.
    :param pol_degree: the degree of the polynomial to be used in FEM
    approximation.
    :param a: the RHS constant of the PDE.
    :return:
        V - the finite element function space.
        f - the RHS.
        bc - the Dirichlet boundary conditions.
        mesh - Python object representing the discretisation grid.
    """
    mesh = UnitSquareMesh(m, m) # discretisation mesh
    V = FunctionSpace(mesh, 'P', pol_degree) # FEM function space
    # Dirichlet boundary conditions
    high_pressure = 'near(x[0], 0)'
    low_pressure = 'near(x[0], 1)'
    bc_hp = DirichletBC(V, Constant(1), high_pressure)
    bc_lp = DirichletBC(V, Constant(0), low_pressure)
    bc = [bc_hp, bc_lp]
    f = Constant(a) # RHS
    return V, f, bc, mesh

def Y_l_est_fun(N, m_KL, x, y, a, l, m_0, pol_degree=1, rho=0.3, sigma=1):
    '''
    Computes the MC estimate of E[Y_l] and V[Y_l] for a given level.
    :param N: number of samples to be used.
    :param m_KL: number of modes to be included in KL-expansion.
    :param x: the x value at which to compute E[p(x,y)].
    :param y: the y value at which to compute E[p(x,y)].
    :param a: the RHS constant.
    :param l: the level for which to compute E and V.
    :param pol_degree (default 1): the degree of the polynomials used in FEM.
    :param rho (default 0.3): correlation length of the covariance function.
    :param sigma (default 1): variance of the covariance function.
    :return:
        Y_hat - the expectation E[Y_l].
        Y_hat_var - the variance V[Y_l].
        p_hats - the samples Y_l used to compute approximations.
    '''
    p_hats = np.zeros(N)
    m_l = 2**l*m_0 # mesh size on current level
    # FEM setup for the 2 levels (except for level zero)
    if (l != 0):
        V, f, bc, mesh = setup_PDE(m_l//2, pol_degree, a)
        mesh_points = mesh.coordinates()
        xs = np.unique(mesh_points)
        A = setup_randomField(xs, m_l//2, m_KL, rho, sigma)
    V_2, f_2, bc_2, mesh_2 = setup_PDE(m_l, pol_degree, a)
    mesh_points_2 = mesh_2.coordinates()
    xs_2 = np.unique(mesh_points_2)
    A_2 = setup_randomField(xs_2, m_l, m_KL, rho, sigma)
    for i in range(N): # MC loop - solve PDE for each k and compute average
        xi = rng.standard_normal(size=m_KL) # m_KL samples of xi
        # compute solution on the two different grids (except for level 0)
        if (l != 0):
            k = k_RF(xi, A, m_KL, m_l//2)
            u = Function(V)
            v = TestFunction(V)
```

```
            F = k * dot(grad(u), grad(v)) * dx - f * v * dx
            solve(F == 0, u, bc)
        k_2 = k_RF(xi, A_2, m_KL, m_l)
        u_2 = Function(V_2)
        v_2 = TestFunction(V_2)
        F_2 = k_2 * dot(grad(u_2), grad(v_2)) * dx - f_2 * v_2 * dx
        solve(F_2 == 0, u_2, bc_2)
        if (l != 0):
            p_hats[i] += (u_2(x, y) - u(x, y))
        else:
            p_hats[i] += u_2(x, y)
    Y_hat = np.average(p_hats) # MC estimate
    Y_hat_var = 1 / (N-1) * np.sum((p_hats - Y_hat) ** 2) # MC sample variance
    return Y_hat, Y_hat_var, p_hats

def compute_delta(N, x, y, a, pol_degree=1, rho=0.3, sigma=1):
    '''
    Computes delta in E[Q_l - Q_l-1] = C_1 * M ^ (-alpha) + C_2 * K^ (-delta).
    :param N: the number of samples to be used in the Monte Carlo simulations.
    :param x: the x point at which to compute E[p(x,y)].
    :param y: the y point at which to compute E[p(x,y)].
    :param a: the RHS constant of the PDE.
    :param pol_degree (default 1): the degree of the polynomials used in FEM .
    :param rho (default 0.3): correlation length of the covariance function.
    :param sigma (default 1): variance of the covariance function.
    :return:
        m_KLs: list of truncation values used in different approximations.
        Y_hats: list of approximation on different levels.
        delta: the values of delta E[Q_l-Q_l-1]=C_1*M^(-alpha)+C_2*K^(-delta).
        C_delta: the value of C_2 in E[Q_l-Q_l-1]=C_1*M^(-alpha)+C_2*K^(-delta).
    '''
    Y_hats = np.zeros(6)
    m_KLs = np.array([2**l for l in range (4,11)]) # number of modes per level
    for l in range(6):
        p_hats = np.zeros(N)
        # set up mesh and matrix A for two different K_ls
        V, f, bc, mesh = setup_PDE(128, pol_degree, a)
        mesh_points = mesh.coordinates()
        xs = np.unique(mesh_points)
        A = setup_randomField(xs, 128, m_KLs[l], rho, sigma)
        V_2, f_2, bc_2, mesh_2 = setup_PDE(128, pol_degree, a)
        mesh_points_2 = mesh_2.coordinates()
        xs_2 = np.unique(mesh_points_2)
        A_2 = setup_randomField(xs_2, 128, m_KLs[l+1], rho, sigma)
        for i in range(N):
            xi = rng.standard_normal(size=m_KLs[l+1]) # m_KL samples of xi
            # compute solution on the same grid for two different Kls
            k = k_RF(xi[:m_KLs[l]], A, m_KLs[l], 128)
            u = Function(V)
            v = TestFunction(V)
            F = k * dot(grad(u), grad(v)) * dx - f * v * dx
            solve(F == 0, u, bc)
            k_2 = k_RF(xi, A_2, m_KLs[l+1], 128)
            u_2 = Function(V_2)
            v_2 = TestFunction(V_2)
            F_2 = k_2 * dot(grad(u_2), grad(v_2)) * dx - f_2 * v_2 * dx
            solve(F_2 == 0, u_2, bc_2)
            p_hats[i] += (u_2(x,y) - u(x,y))
        Y_hats[l] = np.abs(np.average(p_hats)) # MC estimate
    # compute alpha and C_alpha by finding best linear fit through Y_hats
    delta, C_delta = np.polyfit(x=np.log(m_KLs[1:]), y=np.log(Y_hats), deg=1)
    return m_KLs, Y_hats, -delta, C_delta

def compute_alpha_beta(m_KL, x, y, a, m_0, pol_degree=1, rho=0.3, sigma=1):
    '''
    Computes alpha in E[Q_M-Q]=C*M^(-alpha) and beta in V[Q_M-Q]=C*M^(-beta).
```

```
    :param m_KL: the number of modes to include in KL-expansion.
    :param x: the x point at which to compute E[p(x,y)].
    :param y: the y point at which to compute E[p(x,y)].
    :param a: the RHS constant of the PDE.
    :param pol_degree (default 1): the degree of the polynomials used in FEM.
    :param rho (default 0.3): correlation length of the covariance function.
    :param sigma (default 1): variance of the covariance function.
    :return:
        Ms - different grid sizes used in computing approximations.
        Y_hats - E[Q_l-Q_{l-1}] for different levels l.
        alpha - slope of Y_hats line on log-log scale.
        C_alpha - constant in E[Q_M - Q] = C * M ^ (-alpha).
        Y_hat_vars - V[Q_l-Q_{l-1}] for different levels l.
        beta - slope of Y_hat_vars line on log-log scale.
        C_beta - contant in V[Q_M-Q] = C * M ^ (-beta).
        Y_ls - samples used to estimate alpha.
    '''
    Y_hats = np.zeros(7)
    Y_hat_vars = np.zeros(7)
    Ms = np.zeros(7)
    Y_ls = []
    Ns = [10000, 7500, 3500, 500, 150, 75, 50] # no of samples per level
    for l in range(7):
        # for each level, compute expectation and variance
        Y_hat, Y_hat_var, Y_l = \
            Y_l_est_fun(Ns[l], m_KL, x, y, a, l, m_0, pol_degree, rho, sigma)
        Ms[l] = (m_0 * 2**l) ** 2
        Y_hats[l] = np.abs(Y_hat)
        Y_hat_vars[l] = Y_hat_var
        Y_ls.append(Y_l)
    # compute alpha and C_alpha by finding best linear fit through Y_hats
    alpha, C_alpha = np.polyfit(x=np.log(Ms[1:]), y=np.log(Y_hats[1:]), deg=1)
    # compute beta and C_beta by finding best linear fit through Y_hat_vars
    beta, C_beta = np.polyfit(x=np.log(Ms[1:]), y=np.log(Y_hat_vars[1:]), deg=1)
    return Ms, Y_hats, -alpha, C_alpha, Y_hat_vars, -beta, C_beta, Y_ls

def compute_gamma(N, a, m_0, alpha, C_alpha, delta, C_delta, pol_degree=1, rho
    =0.3, sigma=1):
    '''
    Computes gamma in Cost = C * M ^ gamma.
    :param N: the number of simulations.
    :param a: the RHS constant of the PDE.
    :param m_0: the mesh size on level zero.
    :param alpha: convergence rate in mean of approximation.
    :param C_alpha: constant corresponding to above convergence rate.
    :param delta: convergence rate in mean of KL-expansion.
    :param C_delta: constant corresponding to above convergence rate.
    :param pol_degree (default 1): the degree of the polynomials used in FEM.
    :param rho (default 0.3): correlation length of the covariance function.
    :param sigma (default 1): variance of the covariance function.
    :return:
        Ms - different grid sizes used in computing approximations.
        avg_times - ndarray of time taken on average for one simulation.
        gamma - the slope on the log-log scale of times.
        C_gamma -  the constant in Cost = C * M^gamma.
    '''
    avg_times = np.zeros(7)
    m_ls = np.array([m_0 * 2**l for l in range(7)]) # mesh size per level
    Ms = np.array([m_ls[l] ** 2 for l in range(7)]) # deg of freedom per level
    m_KLs = np.array([int(np.ceil(C_alpha/C_delta * Ms[l]**(alpha/delta))) for l
     in range(7)]) # number of KL modes per level
    for l in range(7):
        times = np.zeros(N)
        # FEM and RF setup
        V, f, bc, mesh = setup_PDE(m_ls[l], pol_degree, a)
        mesh_points = mesh.coordinates()
```

```
        xs = np.unique(mesh_points)
        A = setup_randomField(xs, m_ls[l], m_KLs[l], rho, sigma)
        for i in range(N):
            t0 = time.time()
            xi = rng.standard_normal(size=m_KLs[l]) # m_KL samples of xi
            # compute solution for current sample
            k = k_RF(xi, A, m_KLs[l], m_ls[l])
            u = Function(V)
            v = TestFunction(V)
            F = k * dot(grad(u), grad(v)) * dx - f * v * dx
            solve(F == 0, u, bc)
            t1 = time.time()
            times[i] = t1-t0
        avg_times[l] = np.average(times)
    # compute gamma and C_gamma by finding the best linear fit through avg_times
    gamma, C_gamma = np.polyfit(x=np.log(Ms), y=np.log(avg_times), deg=1)
    return Ms, avg_times, gamma, C_gamma

def MLMC_simulation(x, y, a, epsilon, alpha, C_alpha, gamma, C_gamma, time_per_sim, delta, C_delta, m_0=4, pol_degree=1, rho=0.3, sigma=1):
    '''
    Runs one simulation of MLMC for computing E[p(x,y)]. In particular, implements MLMC algorithm in Cliffe (2011).
    :param x: the x value at which to compute E[p(x,y)].
    :param y: the y value at which to compute E[p(x,y)].
    :param a: the RHS constant.
    :param epsilon: the required accuracy.
    :param alpha: the value of alpha required for Richardson extrapolation.
    :param gamma: the slope of the cost required per simulation.
    :param C_gamma: the constant in Cost = C * M ^ gamma.
    :param m_0 (default 2): the mesh size on the coarsest level.
    :param pol_degree (default 1): the degree of the polynomials used in FEM.
    :param rho (default 0.3): correlation length of the covariance function.
    :param sigma (default 1): variance of the covariance function.
    :return:
        rmse - the final RMSE.
        total_time - time taken for one simulation.
        exp_est - the final expectation estimate.
        N_ls - ndarray with number of samples needed for each level.
    '''
    Ms = np.array([(m_0*(2**l))**2 for l in range(7)]) # deg of freedom per lev
    m_KLs = np.array([int(np.ceil(C_alpha/C_delta * Ms[l]**(alpha/delta))) for l in range(7)]) # number of KL modes per level
    # the cost per level
    C_ls = np.array([(-C_gamma)*(Ms[l]**gamma) + (-C_gamma)*(Ms[l-1]**gamma) for l in range(1,7)])
    C_ls = np.insert(C_ls, 0, (-C_gamma)*(Ms[0]**gamma))
    Ns = [10000, 7500, 3500, 500, 150, 75, 50] # no of samples per level
    V_ls = np.zeros(7)
    exp_ls = np.zeros(7)
    Y_hats = []
    ##### STEP 1 - Start with L = 0
    L = 0
    total_time_t0 = time.time()
    while(True):
        ##### STEP 2 - Estimate variance using initial number of samples
        Y_hat, Y_hat_var, Y_l = \
            Y_l_est_fun(Ns[L],m_KLs[L],x, y, a, L, m_0, pol_degree, rho, sigma)
        V_ls[L] = Y_hat_var
        exp_ls[L] = Y_hat
        Y_hats.append(Y_l)
        ##### STEP 3 - Calculate optimal number of samples
        N_ls = np.array([math.ceil(np.sum(np.sqrt(C_ls[:L+1]*V_ls[:L+1])) * 2 * epsilon ** (-2) * np.sqrt(V_ls[l]/C_ls[l])) for l in range(L+1)])
        ##### STEP 4 - Evaluate extra samples at each level
        for l in range(L+1):
```

```
            Y_hat_l = Y_hats[l]
            N_old = len(Y_hat_l)
            if(N_ls[l] > N_old): # if we need more samples
                Y_hat_l = np.resize(Y_hat_l, N_ls[l])
                N_new_samples = N_ls[l] - N_old
                _, _, Y_hat_extra = Y_l_est_fun(N_new_samples, m_KLs[l], x, y, a, l, m_0, pol_degree, rho, sigma) # compute extra samples
                Y_hat_l[N_old:] = Y_hat_extra
                # new expectation and variance
                exp_l = np.average(Y_hat_l)
                var_l = 1 / (N_ls[l]-1) * np.sum((Y_hat_l - exp_l) ** 2)
                V_ls[l] = var_l
                exp_ls[l] = exp_l
                Y_hats[l] = Y_hat_l
        ### STEP 5 - Test for convergence if L >= 1
        sample_var = np.sum(1 / N_ls * V_ls[:L+1]) # sample var across all levs
        if (L >= 1): # only test for converges if we have at least 2 levels
            disc_error = (exp_ls[L] / (1-4 ** alpha))**2 # discretisation error
            rmse = np.sqrt(sample_var + disc_error) # RMSE (for refernce)
        ### STEP 6 - If not converged, set L = L+1
        if (L == 0):
            L += 1
        elif (disc_error > epsilon**2/2):
            L += 1
        else:
            break # break the loop once we have converged
    exp_est = np.sum(exp_ls[:L+1]) # MLMC expectation estimate
    total_time_t1 = time.time()
    total_time = total_time_t1-total_time_t0 # total time taken
    return rmse, total_time, exp_est, N_ls, exp_ls, V_ls, Y_hats

rng = default_rng() # generator for sampling from the distribution of k
x_val = 7 / 15 # the x value at which to compute E[p(x,y)]
y_val = 7 / 15 # the y value at which to compute E[p(x,y)]
a_val = 1 # RHS constant for the PDE
pol_degree_val = 1 # polynomial degree for computing FEM approximations
sigma = 1 # variance of covariance function of k
rho = 0.3 # correlation length of covariance function of k
m_0_val = 4 # size of mesh on level zero
epsilons = [0.1, 0.075, 0.05, 0.025, 0.01, 0.0075, 0.005, 0.0025, 0.001, 0.00075, 0.0005, 0.00025] # list of different accuracies
N_sim = len(epsilons) # total number of simulations
rmse_vec = np.zeros(N_sim)
time_vec = np.zeros(N_sim)
exp_vec = np.zeros(N_sim)
N_ls_vec = []

m_KLs, Y_hat_vec_delta, delta_val, C_delta = \
    compute_delta(150, x_val, y_val, a_val, m_0_val, pol_degree_val, rho, sigma)
M_vec,Y_hat_vec,alpha_val,C_alpha,Y_hat_var_vec,beta_val,C_beta,Y_ls_vec = \
    compute_alpha_beta(1400,x_val,y_val,a_val,m_0_val,pol_degree_val,rho,sigma)
M_vec, avg_times_vec, gamma_val, C_gamma = \
    compute_gamma(10, a_val, m_0_val, alpha_val, C_alpha, delta_val, C_delta, pol_degree_val, rho, sigma)

for i in range(N_sim): # MLMC simulations for each accuracy
    epsilon = epsilons[i]
    rmse_val, total_time_val, exp_est_val, N_ls_val = \
        MLMC_simulation(x_val, y_val, a_val, epsilon,alpha_val, gamma_val, C_gamma, C_delta, m_0_val, pol_degree_val, rho, sigma)
    rmse_vec[i]= rmse_val
    time_vec[i] = total_time_val
    exp_vec[i] = exp_est_val
    N_ls_vec.append(N_ls_val)
```